

Unified Platform of the Engineering and Works Administration of the Royal Saudi Land Forces

Technical and Functional Documentation, Operations and User Guide

Version 1.0.0

5 September 2026

Document status: Approved final document; sensitive values, secrets, and connection details are not included.

Version History

Table 1: Unified Platform of the Engineering and Works Administration of the Royal Saudi Land Forces Documentation Version History

Change Scope	Status	Date	Version
Documentation of the phases through IncomeSystem and Live Reconciliation 7A	Interim	2026-08-22	0.1
Consolidation of phases 1–14, live DATACORE, the final Coverage Audit, and the Word and PDF versions	Finally Verified	2026-08-23	1.0
Final security verification status, secure coding, components and licenses, SBOM, hosting, backup and recovery, and authentication and session architecture	Updated for Release	2026-09-05	1.0.0

Documentation Scope

Table 2: Included and Deferred Programs and Surfaces

Programs	Classification
ControlPanel, Housing, IncomeSystem, ElectronicBillSystem, Home, Login	Included
HousingCommandCenter, Maintenance, Support, Vehicle, Dashboard, Statistics, PopulationDensity, RealCharts	Deferred
Api, CrudController, ExportsController, NotificationsController, ReportsController, SessionController, SmartComponentController, Shared Views	Shared Surfaces

Final SQL source: The live DATACORE database is the authoritative source for SQL definitions. The SmartFoundation.Database Snapshot is for comparison only and must not be used to revive a claim corrected through live reconciliation.

Table of Contents

Static table of contents updated against the final rendering

Page	Chapter	Page	Chapter
5	2. Architecture	4	1. Executive Summary
18	4. Security and Permissions	14	3. Solution Structure and Scope
30	6. Housing	23	5. ControlPanel
51	8. ElectronicBillSystem	47	7. IncomeSystem
60	10. Live Database	56	9. Home and Login
69	12. Workflows and Diagrams	68	11. Schema Drift
84	14. Development and Deployment Environment	75	13. Reports and Integrations
86	16. Troubleshooting	85	15. Backup and Recovery
93	18. User Guide	88	17. Developer Guide
108	20. Final Coverage Audit	100	19. Gap and Risk Register
116	22. Terms and Abbreviations	111	21. Release 1.0.0 Security and Operational Status

1. Executive Summary

Unified Platform of the Engineering and Works Administration of the Royal Saudi Land Forces is an ASP.NET Core MVC application on .NET 8. Its actual entry point is SmartFoundation.Mvc/Program.cs. The runtime path consists of MVC, UI, Application, and DataEngine, while the SmartFoundation.Database project represents a reference snapshot rather than the live operational database.

The prevailing pattern, particularly in Housing, reads the Session context, then passes through MastersServices, DataSet, and the Masters_DataLoad gateway, and writes through CrudController and Masters_CRUD. Controllers build the SmartRenderer configuration on the server and keep Razor Views thin.

Reconciliation result: Conversations 1–13 and Live Database Reconciliation were completed. A total of 274 live SQL objects were verified within scope: 118 Tables + 44 Views + 76 Stored Procedures + 34 Functions + 2 Triggers.

Reliance boundary: Coverage means completion of the surfaces discovered in the repository and reading of the live catalog; it does not mean execution of business procedures, testing of actual calculations, or E2E verification.

2. Architecture

Current Architecture Update for Version 1.0.0

Verified from the current code: The application registers Cookie Authentication, Claims, Session, and Authorization. The security portion of the pipeline is now ordered as UseRouting -> UseSession -> UseAuthentication -> UseAuthorization -> SessionGuardMiddleware -> endpoint mapping. SessionGuard maintains consistency between the authenticated identity and the Session context, and fails closed when either is missing or they do not match.

Current operational decision: The deployment model uses a Single Application Server with a local In-Memory Session. This is appropriate for the current model and is not recorded as an open issue. Any scale-out requires reassessment of distributed session storage, shared Data Protection keys, load balancing, shared storage where required, health checks, and centralized monitoring.

CSP decision: The policy is implemented and active, with compatibility settings required by the current front-end libraries. It must be reassessed if Alpine.js or other front-end libraries change, or if JavaScript is redesigned to allow a stricter policy.

Any later, older descriptions of Authentication or SessionGuard retain historical value only and are superseded by this current state and the closed-gap register.

Note: Final update 2026-08-23: DATACORE and ERD documentation proves that included programs actually use only dbo and Housing; IncomeSystem and ElectronicBillSystem are program names, not live Schemas. It also documents 274 in-scope objects and four ERDs.

Confidence Level Key

Meaning	Label
Supported by specific active implementation or configuration files.	Verified
A logical conclusion from multiple items of evidence, but not proven by a runtime test or live environment.	Inferred
Repository evidence is insufficient to prove it, or it requires subsequent operational/environmental verification.	Not Verified

Architecture Summary

Verified: The actual application is ASP.NET Core MVC on .NET 8, and its composition and runtime entry point is SmartFoundation.Mvc/Program.cs. The primary application path consists of four project layers: MVC for runtime and presentation, UI for shared presentation components, Application for services and mapping operations to entry procedures, and DataEngine for Dapper execution and SQL Server connectivity.

Verified: The current Housing pattern is the strongest reference for the prevailing path. Reads typically pass from Controller to MastersServices, then ProcedureMapper, SmartComponentService, and dbo.MastersDataLoad, and from there to a downstream feature procedure. Writes pass from SmartFoundation.UI models to /crud/insert|update|delete and then CrudController, which converts p01..p50 to parameter01..parameter50 before reaching dbo.MastersCRUD.

Verified: Housing interfaces are thin at the Razor level. The Controller builds the page configuration on the server, then the View invokes SmartRenderer, which distributes the model to SmartForm, SmartTableDS, SmartDatePicker, SmartPrint, and SmartCharts according to the available properties.

Overview diagram: 01-System-Layers-and-References.md.

The Four Actual Projects and Layers

Key Evidence	Verified Responsibility	Project	Layer
SmartFoundation.Mvc.csproj, Program.cs	Composition Root, Controllers, Views, static files, routing, Session, middleware, and web-application services	SmartFoundation.Mvc	Runtime and Presentation
SmartFoundation.UI.csproj, SmartRendererViewComponent.cs, SmartPageViewModel.cs	Razor Class Library for reusable ViewComponents and ViewModels	SmartFoundation.UI	Shared UI
MastersServices.cs, BaseService.cs, ProcedureMapper.cs	Application services, data-response conversion, and mapping module/operation to an entry stored procedure	SmartFoundation.Application	Application
ConnectionFactory.cs, SmartComponentService.cs	Create SQL connections, build Dapper parameters, execute stored procedures, and return SmartResponse with multiple result sets	SmartFoundation.DataEngine	Data Execution

The SmartFoundation.Database project is outside the four application layers; it is a reference SQL project/Snapshot. SmartFoundation.Application.Tests is a separate test project and not a runtime layer.

Project References

Verified from the project files:

Direct Project References	Source Project
SmartFoundation.Application, SmartFoundation.DataEngine, SmartFoundation.UI	SmartFoundation.Mvc
SmartFoundation.DataEngine	SmartFoundation.Application
SmartFoundation.DataEngine	SmartFoundation.UI
No Project Reference to another solution project	SmartFoundation.DataEngine

Verified architectural note: MVC can access DataEngine and UI directly in addition to Application; therefore, the dependency structure is not strict Onion/Clean Architecture. UI also depends on DataEngine, although the reference Housing path typically passes through Application. This alone does not prove that every Reference is used in every feature.

Actual Entry Point

Verified: The only entry point relied upon in this analysis is SmartFoundation.Mvc/Program.cs. The Program.cs file at the repository root is not the application entry point.

The entry point performs the following:

1. Creates WebApplicationBuilder and uses the default ASP.NET Core Configuration and Environment.
2. Configures the QuestPDF license and registers a font from wwwroot/fonts during startup.
3. Registers MVC with camelCase JSON serialization and registers Razor Pages.
4. Registers distributed memory cache, Session, response compression, antiforgery, and HSTS.
5. Registers data, application, supporting UI, PDF, and Chart services.
6. Builds the middleware pipeline in the documented order below.
7. Maps Razor Pages, attribute controllers, and the default conventional route to Login/Index, then runs the application.

Inferred: Failure to open the font file, or its absence during startup, will prevent startup from completing because reading and registration occur without local exception handling. A runtime test is required to confirm the impact in the actual deployment package.

Middleware and Its Actual Order

The pipeline order is verified from Program.cs:

8. UseResponseCompression().
9. UseHsts().
10. UseHttpsRedirection().
11. UseCookiePolicy(...) with Secure enforced.
12. Inline middleware adds response security headers and CSP at OnStarting, with a stricter policy for the Login surface and a development policy for local connectivity.
13. UseStaticFiles().
14. UseRouting().
15. UseAuthentication().
16. UseAuthorization().
17. UseSession().
18. Endpoint mappings: Razor Pages, attribute controllers, then the conventional route {controller=Login}/{action=Index}/{id?}.

Detailed diagram: 02-HTTP-Request-Pipeline.md.

Accuracy Notes:

- Historical state in the 2026-08-21 analysis: SessionGuardMiddleware existed but was not registered. Version 1.0.0 registers the middleware in practice, as shown in the current architecture update at the beginning of this chapter.
- Verified: UseSession() comes after UseAuthentication() and UseAuthorization(). Controllers can use Session because endpoints execute after the pipeline has completed.
- Verified: Program.cs contains neither UseExceptionHandler() nor UseDeveloperExceptionPage().
- Inferred: According to the code, UseHsts() is applied in all environments and is not placed inside a Production condition. Header testing in the actual hosting environment is required to verify external behavior behind proxy/IIS.

Dependency Injection

Core Registrations

Implementation/Purpose	Lifetime	Service
Retains the Connection String read from Configuration and creates SqlConnection on demand	Singleton	ConnectionFactory
SmartComponentService, the Dapper execution engine	Scoped	ISmartComponentService
Registered as a service in addition to being a Controller, and injected into Controllers to build DDL helpers	Scoped	CrudController
DataSet gateway and shared operations	Scoped	MastersServices
Application services registered through AddApplicationServices()	Scoped	EmployeeService, DashboardService
QuestPdfExportService	Scoped	IPdfExportService
Indirect access to HttpContext	Framework registration	IHttpContextAccessor
Chart service in MVC	Scoped	Chart

Verified: AddApplicationServices() registers only EmployeeService, DashboardService, and MastersServices. The presence of ChartDataService and VehicleService in the project does not mean they are registered here.

Verified: Reliance on concrete services is common, particularly direct injection of MastersServices and CrudController. SmartComponentService uses an interface.

Inferred: The Singleton ConnectionFactory is safe in the sense that it does not retain an open connection; Create() creates a new connection. No operational concurrency test was performed.

Configuration and Runtime Environments

Configuration sources verified in the MVC project:

- appsettings.json: Default connection configuration, SmartData, Logging levels, and AllowedHosts.
- appsettings.Development.json: Logging overrides.
- appsettings.Production.json: Connection and logging overrides.
- Properties/launchSettings.json: HTTP, HTTPS, and IIS Express launch profiles set ASPNETCOREENVIRONMENT=Development locally.
- Environment variables and command-line configuration are part of the default behavior of WebApplication.CreateBuilder, but the actual values in the deployment environment are Not Verified.

Verified: The three appsettings files are defined as Content and are copied to output when required. AI model files and AI documentation defined in the .csproj are also copied.

Verified without disclosing values: The configuration repository contains a sensitive and usable default Connection String, and commented alternatives exist in the configuration files. This is stated only as a configuration fact; its assessment and remediation are deferred to the security task.

Not Verified: The final Configuration source in Production, values that override the files, the actual environment name, secrets management, IIS/reverse proxy settings, and access to the font, model, and knowledge-base paths.

Session

Verified from Program.cs:

- The Session store is a distributed memory cache within the application process.
- IdleTimeout is ten minutes.
- Session cookie: HttpOnly=true, IsEssential=true, and SecurePolicy=Always.

Verified from Login: After GetLoginsDataSetAsync succeeds and user data is extracted, LoginController stores the user, organization, administration, department, section, identity, images, theme, host name, password-change status, and last-activity time in Session. Index and Logout clear the Session.

Verified: Core Controllers such as Housing use InitPageContext to validate userID, then read shared Session values and expose some values through ViewBag. SessionController updates LastActivityUtc and can clear the Session.

Inferred: Because the store is in-memory, Session is not distributed across multiple instances and does not persist after a process restart unless the infrastructure handles this outside the code. There is no repository evidence of sticky sessions.

Not Verified: The final cookie name and effective SameSite setting, the hosted Data Protection keys policy, and the heartbeat invocation mechanism and its compatibility with the ten-minute timeout.

Authentication and Authorization — Initial Architecture View

Authentication

Verified: The Login form sends the National ID and password to LoginController.CheckLogin with antiforgery protection. The Controller calls MastersServices.GetLoginsDataSetAsync, which reaches dbo.GetSessionInfoForMVC through the auth:sessions mapping. Authentication success is determined from the returned data, after which the user context is stored in Session.

Verified: AllowAnonymous is present on GET Login and POST CheckLogin. Cookie Authentication imports exist in Login, but there is no active SignInAsync; Logout clears the Session and does not execute SignOutAsync.

Verified: UseAuthentication() and UseAuthorization() are present, but neither AddAuthentication() nor AddAuthorization(), nor a custom scheme/handler or policy, was found in the composition root or active registration extension. Therefore, there is insufficient evidence of an effective Claims/Cookie identity.

Authorization

Verified in the Housing pattern: The read gateway returns permissions in the first result set, and the Controller reads permissionTypeNameE and builds flags that control the display of UI actions. In the write path, ActionType and pageName are passed to MastersCRUD, and the reference SQL copy shows that the gateway checks permission before invoking the downstream procedure.

Verified: Core Controllers check for userID in Session and redirect to Login when it is absent. This is distributed application-level validation within Controllers, not verified general middleware.

Not Verified: Complete coverage of every endpoint by these checks, the effectiveness of [Authorize] at runtime, and the consistency of live-database permissions compared with the snapshot. These are matters for the detailed security task.

Logging and Error Handling

Verified: The application uses ASP.NET Core built-in logging through ILogger<T>. The default level is Information, while Microsoft.AspNetCore is set to Warning in the reviewed configuration files.

Verified handling layers:

- BaseService logs operation start, success, mapping errors, and execution errors, and returns JSON success/failure responses for narrowly scoped services.
- MastersServices logs parameters and the results of some operations, converts SmartResponse to DataSet, and throws InvalidOperationException when execution fails.
- SmartComponentService measures execution duration, logs the number of result sets and records, and captures exceptions in SmartResponse with Success=false and Error.
- CrudController reads the IsSuccessful/Message result contract, maps it to the TempData buckets Success, Warning, Error, or Info, and then redirects.
- Other Controllers use TempData or local JSON errors.
- The reference SQL gateway distinguishes and returns business errors and records unexpected errors in dbo.ErrorLog; this requires later verification against the live database.

Verified: No global exception middleware is visible in Program.cs. Handled exceptions therefore depend on local catch blocks or the default hosting/framework behavior.

Architectural note only: Some locations log parameters or complete responses, and some error messages include exception text for the user. Exposure assessment and remediation are outside this task and are deferred to the security analysis.

HTTP Request Flow

Typical Page Request

19. The HTTPS request reaches the pipeline and passes through compression, HSTS, redirection, cookie policy, and security headers.
20. Static files are served before routing when matched.
21. Routing selects the endpoint, then the authentication and authorization stages registered in the pipeline run, after which Session becomes available.
22. The endpoint selects the Controller/Action through attribute routing or the conventional route.
23. In Housing, the Action calls InitPageContext; absence of userID results in a Redirect to Login.
24. The Controller builds the load parameters and requests a DataSet from MastersServices.
25. SplitDataSet separates the tables: permissions first, followed by feature datasets.
26. The Controller builds FormConfig, SmartTableDsModel, and SmartPageViewModel with permission gating.
27. The Razor View invokes SmartRenderer, and the HTML is then composed from the child ViewComponents.
28. The response returns through the middleware; headers are added when transmission begins, and the response may be compressed.

Typical CRUD Request

29. SmartForm posts to /crud/insert, /crud/update, or /crud/delete.
30. CrudController reads the form fields and hidden context.
31. It converts p01..p50 to parameter01..parameter50 and carries the context fields.
32. It executes MastersServices.GetCrudDataSetAsync.
33. It interprets the procedure result, places a Toastr message in TempData, and redirects to a specified URL or Controller/Action.

Data Flow from View to SQL Server

Complete diagram of the read and write paths: 03-Data-Read-and-CRUD-Flows.md.

Read: Verified Housing Path

Browser -> MVC Action -> InitPageContext -> MastersServices.GetDataLoadDataSetAsync -> ProcedureMapper(MastersDataLoad:getData) -> SmartRequest(Operation=sp) -> ISmartComponentService.ExecuteAsync -> ConnectionFactory -> Dapper QueryMultipleAsync -> dbo.MastersDataLoad -> feature DL -> SmartResponse.Datasets -> DataSet -> SplitDataSet -> SmartPageViewModel -> Razor -> SmartRenderer -> HTML

Specifically in WaitingListByResident:

- The Controller reads query parameters and the Session context.
- It passes pageName, idaraID, entrydata, and hostname, followed by positional extras.
- MastersServices names extras through parameter20 in the current load overload.
- table 0 contains permissions, while subsequent tables contain resident data, waiting lists, letters, transfer requests, and lookup datasets according to the page.
- Each DataRow is converted to a dictionary, and aliases such as p01 are added to populate modal forms.

Write: Verified CRUD Path

SmartForm -> POST /crud/{operation} -> CrudController -> core context + p01..p50 mapping -> MastersServices.GetCrudDataSetAsync -> ProcedureMapper(MastersCrud:crud) -> SmartComponentService -> dbo.MastersCRUD -> permission/action routing -> feature SP -> IsSuccessful/Message -> DataSet -> TempData -> Redirect

Evidence boundary: Routing to WaitingListByResidentDL/SP is supported by the SQL snapshot and repository instructions, but this does not prove that the live database matches at runtime.

MastersServices

Verified: MastersServices inherits BaseService and depends on ISmartComponentService and ILogger. It is a shared gateway for multiple patterns:

- DataSet loading through MastersDataLoad.
- Additional DataSet loading through MastersExtraDataLoad.
- Write DataSet operations through MastersCRUD.
- Authentication and password changes through separate mappings.
- Menu/notification operations and some AI operations through ExecuteOperation or shared mappings.

Its primary architectural function is to normalize parameter names, create SmartRequest, invoke the data engine, and then convert SmartResponse.Data/Datasets into System.Data.DataSet/DataTable objects consumed by the current Controllers.

Verified: The name in the code is MastersServices with its current spelling, and it is a widely used actual contract.

ProcedureMapper

Verified: A fixed central mapping from a module:operation key to a stored procedure name. The primary gateway keys are:

- MastersDataLoad:getData -> dbo.MastersDataLoad.
- MastersExtraDataLoad:getData -> dbo.MastersExtraDataLoad.

- MastersCrud:crud -> dbo.MastersCRUD.
- auth:sessions -> dbo.GetSessionInfoForMVC.
- auth:changePassword -> dbo.ReSetUserPassword.

Verified: The mapper contains entry/gateway procedures rather than every downstream feature procedure. This preserves routing within the SQL gateways according to pageName and ActionType.

SmartComponentService and ConnectionFactory

Verified: SmartComponentService.ExecuteAsync:

34. Validates that SpName is present.
35. Reads SmartData:Whitelist; if the list is non-empty, it rejects a name that is not allowed, and if the list is empty, it applies no name restriction.
36. Builds DynamicParameters and internally adds the @ prefix to parameter names; therefore, Application code does not add it.
37. Applies paging/sort/filter helpers only when Operation == "select".
38. Opens a new connection from ConnectionFactory.
39. Attempts QueryMultipleAsync with CommandType.StoredProcedure and reads all result sets into dictionaries.
40. If reading multiple sets fails, it falls back to QueryAsync.
41. Returns SmartResponse with success/data/datasets/total/duration or error.

Verified: ConnectionFactory reads ConnectionStrings:Default when constructed and returns a new SqlConnection; the document does not disclose its value.

CrudController and the Parameter Contract

Verified: The primary Routes are:

- POST /crud/insert.
- POST /crud/update.
- POST /crud/delete.
- Additional shared endpoints exist for DDL, extra data load, and render form, but they are not the focus of the primary CRUD path.

Context Fields

The recurring contract preserves the following names and casing:

- pageName
- ActionType
- idaraID
- entrydata
- hostname
- redirectAction
- redirectController
- redirectUrl when available

Positional Mapping

For each index from 01 to 50:

p01 -> parameter01, p02 -> parameter02, ... , p50 -> parameter50.

Empty values are generally converted to DBNull.Value. ActionType is converted to uppercase. MastersServices retains the core names expected by the gateway procedure.

Verified: CrudController also supports file fields named pNN, saves the files, and then places JSON containing their paths in parameterNN. File-upload security details are deferred to the security task.

Architectural rule: The order of pNN carries positional business meaning defined by each ActionType/downstream procedure; it must not be renamed or reordered without a synchronized contract change across UI, Controller, and SQL.

SmartFoundation.UI and SmartRenderer

The SmartFoundation.UI Razor Class Library contains shared presentation models and components. SmartPageViewModel is a container that can hold:

- FormConfig.
- A general table or TableDS through TableDS5.
- DatepickerViewModel.
- SmartChartsConfig.
- SmartPrintConfig.
- Panel metadata and alerts.

Verified: SmartRendererViewComponent itself is simple and returns the View with the model. The composition logic in Views/Shared/Components/SmartRenderer/Default.cshtml checks non-null properties and invokes the appropriate ViewComponents, and can display more than one table on a page.

Verified: The reference View WaitingListByResident.cshtml does not reconstruct the data; it sets the title and invokes SmartRenderer. The Controller determines the fields, tables, buttons, guards, and display of actions according to permission flags.

Architectural conclusion: The UI here uses server-configured component composition: business/page orchestration in the MVC Controller, reusable rendering in UI, and data execution in Application/DataEngine.

Verified Execution Paths

SQL Gateway	Starting Point	Evidence Level	Path
dbo.MastersDataLoad	HousingController.WaitingListByResident	Verified through the gateway; downstream supported by the Snapshot	Housing page load
update	/crud/insert	Verified through the gateway; downstream supported by the Snapshot	CRUD for a Housing page
dbo.GetSessionInfoForMVC	LoginController.CheckLogin	Verified through entry-procedure mapping	Login
dbo.ReSetUserPassword	LoginController.ChangePassword	Verified through entry-procedure mapping	Password change
SmartRenderer, then nested ViewComponents	Razor View	Verified	Page rendering

Primary Evidence

- SmartFoundation.Mvc/Program.cs
- .csproj files for the four projects
- SmartFoundation.Application/Extensions/ServiceCollectionExtensions.cs
- SmartFoundation.Application/Services/MastersServices.cs
- SmartFoundation.Application/Services/BaseService.cs
- SmartFoundation.Application/Mapping/ProcedureMapper.cs
- SmartFoundation.DataEngine/Core/Services/SmartComponentService.cs
- SmartFoundation.DataEngine/Core/Utilities/ConnectionFactory.cs
- SmartFoundation.Mvc/Controllers/CrudController.cs
- SmartFoundation.Mvc/Controllers/Login/LoginController.cs
- SmartFoundation.Mvc/Controllers/Housing/HousingController.Base.cs
- SmartFoundation.Mvc/Controllers/Housing/WaitingList/HousingController.WaitingListByResident.cs
- SmartFoundation.Mvc/Views/Housing/WaitingList/WaitingListByResident.cshtml
- SmartFoundation.UI/ViewModels/SmartPage/SmartPageViewModel.cs
- SmartFoundation.UI/ViewComponents/SmartRenderer/SmartRendererViewComponent.cs

- SmartFoundation.UI/Views/Shared/Components/SmartRenderer/Default.cshtml
- SmartFoundation.Mvc/appsettings.json and Properties/launchSettings.json, without copying secrets

Live Reconciliation Update 2026-08-22

The MastersDataLoad and MastersCRUD paths and downstream definitions for conversations 1–7 were verified against live DATACORE. The result was 51 procedures: 40 matching and 11 different, with no required object missing. This update replaces the Live Procedure Status gap for this scope only; see 07A-Live-Database-Reconciliation.md.

3. Solution Structure and Scope

Solution and Projects

The solution is named SmartFoundation.sln and consists of the following projects:

Verified Primary Role	Framework	Type	Project
Web application and actual entry point in SmartFoundation.Mvc/Program.cs	.NET 8	ASP.NET Core MVC	SmartFoundation.Mvc
Application services and operation-to-entry-procedure mapping through ProcedureMapper	.NET 8	Class Library	SmartFoundation.Application
Execute Dapper requests and connect to SQL Server	.NET 8	Class Library	SmartFoundation.DataEngine
ViewComponents, ViewModels, and shared rendering components	.NET 8	Class Library	SmartFoundation.UI
Current xUnit tests	.NET 8	Test Project	SmartFoundation.Application.Tests
Snapshot/reference for database definitions, not the final live source	SQL project	SQL Server Database Project	SmartFoundation.Database

The Four Actual Layers

The initial structure verified from Project References and the execution path is:

42. Presentation and runtime layer: SmartFoundation.Mvc.
43. Shared user-interface component layer: SmartFoundation.UI.
44. Application and services layer: SmartFoundation.Application.
45. Data execution layer: SmartFoundation.DataEngine, which connects to the live SQL Server database.

SmartFoundation.Database exists as a database-structure reference outside the four application layers, and there is a separate test project.

Key Project References and Packages

- SmartFoundation.Mvc depends on Application, DataEngine, and UI.
- SmartFoundation.Application depends on DataEngine.
- SmartFoundation.UI depends on DataEngine.
- Key packages discovered include Dapper, Microsoft.Data.SqlClient, ExcelDataReader, QuestPDF, xUnit, Moq, and FluentAssertions.

Configuration and Entry Point

The following were initially verified from SmartFoundation.Mvc/Program.cs:

- MVC Controllers with Views and Razor Pages.
- Distributed Memory Cache and Session.
- Response Compression, Antiforgery, HSTS, and HTTPS Redirection.
- Authentication and Authorization in the HTTP pipeline.
- Registration of ConnectionFactory, ISmartComponentService, CrudController, the PDF service, and application services.
- Attribute Controller routes and a conventional MVC route.

Authentication, Authorization, and Middleware-order details are documented in the architecture and security work; this list is not a final analysis.

Documentation Scope

Included Programs

Status	Associated Views	Logical Controllers	Controller Files	Primary Controller Path	Program
Included	3	1	4	Controllers/ControlPanel	ControlPanel
Included	20	3	20	Controllers/Housing	Housing
Included	5	1	6	Controllers/IncomeSystem	IncomeSystem
Included	4	1	5	Controllers/ElectronicBillSystem	ElectronicBillSystem
Included	2	1	2	Controllers/Home	Home
Included	1	1	1	Controllers/Login	Login
—	35	8	38	—	Total

The difference between 38 files and 8 logical Controllers is caused by the system splitting large Controllers through partial class. The logical Controllers are ControlPanelController, HousingController, UploadExcelController, UploadLabController, IncomeSystemController, ElectronicBillSystemController, HomeController, and LoginController.

The Housing count includes the UploadExcel and UploadLab pages because their Controllers are under Housing, although their Views are in two separate folders under Views.

Deferred Programs

- HousingCommandCenter
- Maintenance
- Support
- Vehicle
- Dashboard
- Statistics
- PopulationDensity
- RealCharts

These modules are outside the current Coverage. Only any necessary relationship with an included program will be documented in the detailed phases.

Shared Surfaces or Surfaces Not Resolved as Independent Modules

Shared Controllers and components include CrudController, ExportsController, ReportsController, NotificationsController, SessionController, SmartComponentController, API, and Shared Views. They are not treated as independent programs, but are documented where they serve included programs.

Pages Discovered Within Scope

ControlPanel

- PagesManagment
- Permission
- Users

Housing

- BuildingClass
- BuildingDetails
- BuildingType
- BuildingUtilityType
- MilitaryLocation
- HousingExit
- HousingExtend
- HousingHandover
- HousingResident
- Assign
- AssignStatus
- OtherWaitingList
- RentExemption
- Residents, in addition to ResidentsPrint
- WaitingList
- WaitingListByResident
- WaitingListMoveList
- UploadExcel
- UploadLab

IncomeSystem

- DeductListReport
- FinancialAuditForExtendAndEvictions
- FinancialAuditForUser
- ImportExcelForBuildingPayment
- ExtendInsurance

ElectronicBillSystem

- Meters
- AllMeterRead
- MeterReadForOccubentAndExit
- MeterServiceTypeFixedAmount

Home and Login

- Home: Index and Privacy.
- Login: Index.

Services and Data Layer

Core services discovered:

- MastersServices: The active DataSet gateway for loading, CRUD, and authentication.
- BaseService: A base for narrowly scoped services with JSON responses.
- EmployeeService, DashboardService, ChartDataService, and VehicleService: Existing services whose precise relationship to the scope will be determined later.
- SmartComponentService: The Dapper engine that executes SmartRequest and supports multiple result sets.
- ConnectionFactory: Responsible for creating the SQL Server connection.
- CrudController: A shared CRUD contract across many pages that converts p01..p50 to parameter01..parameter50.

The prevailing path initially verified for the included pages is:

View -> MVC Controller -> MastersServices -> SmartComponentService -> dbo.MastersDataLoad /
dbo.MastersCRUD -> downstream procedure -> SQL objects

Live Database: Metadata Inventory

A connection was made to the live DATACORE database and only sys.schemas and sys.objects were read. Results at inventory time:

Triggers	Functions	Procedures	Views	Tables	Schema
0	13	35	25	82	dbo
0	0	1	0	1	Demo
2	33	61	40	86	Housing
0	0	14	9	14	Maintenance
0	2	16	0	1	MoveData
0	0	10	0	10	support
0	0	0	0	30	Tickets
0	0	77	2	20	VIC
0	3	1	11	15	WH

The presence of a Schema in the database does not automatically include it in the documentation scope. Tables, Views, and Functions important to the included programs will be identified through reference tracing in the database task.

Stored Procedures Initially Identified

Verified Shared Entry Procedures

- dbo.MastersDataLoad
- dbo.MastersCRUD
- dbo.MastersExtraDataLoad
- dbo.GetSessionInfoForMVC
- dbo.ReSetUserPassword
- dbo.GetUserMenuTree

Initial Mapping from In-Scope Pages to Live Procedures

- ControlPanel: dbo.PagesManagmentSP, dbo.PermissionSP, dbo.UsersDL, dbo.UsersSP.
- Housing definitions and procedures: DL and SP pairs matching BuildingClass, BuildingDetails, BuildingType, BuildingUtilityType, MilitaryLocation, HousingExit, HousingExtend, HousingHandover, and HousingResident.
- Housing waiting-list features: DL and SP pairs matching Assign, AssignStatus, OtherWaitingList, RentExemption, Residents, WaitingList, WaitingListByResident, and WaitingListMoveList.
- IncomeSystem: Housing.ExtendInsuranceDL/SP, Housing.FinancialAuditForExtendAndEvictionsDL/SP, Housing.FinancialAuditForUserDL/SP, and the three discovered ImportExcelForBuildingPayment procedures.
- ElectronicBillSystem: DL and SP pairs matching Meters, AllMeterRead, MeterReadForOccubentAndExit, and MeterServiceTypeFixedAmount.
- Home: dbo.HomeDL.

A total of 62 preliminary procedures were identified, including entry gateways, authentication, and live name matches. This is not final Coverage: routing and actual use still require verification, particularly for DeductListReport, the UploadExcel/UploadLab pages, and procedures that SQL gateways may invoke under names that do not match the page.

Phase Result

The Inventory and scope are complete, and the project is ready for the architecture and shared-components analysis task without repeating the baseline inventory.

4. Security and Permissions

Current State for Version 1.0.0

Verified from the current code: The application uses Cookie Authentication and creates Claims after successful verification, with Session retaining the user operational context. It registers `AddAuthentication().AddCookie()`, invokes `SignInAsync` in the login path and `SignOutAsync` in the logout path, and applies `SessionGuardMiddleware` to compare the Claim identity with `Session["userID"]` and fail closed when the session is missing or does not match.

The relevant middleware order is: Routing -> Session -> Authentication -> Authorization -> `SessionGuardMiddleware` -> Endpoints.

The current operational model is a Single Application Server, and local In-Memory Session storage is acceptable for this model. When scaling to multiple instances, a Load Balancer, or High Availability, distributed session storage, shared Data Protection keys, load balancing, health checks, and centralized monitoring must be reassessed.

The following sections retain the 2026-08-21 analysis as Historical State. Statements denying Cookie Authentication, Claims, or `SessionGuard` registration do not represent the Current State of version 1.0.0; the later gap register documents closure and verification.

Note: Database update 2026-08-23: The ControlPanel and permissions ERD verifies live FKs and corrects assumptions based on FK names: Idara links in Permission and some DeptSecDiv links are inferred relationships, not actual constraints.

Status and Boundaries

- Task: Security analysis only, dated 2026-08-21.
- The analysis covered Login/Logout, Authentication/Authorization, Roles/Permissions, Claims, Session, endpoint protection, `permissionTypeNameE`, `ftUserPagePermissions`, password management, logs, notifications, Antiforgery/Cookies/HSTS, flows, and risks.
- No write or business Stored Procedure was executed, no actual user or log rows were read, and no secrets or real identities were exposed.
- Active code is the primary source. SmartFoundation.Database snapshot files are for reference only.
- Superseded historical note: When this document was prepared on 2026-08-21, only names were verified and `sys.sqlmodules` was unavailable because of TLS; details were therefore based on the Snapshot at that time. Live reading was completed on 2026-08-22; its results in the update section and 07A are authoritative.

Executive Summary

Historical state at the time of the 2026-08-21 analysis: Authentication was then neither ASP.NET Core Identity nor Cookie Authentication. The login POST invoked `dbo.GetSessionInfoForMVC`, the result determined login success, and the application then stored the user identity and organizational context in server-side Session.

The application did not create `ClaimsIdentity/ClaimsPrincipal`, invoke `SignInAsync`, or register an Authentication Scheme or Authorization Policies. `UseAuthentication()` and `UseAuthorization()` alone do not provide protection. Actual protection was distributed among Session checks within some Actions, UI display based on `permissionTypeNameE`, and SQL checks in the CRUD gateway.

Historical state at the time of analysis: The general guard was not active; `SessionGuardMiddleware` existed but was not registered. This state was later closed, and the current state is documented at the beginning of the chapter.

Authentication and Login/Logout

Verified Path:

POST /Login/CheckLogin -> `MastersServies.GetLoginsDataSetAsync` -> `ProcedureMapper(auth:sessions)` -> `dbo.GetSessionInfoForMVC` -> `ExtractAuth` -> Session -> /Home/Index

- GET Login and POST CheckLogin carry `[AllowAnonymous]`, and the POST carries `[ValidateAntiForgeryToken]`.
- National ID, password, and host are passed as `Dapper` parameters.
- Success requires a non-empty `userId` and `usersActive != 0`.

- GET Login clears Session and prevents caching.
- No MFA, logout, rate limiting, or CAPTCHA is verified.

GetSessionInfoForMVC is verified live by name, and its definition is supported by the snapshot. The reference definition searches for an active user within the validity period, reads the latest active UsersPassword, compares HASHBYTES('SHA2256', Salt + password-bytes) with PasswordHash, then validates user details and the administrative structure and returns the session context.

Actual Logout is Session.Clear() through GET /Login/Logout or POST /session/logout. There was no active SignOutAsync and no evidence of Session ID rotation.

Diagram: Security-Authentication-and-Session-Flow.md.

Claims and Roles

- There was no active registration of AddAuthentication, AddCookie, AddIdentity, or policies.
- There was no SignInAsync, Claims creation, or security reliance on HttpContext.User.
- There was no [Authorize] or [Authorize(Roles=...)] within the reviewed scope.
- Role in the system is a database entity within the permissions network, not an ASP.NET Core role claim.

Session

Verified configuration: DistributedMemoryCache within one instance, a 10-minute idle timeout, and a Session cookie with HttpOnly=true, IsEssential=true, and Secure=Always. The cookie name, SameSite, and Data Protection keys policy are not customized in code.

Keys written by Login:

Keys	Category
usersID, nationalID, GeneralNo	Identity
fullName, usersActive, ChangedPassword	Name and Status
OrganizationID/Name, IdaraID/Name, DepartmentID/Name, SectionID/Name, DivisonID/Name, DeptCode	Organization
AdminTypeID, AdminTypeName	Administration
photoBase64, ThameName, OrganaiztionLogo, IdaraLogo	Presentation
HostName, LastActivityUtc	Runtime

The interface also uses MENUTREE. Inconsistent names exist, such as usersActive/useractive and usersID/UserId. POST /session/keepalive updates activity without an identity check. In-memory Session is not distributed across instances and does not persist after restart.

Protection of Controllers and Actions

Primary Actions in Home, ControlPanel, Housing, IncomeSystem, and ElectronicBillSystem generally invoke InitPageContext, which checks Session[usersID]. Notifications APIs check the key locally. Coverage, however, is not universal:

- Base Index in several Controllers returns a View without InitPageContext.
- CrudController had no [Authorize] or general Session check on insert/update/delete and supporting endpoints.
- GetDDLValues2, extradataload, and renderform accept JSON without identity verification inside the Action.
- SessionController endpoints had no general guard.
- Antiforgery attributes were present only on Login and some uploads.
- Protecting the page GET does not automatically protect the shared POST.

Authorization and Permissions

Data Model

The following relationships are supported by the snapshot:

- Users, UsersDetails, UsersPassword, UsersAuthType.
- Role.
- Distributor optionally associated with Role, and UserDistributor for assigning a user with an active period.
- Menu and MenuDistributor.
- PermissionType, which contains permissionTypeNameE.
- DistributorPermissionType, which associates a distributor with permission types.
- Permission, which can target a User, Role, DSD, or Distributor with an active period.
- Supporting Views: VGetListUserPermission, VGetListUsersInDSD, VGetFullStructureForDSD.

A permission may be granted directly to the user, through Role/Distributor or DSD, or as a general grant.

ftUserPagePermissions and permissionTypeNameE

The reference function returns distinct English permission names for a user and page. It matches menuNameE = PageName and combines direct grants, Role, DSD, Distributor, and general grants, with activity and date filters.

MastersDataLoad returns them in the first result set. Controllers convert permissionTypeNameE into flags used to display INSERT/UPDATE/DELETE and specialized actions. This is UI gating, not a security boundary.

The reference MastersCRUD compares the page, action, and user against permissions before many write branches, then routes to the feature SP. Diagram: Security-Authorization-and-Permission-Flow.md.

Critical CRUD Trust Boundary

Critical risk verified from code: /crud/insert|update|delete read pageName, ActionType, idaraID, entrydata, and hostname from the form. extradataload reads them from JSON. The Controller does not replace entrydata with the Session user or idaraID with the Session Idara.

Because SQL uses entrydata for permission checks and auditing, the client can alter the identity context before it reaches the database. No exploit or write test was performed. Fixed default numeric values in update/delete increase the impact of missing fields.

User and Password Management

- The ControlPanel Users page is protected by a Session check and uses DataLoad/CRUD and page permissions; the program was not documented again in detail.
- POST /Login/ChangePassword obtains the user ID from Session, verifies that the old and new passwords are present and that the length is 8, then invokes ReSetUserPassword with the CHANGEUSERPASSWORD action.
- The reference definition verifies the old password using salt/hash, creates a random salt, deactivates the old password, creates a new record, and sets ChangedPassword=1.
- JavaScript requires an uppercase letter, a lowercase letter, and a digit, while the Controller verifies only the length. The live SQL definition requires at least 8 characters with a digit and one English letter and prevents the new password from matching the current one; it does not require both uppercase and lowercase, so that condition remains UI-only.
- The administrative reset branch in the snapshot uses a known fixed default password within the procedure and sets ChangedPassword=0. Its value is not stated here. This is a critical risk until the live definition is shown to differ or compensating controls are verified.
- There is no evidence of password history, expiry enforcement, logout, or breached-password checking.

AuditLog and ErrorLog

The reference AuditLog contains TableName, ActionType, RecordID, PerformedBy, Notes, and PerformedAt. Many procedures write to it. Because entrydata often becomes PerformedBy, the modifiable context weakens attribution of an action to its performer. There is no evidence of immutability, retention, or read protection.

The reference ErrorLog contains the error message, severity, state, SP name, and time/user/host. The reference MastersCRUD returns business errors 50001..50999 without logging, logs unexpected errors, and returns a generic

code. In MVC, some locations return ex.Message to the client, including Login and some CRUD/extra-data paths, which may disclose internal details.

No actual AuditLog or ErrorLog rows were read.

Permission-Related Notifications

The reference NotificationsCreate creates a Notification and then selects recipients by User, Distributor, or Role; by Idara for some patterns; by DSD; by Menu + Idara; by PermissionType(s) + Idara; or as all active users. It uses user status and the active periods of permission relationships, then creates UserNotifications while preventing duplicates.

This is permission-based recipient distribution, not a permission grant. The effectiveness of IDOR prevention when updating a notification depends on live procedures whose definitions were not read in detail.

Antiforgery, Cookies, and HSTS

- Antiforgery is registered, and its cookie is HttpOnly/Secure.
- Only Login and some upload POSTs carry [ValidateAntiForgeryToken]; there is no global AutoValidateAntiForgeryToken.
- CRUD, ChangePassword, Notifications, and Session POSTs have no verified validation, and the primary Logout is GET.
- Cookie Policy enforces only Secure. The Session cookie is HttpOnly/Secure/Essential. SameSite and the name/domain/path are not customized.
- UseHttpsRedirection is active.
- HSTS is configured for one year with IncludeSubDomains and is applied without a visible Production condition; the same header is also added manually.
- There is no repository verification of TLS termination, Forwarded Headers, or the suitability of IncludeSubDomains.
- CSP, X-Frame-Options, nosniff, COOP/COEP/CORP, and Permissions-Policy are present. The internal CSP permits unsafe-inline, unsafe-eval, and general https: sources, reducing its effectiveness against XSS.

Related Objects

Objects	Domain
Users, UsersDetails, UsersPassword, UsersAuthType, GetSessionInfoForMVC, ReSetUserPassword	Login
Role, Distributor, UserDistributor, Menu, MenuDistributor, PermissionType, DistributorPermissionType, Permission, ftUserPagePermissions, VGetListUserPermission	Permissions
MastersDataLoad, MastersCRUD	Gateways
AuditLog, ErrorLog	Logs
Notifications, UserNotifications, NotificationsCreate	Notifications
VGetListUsersInDSD, VGetFullStructureForDSD	Support

Result

The security task was completed within scope, without starting documentation of a program or a subsequent task.

Live Reconciliation Update 2026-08-22

The definitions of GetSessionInfoForMVC, ReSetUserPassword, SetUserPassword, ftUserPagePermissions, VGetListUserPermission, and NotificationsCreate were verified live and match the Snapshot. Live MastersCRUD differs in routing but still trusts entrydata/pageName/ActionType/IdaraID received from the application; therefore, the client-context risk remains. The live HousingHandover branch still does not match ActionType precisely. The live collation was confirmed as case-insensitive, but permission data was not read.

Live Home and Login Update 2026-08-23

- GetSessionInfoForMVC, ReSetUserPassword, GetUserMenuTree, HomeDL, ChartsDL, and MastersDataLoad were reread through SELECT-only catalog access; all match the Snapshot.
- It was verified live that the administrative reset still uses a fixed default value (not disclosed), and that storage uses a random salt and SHA2-256.
- The SQL password-change policy was confirmed as 8 characters + a digit + an English letter + difference from the current password; uppercase/lowercase requirements exist only in the UI.
- GetSessionInfoForMVC enforces the account, details, active period, structure, and password, but does not use the hostName parameter in its body.
- GetUserMenuTree applies direct, Role, DSD, Distributor, and general grants with active periods, and returns parent and program nodes.
- Remaining risks were: no general guard/claims, partial CSRF protection, disclosure of ex.Message during login, fixed reset, no rate limiting/lockout/MFA, UI-only forced change, and logging of the complete menu JSON.
- Home adds two risks beyond the earlier analysis: a GET executes a billing Business SP, and Privacy does not invoke a Session check. The procedure was not executed during documentation.

5. ControlPanel

Note: Database update 2026-08-23: Final live keys and relationships are documented in DATACORE, the ERD, and the permissions ERD. Any relationship labeled Inferred there is not an actual FK.

Functional Purpose

ControlPanel is the central administration program for three interrelated areas:

- 46. Defining programs, side menus, pages, and page permissions through PagesManagment.
- 47. Assigning permissions to a user, distributor, role, administration, or organizational structure through Permission.
- 48. Creating, updating, and disabling users and resetting passwords through Users.

Page administration produces Program, Menu, Distributor, MenuDistributor, PermissionType, and DistributorPermissionType objects. The Permission page then uses these definitions to create Permission records. Other Unified Platform of the Engineering and Works Administration of the Royal Saudi Land Forces programs use the resulting structure to build menus and validate operations.

Completed Inventory

Items	Count	Type
ControlPanelController	1	Logical Controller
Base, PagesManagment, Permission, Users	4	Controller Files
Index, PagesManagment, Permission, and Users	4	Actual Actions
PagesManagment, Permission, Users	3	Views
12 + 4 + 4	20	UI Write Operations
MastersDataLoad, MastersCRUD, PagesManagmentSP, PermissionSP, UsersDL, UsersSP, ReSetUserPassword, SetUserPassword	8	Direct SQL Procedures/Verified Gateways
ftUserPagePermissions	1	Direct Security Function

An Index Action exists in the base file, but no View named Views/ControlPanel/Index.cshtml was discovered; therefore, it is not counted as a complete page among the three pages.

ControlPanelController and Its Parts

Base File

ControlPanelController.Base.cs injects MastersServies, CrudController, and ILogger<ControlPanelController>. It retains Session context in fields such as userId, Idarald, and HostName, as well as permissionTable and dt1..dt13.

InitPageContext rejects the request when Session[userId] is absent and redirects to Login/Index?logout=1, then reads user and organizational data. SplitDataSet establishes the following contract: table 0 for permissions and tables 1 through 13 for page data and DDL. There is no [Authorize] on the Controller; the effective GET guard is the local Session check.

Actions

Output	Inputs	Actual HTTP	Action
View(); View not discovered	None	GET through the conventional route	Index()

Output	Inputs	Actual HTTP	Action
SmartPageViewModel to MenuManagment/PagesManagment	Session	GET	PagesManagment()
Search form and optional table to Permission/Permission	Query filters + Session	GET	Permission()
User table and four CRUD forms to Permission/Users	Session	GET	Users()

There are no program-specific write Actions; all forms submit to /crud/insert, /crud/update, or /crud/delete.

Views, Models, and ViewModels

The three Views receive SmartPageViewModel and invoke SmartRenderer. Permission and Users remove the query string from the browser address after loading by using history.replaceState, without another request. PagesManagment also contains a Debug block that appears only when the Controller places a value in ViewBag.DebugInfo.

There are no ControlPanel-specific domain Models. The models used are shared from SmartFoundation.UI:

- SmartPageViewModel: Container for the page, form, and tables.
- SmartTableDsModel, TableColumn, TableToolbarConfig, and TableAction: The table, columns, and operations.
- FormConfig, FieldConfig, FormButtonConfig, and OptionItem: Forms, fields, and DDL.
- SQL rows are converted to Dictionary<string, object?>, and aliases named p01..pNN are added to populate the modal and pass through the positional CRUD contract.

PagesManagment Page

Function and Data

The page loads system programs, menus, pages, and page permissions. Reads pass to MastersDataLoad with pageName = PagesManagment. In the SQL snapshot there is no downstream DL; the gateway executes direct queries and returns tables for programs, menus, pages, their permissions, and authorization-level lists.

Operations and Permissions

Effect	Required Gateway Permission	ActionType	Group
Insert Program	ADDPROGRAM	AddProgram	Program
Update program data	EDITPROGRAM	EditProgram	Program
Change program status; logical deletion	DELETEPROGRAM	DeleteProgram	Program
Insert a Menu linked to a Program	ADDMENU	AddMenuList	Menu
Update menu	EDITMENU	EditMenuList	Menu
Change menu status	DELETEMENU	DeleteMenuList	Menu
Insert Menu as a page, then Distributor and MenuDistributor	ADDMENU	AddPage	Page
Update the page and its associated distributor	EDITMENU	EditPage	Page
Change the status of Menu, MenuDistributor, and associated distributor	DELETEMENU	DeletePage	Page

Effect	Required Gateway Permission	ActionType	Group
Create PermissionType and link it to the distributor	ADDPERMISSION	AddPagePermission	Page Permission
Update the DistributorPermissionType permission level	EDITPERMISSION	EditPagePermission	Page Permission
Change DistributorPermissionType status	DELETEPERMISSION	DeletePagePermission	Page Permission

The interface reads the preceding nine permissions from table 0 and displays the corresponding tools. The gateway maps mixed-case Action names to uppercase permission names and then checks VGetListUserPermission; therefore, the server-side check is the final control.

Key Fields and Business Rules

- Program: Arabic and English names, description, link, icon, and order; names must not be duplicated, order must be positive, and status must be 0 or 1.
- Menu: Parent program, two names, description, and order; a valid program identifier is required, and a duplicate active name is not accepted.
- Page: Parent program or menu, Arabic/English name, description, link, order, and level; the name or link must not be duplicated among active pages.
- Adding a page creates three linked records: Menu -> Distributor -> MenuDistributor.
- Page permission: Arabic/English name and authorization level; neither the permission type nor its association with the distributor may be duplicated.
- Deletion in this path changes Active status and is not a physical deletion.
- The procedures use transaction, XACTABORT, and TRY/CATCH, and write to AuditLog. Errors 50001 are expected business errors, while 50002 are execution errors.

Permission Page

Function and Search

The page supports permission searches by SearchID: user, distributor, role, administration, or Department/Section/Division. The Controller reads query parameters (UserID, RoleID, IdaraID, Dept, Section, Divison, distributorID) and converts them to positional parameters. The table is not displayed until the selected search criterion is ready.

MastersDataLoad executes Permission queries internally and returns, in addition to the result, DDL data for users, distributors, permission types, administrations, departments, sections, divisions, roles, programs, and pages. The isAdmin level restricts PageLvl and permissionAuthLvl to 1/2/3, 2/3, or 3.

Operations

Rule	Required Permission	ActionType
Assign a specific permission to one target type with optional time context	Same name	INSERTPERMISSION
Add all distributor permissions not yet assigned to the target	Same name	INSERTFULLACCESS
Deactivate the old record and create a new version with updated values	Same name	UPDATEPERMISSION
Logically deactivate the permission	Same name	DELETEPERMISSION

The interface displays “Add Full Access” under canInsertPERMISSION even though the Controller also reads canInsertFullAccess; the specialized flag is therefore not used in ShowAdd1. MastersCRUD does enforce INSERTFULLACCESS, preventing execution when it is absent. This is a documented UI consistency gap.

Validation and Business Rules

- Action is required, and the performer identifier and host name are required by the procedure.
- Granting highly sensitive permissions is prevented by special conditions within PermissionSP, including checks associated with Role 20 and user 4 as represented in the snapshot; their organizational meaning requires confirmation from the system owner.
- DistributorPermissionTypeID must be specified for a single assignment, and an active duplicate for the same target is prevented.
- Full access adds only active, currently valid permissions that do not already exist.
- The end date must be later than the start date when the branch applies temporal validation.
- UPDATE deactivates the old record and then inserts a new version to preserve the audit trail, while DELETE is a soft delete.
- All branches write to AuditLog and return IsSuccessful/Message.

Users Page

Function and Data

Users displays accounts and identity, organizational, and classification data. Reads follow MastersDataLoad -> UsersDL. UsersDL returns user data, followed by lookup tables for UsersAuthType, UserType, Idara, Distributor, organizational structure, Gender, Nationality, Religion, MaritalStatus, and Education.

Core fields: National identity and general number, required Arabic name, optional English name, system validity, user type, administration and distributor, identity issue date and date of birth, gender, nationality, religion, marital status, education, and notes. p35 is the update/disable reason and p36 is the distributor.

Operations and Permissions

Effect	Required Permission	ActionType
Insert Users, UsersDetails, and UserDistributor and create a password	Same name	INSERTUSERS
Update account, details, and organizational association	Same name	UPDATEUSERS
Deactivate the user and update related links	Same name	DELETEUSERS
Call ReSetUserPassword and update usersPassword	Same name	RESETUSERPASSWORD

UPDATENATIONALID is a subsidiary permission rather than an independent ActionType; it makes the identity field editable in the UPDATE form and displays a warning. The gateway checks only UPDATEUSERS when saving, so enforcement against identity changes in this snapshot depends on Readonly in the UI rather than a visible independent check in MastersCRUD/UsersSP; this is a security gap that requires live verification.

Password reset is defined as a form and Gateway operation, but the Controller flag list does not read RESETUSERPASSWORD. It must be verified whether the button is connected to another Toolbar field or is not actually displayed; the gateway will prevent the operation without the matching permission in all cases.

Validation and Business Rules

- The identity and general number have a maximum UI length of 10 and numeric input style, and Arabic names are required.
- Required lookup identifiers must be valid.
- UsersSP prevents duplicate identity and general numbers and verifies that the user exists during update/disable operations.
- Insert and update operations are distributed across Users, UsersDetails, and UserDistributor, and write to AuditLog.
- Password creation invokes SetUserPassword; reset invokes ReSetUserPassword. The password is stored in usersPassword according to the procedure logic, and this document exposes no secrets or values.
- Update/disable reasons are required in the interface for UPDATE/DELETE.

- DELETE is a logical disable operation, not physical deletion of the account row.

Services and Data Access

49. The Controller invokes MastersServices.GetDataLoadDataSetAsync for reads.
50. MastersServices resolves MastersDataLoad:getData to dbo.MastersDataLoad through ProcedureMapper.
51. SmartComponentService builds Dapper parameters (adding @) and executes QueryMultipleAsync through ConnectionFactory.
52. The result is converted from SmartResponse.Datasets to DataSet and then SplitDataSet.
53. SmartRenderer displays the Form/Table configurations built by the Controller.
54. Writes are sent to CrudController, which converts p01..p50 to parameter01..parameter50 and invokes MastersServices.GetCrudDataSetAsync.
55. ProcedureMapper resolves MastersCrud:crud to dbo.MastersCRUD, which verifies permission and invokes the specialized procedure.

CrudController.DDLFiltered is used for dependent lists in Permission and Users; it reads a specified table from DataLoad and filters it by FK to build dependent-select options.

Database Relationships

Procedures

Role	Procedure
Read and permission gateway; handles Permission and PagesManagment directly and routes Users to UsersDL	dbo.MastersDataLoad
Write gateway, VGetListUserPermission check, and routing	dbo.MastersCRUD
Program, menu, page, and page-permission rules	dbo.PagesManagmentSP
Assign/copy/deactivate permissions	dbo.PermissionSP
User data and lookups	dbo.UsersDL
Create/update/deactivate users	dbo.UsersSP
Reset user password	dbo.ReSetUserPassword
Create/update password; called by UsersSP	dbo.SetUserPassword

Tables

- Page administration: Program, Menu, Distributor, MenuDistributor, PermissionType, DistributorPermissionType, permissionAuthLvl.
- Permissions: Permission, UserDistributor, Role, Idara, Department, Section, Divison, DeptSecDiv.
- Users: Users, UsersDetails, UsersAuthType, UserType, usersPassword, Gender, Nationality, Religion, MaritalStatus, Education.
- Shared: AuditLog, ErrorLog.

Views and Functions

- VGetListUserPermission: Displays effective permissions and is used for loading and validating every write.
- VGetListUsersInDSD and VGetFullStructureForDSD: Users and organizational structure.
- VGetFullSystemUsersDetails: Extended user details.
- ftUserPagePermissions(user,page): Returns page permission names in the first result set.

The documented path does not rely on SQL triggers. All names above are verified from the reference definitions; the status of live objects was not rechecked in this task.

Relationships with Other Programs

- Every program uses the navigation menu produced from Program/Menu/Distributor/MenuDistributor.
- Every Housing-style write depends on PermissionType/DistributorPermissionType/Permission and VGetListUserPermission managed by ControlPanel.
- Login and Session provide ControlPanel with usersID, IdaraID, and organizational context.
- GetUserMenuTree builds the user menu from the same menu and permission definitions.
- AuditLog and ErrorLog are shared between ControlPanel and the other programs.

Actual Workflows

Publishing a New Authorizable Page

56. Create/select a program.
57. Create an optional side menu.
58. Add the page; the procedure creates Menu, Distributor, and MenuDistributor.
59. Add operation types for the page; this creates PermissionType and DistributorPermissionType.
60. Open Permission, select the target, and assign a permission or Full Access.
61. When the page is used, the first result set determines the UI buttons, and MastersCRUD rechecks the operation on POST.

User Lifecycle

62. UsersDL loads accounts and lookups.
63. INSERT creates the account, details, association, and initial password.
64. UPDATE modifies the data and requires a reason in the UI.
65. UPDATENATIONALID controls whether the identity is editable in the UI.
66. RESETUSERPASSWORD passes to the specialized procedure after permission validation.
67. DELETE disables the account with a reason and does not physically delete it.

Permission Lifecycle

68. Select the target type and filters.
69. DataLoad displays matching permissions and DDL restricted by administration level.
70. Add a single permission or all permissions for a distributor.
71. UPDATE deactivates the previous record and creates a new version.
72. DELETE deactivates the record.

Security and Constraints

- Verified: GET pages require Session[usersID].
- Verified: Display of operations is restricted by the ftUserPagePermissions result.
- Verified: Every POST passes through antiforgery in CrudController and an independent permission check in MastersCRUD.
- Verified: SQL uses transactions, AuditLog, and business error messages.
- Gap: There is no [Authorize] and no authentication scheme verified in the current composition; protection depends on Session and the gateway.
- Historical note: SessionGuardMiddleware was not registered at the time of the initial analysis; the condition was later closed and is recorded in the current architecture documentation for version 1.0.0.
- Gap: entrydata, idaraID, and hostname are provided as hidden fields. It must be ensured that CrudController re-derives them from Session or that SQL does not trust them without validation. The current code passes posted form values.
- Gap: The UPDATENATIONALID permission is enforced only in the UI according to the reviewed path.
- Gap: Consistency of the INSERTFULLACCESS and RESETUSERPASSWORD buttons with UI flags requires functional testing.
- Gap: Index has no discovered View, and some catch blocks display the exception message in ViewBag, which may disclose internal details in an uncontrolled environment.

Live Reconciliation Update 2026-08-22

- Specialized ControlPanel/security procedures match live definitions, except UsersDL.
- The live UsersDL adds distributorID, distributorNameA, DepartmentID, and DepartmentName from the latest active department association to the user result set.
- The live MastersDataLoad adds descending DistributorPermissionType ordering, and MastersCRUD differs in other branches documented in 07A.
- The live parity gap was closed; permission, UI, and magic-identity tests remain unperformed.

6. Housing

Housing Definitions

Note: Database update 2026-08-23: The live Housing ERD corrected relationships according to 75 live FKs in the Housing Schema. BuildingDetailsMeterServices linkage remains procedural/inferred, not an FK constraint.

Actual Shared Pattern

The five pages are parts of the partial HousingController. Each GET begins with InitPageContext, invokes MastersServies.GetDataLoadDataSetAsync, and then uses SplitDataSet to separate Table 0 for permissions and dt1..dt9 for the remaining results. Absence of a permission row returns the user to Home/Index with a denial message.

There are no dedicated domain models. The Controller converts each DataRow to Dictionary<string, object?> and adds p01..pNN, then builds FieldConfig/FormConfig and TableColumn/SmartTableDsModel within SmartPageViewModel. The Views are thin and invoke SmartRenderer; the building View adds only window.UtilityTypeId for printing.

Read: GET -> HousingController -> MastersServies -> ProcedureMapper -> dbo.MastersDataLoad -> feature DL -> DataSet -> SmartRenderer.

Write: modal -> /crud/insert|update|delete -> CrudController -> pNN to parameterNN -> dbo.MastersCRUD -> permission check -> feature SP -> IsSuccessful/Message -> TempData -> redirect.

Diagrams: 05-Housing-Definitions-Flows.md and 06-Housing-Definitions-ERD.md.

Page Matrix

DDL	Result sets after permission	View	Controller Actions	Purpose	Page
None	1	HousingDefinitions/Bu ildingClass	BuildingClass()	Define building classes	BuildingClass
Extra unused Cities	2	HousingDefinitions/Bu ildingType	BuildingType(int pdf=0), Print()	Define types	BuildingType
Local Yes/No	1	HousingDefinitions/Bu ildingUtilityType	BuildingUtilityType()	Utility types, validity, and whether rent is required	BuildingUtilityType
Cities from Table 2	2	HousingDefinitions/Mil itaryLocation	MilitaryLocation()	Locations/neighborho ods and city linkage	MilitaryLocation
Tables 2..6 + local options	8	HousingDefinitions/Bu ildingDetails	BuildingDetails(int pdf=0)	Buildings and definition, rent, and service links	BuildingDetails

BuildingClass

Purpose, UI, and Models

Displays active general or administration-specific classes, with search, sorting, pagination, and column visibility. The key is buildingClassID, and the table hides buildingClassOrder and buildingClassActive. The final model is SmartPageViewModel.TableDS with the title "Building Classes".

Field Contract	Operation
p01 required Arabic, p02 English, p03 notes	Insert
p01 identifier, p02 Arabic, p03 English, p10 notes	Update

Field Contract	Operation
p01 identifier, with p02/p03 readonly	Delete

DataSet, Services, and SQL

- Table 0: permissionTypeNameE.
- dt1: Identifier, both names, description, order, and active from Housing.BuildingClass; active records only, with general or matching Idara, ordered by descending ID.
- Services: MastersServices, SmartComponentService, ConnectionFactory, and CrudController for writes.
- Snapshot routing: BuildingClass -> Housing.BuildingClassDL; writes go to Housing.BuildingClassSP.
- Direct SQL: dbo.MastersDataLoad, dbo.MastersCRUD, dbo.ftUserPagePermissions, dbo.VGetListUserPermission, Housing.BuildingClassDL/SP, Housing.BuildingClass, dbo.AuditLog.

Permissions and CRUD

INSERTBUILDINGCLASS, UPDATEBUILDINGCLASS, DELETEBUILDINGCLASS. These control display of Add/Edit/Delete; update and delete apply to one record. Snapshot: MastersCRUD rechecks the user, page, and ActionType before the SP.

Validation and Business Rules

- The UI requires the Arabic name and uses Arabic TextMode.
- Snapshot: Action and the Arabic name are required; a duplicate active Arabic name within the administration scope is prevented; update/delete require an identifier and an active record.
- Delete is logical (buildingClassActive=0); all operations record AuditLog.
- Gap: The row places the description in p04 while the update form expects p10.
- There is no rule in the SP preventing soft delete when the class is associated with buildings; the old help-file claim of such prevention is Not Verified.

User Steps

73. Open Building Classes and search or sort.
74. To add a record, enter the Arabic name and save.
75. To update or delete, select one record and use the button displayed according to permission.

BuildingType

Purpose, UI, and Models

Displays active general or administration-specific building types. The table supports search and sorting, its key is buildingTypeID, and it displays the code, both names, and description.

Field Contract	Operation
p01 code, p02 Arabic, p03 English, p04 description	Insert
p01 identifier, p02 code, p03 Arabic, p04 English, p10 description	Update
p01 identifier, with details readonly	Delete

There are two PDF paths: pdf=1 for a report and pdf=2 for an experimental letter. However, ShowPrint=false and ShowPrint1=false, so neither button is displayed. The Print() action reads TempData[PdfBytes] and is not connected to the current PDF path.

DataSet, DDL, and SQL

- Table 0: permissions.
- dt1: Active administration-specific/general types in descending order.
- dt2: Active dbo.City records, but the Controller does not consume them; this is an extra DDL result set.
- Snapshot routing: BuildingType -> Housing.BuildingTypeDL/SP.

- SQL: The gateways and shared permission objects, Housing.BuildingTypeDL/SP, Housing.BuildingType, dbo.City, and dbo.AuditLog.

Permissions and Rules

- INSERTBUILDINGTYPE, UPDATEBUILDINGTYPE, DELETEBUILDINGTYPE.
- The UI requires the code and Arabic name; the Arabic name has MaxLength=50 and Arabic TextMode.
- Snapshot: The Arabic name is required and unique among active records within scope; update/delete require an identifier and an active record; delete is logical, with an Audit entry for every operation.
- Gap: The row alias for the description is p05, while update expects p10.
- Gap: The report contains a partially hard-coded header, and there is no visible button for accessing it.

User Steps

76. Open Building Types and review the list.
77. Add the code and Arabic name, then save.
78. Select one record for update or soft delete.

BuildingUtilityType

Purpose, UI, and Models

Defines utility types, their validity period, and whether they require rent. It displays buildingUtilityIsRent as Yes/No text and hides active. The read result is ordered ascending.

Field Contract	Operation
p01 Arabic, p02 English, p03 description, p04 start, p05 end, p06 requires rent	Insert
p01 identifier, p02 Arabic, p03 English, p06 start, p07 end, p08 rent, p10 description	Update
p01 identifier and both names readonly	Delete

The Yes/No OptionItem(1/0) list is local.

DataSet, SQL, and Permissions

- Table 0: permissions; dt1: type data, dates in yyyy-MM-dd format, and the flag as 0/1 text.
- Permissions: INSERTBUILDINGUTILITYTYPE, UPDATEBUILDINGUTILITYTYPE, DELETEBUILDINGUTILITYTYPE.
- Snapshot routing: The read path compares @pageName='buildingUtilityType' and executes Housing.buildingUtilityTypeDL; the write path uses the BuildingUtilityType casing and routes to Housing.BuildingUtilityTypeSP. Successful matching depends on the collation if it is case-sensitive.
- SQL: The gateways/permissions, Housing.buildingUtilityTypeDL, Housing.BuildingUtilityTypeSP, Housing.BuildingUtilityType, and dbo.AuditLog.

Validation and Business Rules

- The UI requires the Arabic name, start date, and rent flag; the end date is optional.
- Snapshot: The name is required; when both dates exist, the end must be after the start; active duplicates are prevented; update/delete require an active record; delete is logical, with Audit logging.
- Gap: The description alias is p04, while update expects p10.
- BuildingDetails is linked to the type by an FK. The SP does not prevent soft deletion of a type in use; associated buildings may disappear from BuildingDetailsDL because it requires the type to be active.

User Steps

79. Open Utility Types.
80. Enter the name and start date, and specify whether the utility is rented.
81. If an end date is entered, make it later than the start date.
82. Select one record for update or logical deletion.

MilitaryLocation

Purpose, UI, and Models

Defines locations/neighborhoods with a code, city, two names, coordinates, and description. The table hides the city FK, active, and Idara fields, and displays the city name from a join. Excel export is visible.

Field Contract	Operation
p01 code, p02 city, p04 Arabic, p05 English, p06 coordinates, p07 description	Insert
p01 identifier, p02 code, p03 city, p05 Arabic, p06 English, p07 coordinates, p08 description	Update
p01 identifier, with details readonly	Delete

DataSet, DDL, and SQL

- Table 0: permissions.
- dt1: Active administration-specific/general locations with an inner join to Housing.MilitaryAreaCity; a location without a matching city is not displayed.
- dt2: Active cities; GetDDLValues(..., tableIndex="2") converts them into CityOptions for Select2.
- Snapshot routing: MilitaryLocation -> Housing.MilitaryLocationDL/SP.
- SQL: The gateways/permissions, Housing.MilitaryLocationDL/SP, Housing.MilitaryLocation, Housing.MilitaryAreaCity, and dbo.AuditLog.
- Snapshot relationships: Location -> City, and Location -> dbo.Idara.

Permissions and Rules

- INSERTMILITARYLOCATION, UPDATEMILITARYLOCATION, DELETEMILITARYLOCATION.
- The UI requires the code, city, and Arabic name.
- Snapshot: The SP enforces only the Arabic name and prevents an active duplicate within the administration; update/delete require an identifier and an active record; delete is logical, with Audit logging.
- Gap: The UI marks the code and city as Required, but the SP has no corresponding explicit validation; the city FK is nullable in the table.
- The SP does not prevent soft deletion of a location associated with buildings; those buildings may disappear from reads because they require an active location.

User Steps

83. Open Locations and search the table.
84. Select the city, enter the code and Arabic name, then save.
85. Select one location for update or deletion.

BuildingDetails

Purpose and Controller Actions

Manages physical buildings. BuildingDetails(int pdf=0) reads the U query parameter as the utility type and does not display the table until it is present. The load argument is ["BuildingDetails", IdaraId, userId, HostName, U]. When pdf=1, it builds a QuestPDF report from dt1 itself without an additional DB call. There are no local POST actions; CRUD is shared.

View and UI Configuration

- SmartPageViewModel.Form: Select the utility type; sfNav issues a GET to /Housing/BuildingDetails?U={value}&S=1.
- TableDS appears only after selection and supports search, pagination, cell copying, column visibility, and a filter row.
- Rent, status, and service fields are hidden for a non-rental type.
- The report button submits pdf=1 and U, but it is currently tied to canInsertBUILDINGDETAILS.

- The View invokes SmartRenderer and sets window.UtilityTypeID.

Models and ViewModels

SmartPageViewModel, FormConfig/FieldConfig/OptionItem, SmartTableDsModel/TableColumn/TableToolbarConfig/TableAction, and ReportColumn/ReportResult/DataTableReportBuilder/QuestPdfReportRenderer. Table rows are dictionaries with p01..p21.

DataSet Result Sets and DDL

Purpose	Controller	#
Permissions	permissionTable	0
Buildings matching U and Idara with definitions, current rent, latest action, and service meters	dt1	1
Active utility types + buildingUtilityIsRent; filter DDL and rent decision	dt2	2
Active rent type with ID=1	dt3	3
Active building types within administration/general scope	dt4	4
Active locations; no administration scope in current SQL	dt5	5
Active classes within administration/general scope	dt6	6
Electricity/water/gas availability for the administration from linked service + fixed amount	dt7	7
Rent flag for selected type; unused by Controller	dt8	8

GetDDLValues reads Tables 2..6 from the same DataSet. The Yes/No service options are local.

CRUD Contract

Insert: p02 building number, p03 rooms, p04 floors, p05 area, p06 coordinates, p16 type, p07 class, p08 location, p15 utility type, p09/p10 the two telephone numbers, p13/p18 start/end dates, and p14 notes. For a rental type: p11/p12 rent type and amount. For available services: p19/p20/p21 electricity/water/gas.

Update: p01 identifier; p07 type, p08 utility, p09 location, p10 class, p11/p12 the two telephone numbers, p13/p14 rent, p15 start date, p16 notes, p17 hidden U, p18 end date, and p19..p21 services.

Permissions and Routing

- INSERTBUILDINGDETAILS: Insert and, currently, report visibility.
- UPDATEBUILDINGDETAILS: Update one record.
- DELETEBUILDINGDETAILS: Delete one record.
- Snapshot read: BuildingDetails -> Housing.BuildingDetailsDL, and @parameter01 -> @buildingUtilityTypeIDFK.
- Snapshot write: The three Actions -> Housing.BuildingDetailsSP through different positional contracts.

Procedures, Tables, Views, and Functions

- Procedures: dbo.MastersDataLoad, dbo.MastersCRUD, Housing.BuildingDetailsDL/SP.
- Direct tables: Housing.BuildingDetails, BuildingType, BuildingUtilityType, MilitaryLocation, BuildingClass, BuildingRent, BuildingRentType, BuildingDetailsMeterServices, MeterServiceTypeLinkedWithIdara, MeterServiceTypeFixedAmount, BuildingAction, BuildingActionType, and dbo.AuditLog.
- Direct View: Housing.VLastActionForBuilding.

- No Housing function is called directly by the DL/SP. The mere reference of other functions to buildings does not make them part of the verified page path.

Validation and Business Rules

- The UI requires the building number, rooms, floors, area, coordinates, definitions, and start date; the two telephone numbers, notes, and end date are optional.
- Snapshot insert: The building number, administration, and utility must be valid; the end date must be later than the start date when compared; a rental type requires a rent type and numeric amount; a duplicate active building number within the administration is prevented.
- Snapshot update: Requires an active record and prevents a duplicate building number on another record, then updates the building.
- Rent: Insert creates BuildingRent; update closes the previous active record and creates a new one, or closes the rent if the type is no longer rental.
- Service IDs are fixed: 1 electricity, 2 water, 3 gas. Insert creates the selected links; update deactivates those to be removed and adds missing ones.
- Delete is logical for the building, sets the current end date, and closes the active rent; it does not explicitly deactivate meter-service links.
- Transaction + XACTABORT; 50001 is a business error, and 50002..50004 are execution errors; Audit entries are recorded for the main operations.

Relationships and Workflow

BuildingDetails is the hub of the four definitions through FKs. It has rent history through BuildingRent, services through BuildingDetailsMeterServices, and a latest status through VLastActionForBuilding -> BuildingAction -> BuildingActionType.

User Steps

86. Select the utility type; the table is not displayed before that.
87. Review the buildings and use search/filters.
88. For insert, enter the basic data and definitions; rent and service fields appear only when their conditions are met.
89. For update, select one building and update its data, rent, and services.
90. For deletion, select a building and confirm; the reference implementation performs a soft delete and closes the rent.
91. Print the report if it appears; its visibility is currently tied to insert permission.

Important Gaps

92. The update validation block in the SP uses @Action IN('UPDATE'), while the gateway sends UPDATEBUILDINGDETAILS; therefore, that block does not execute according to the Snapshot.
93. The delete form sets p01 with MirrorName="UtilityTypeID", while the gateway expects buildingDetailsID; it may send U instead of the building identifier.
94. Update/delete set Value=buildingUtilityIsRent.ToString() in the p02 building-name field; actual population requires testing.
95. dt8 is extra and unused, and dt5 locations are not restricted by administration.
96. No FK declarations for BuildingDetailsMeterServices appear in the Snapshot despite the logical dependency.
97. The report button is governed by insert permission rather than a print permission.

ERD and Verified Relationships

See 06-Housing-Definitions-ERD.md. FKs verified in the Snapshot:

- BuildingDetails -> BuildingClass/BuildingType/BuildingUtilityType/MilitaryLocation.
- MilitaryLocation -> MilitaryAreaCity -> MilitaryArea, and MilitaryLocation -> dbo.Idara.
- BuildingRent -> BuildingDetails/BuildingRentType.
- BuildingAction -> BuildingDetails/BuildingActionType.

Consolidated SQL List

Procedures

dbo.MastersDataLoad, dbo.MastersCRUD, Housing.BuildingClassDL/SP, Housing.BuildingTypeDL/SP, Housing.buildingUtilityTypeDL, Housing.BuildingUtilityTypeSP, Housing.MilitaryLocationDL/SP, Housing.BuildingDetailsDL/SP.

Tables

Housing.BuildingClass, BuildingType, BuildingUtilityType, MilitaryLocation, MilitaryAreaCity, MilitaryArea, BuildingDetails, BuildingRent, BuildingRentType, BuildingDetailsMeterServices, MeterServiceTypeLinkedWithIdara, MeterServiceTypeFixedAmount, BuildingAction, BuildingActionType, dbo.City, dbo.Idara, dbo.AuditLog.

Views and Functions

- Housing.VLastActionForBuilding, dbo.VGetListUserPermission.
- dbo.ftUserPagePermissions is a shared table-valued function in the read gateway.
- There are no direct Housing functions in the five feature DL/SP procedures.

Live Reconciliation Update 2026-08-22

The ten Definitions procedures were verified live and match the Snapshot, as do VLastActionForBuilding and the primary FKs in the ERD. The absence of a discovered FK for BuildingDetailsMeterServices was confirmed live. An active Housing.trgBuildingActionAudit trigger was found on the actions table; it does not directly change Definitions CRUD but affects subsequent actions. No business operations or actual permission checks were executed.

Housing Operations

Note: Database update 2026-08-23: Final analysis distinguishes live FKs from inferred residence-lifecycle links and documents the live audit Trigger on BuildingAction that is absent from the Snapshot.

Shared Execution Path

Each GET page is part of the partial HousingController: it executes InitPageContext, reads userId/IdaraId/HostName, and then sends a positional array to MastersServices.GetDataLoadDataSetAsync. MastersServices converts the values into pageName, idaraID, entrydata, hostname, and parameter01..50, and resolves MastersDataLoad:getData through ProcedureMapper to dbo.MastersDataLoad. The gateway returns the permission result set first, then executes the page-specific DL.

The four Views are thin and identical: they set the title and invoke SmartRenderer. The Controller builds SmartPageViewModel, SmartTableDsModel, FormConfig, and p01..pNN fields. Writes pass through /crud/insert|update|delete to CrudController, then MastersServices.GetCrudDataSetAsync and dbo.MastersCRUD. The gateway checks the page and permission, converts the parameters into the SP contract, then returns IsSuccessful/Message to TempData and redirects.

Page Matrix

Purpose	Operations	Result sets after permissions	Action/View	Page
Custody, occupancy approval, occupancy cancellation	3	1 actual; 5 DDL calls request tables that are not returned	HousingResident(int pdf=0) / HousingProcedures/HousingResident	HousingResident
Move the building among handover states/parties	1	3	HousingHandover() / HousingProcedures/HousingHandover	HousingHandover
Extension request, financial review, insurance, approval	6	3	HousingExtend(int pdf=0,int? rowId=null) / incorrectly returns HousingResident View	HousingExtend

Purpose	Operations	Result sets after permissions	Action/View	Page
Evacuation request, penalties, finance, approval	6	1 actual; 5 DDL calls request tables that are not returned	HousingExit(int pdf=0,int? rowId=null) / HousingProcedures/HousingExit	HousingExit

The pdf modes are branches within Actions, not independent Actions. HousingExit supports searching by NID and passes it as parameter01; if no result exists, it sets a warning. HousingHandover requires query parameter U for the current status and displays status selection before the table.

HousingResident

Interface and Contract

Displays VWaitingList records within the administration, including beneficiary, building, latest action, and meter-count data. The operations are:

Key Fields	User Interface	ActionType / Permission
p01 ActionID, p02 residentInfoID, p12 CustdyRecord/notes, p18 buildingDetailsID, p21 LastActionID, p22 meterscount	Record custody and notes	HOUSINGESRESIDENTSCUSTDY
Previous + p23 letter number, p24 its date, p25 occupancy date	Final occupancy	HOUSINGESRESIDENTS
Previous + p12 cancellation reason; letter/occupancy data hidden	Cancel occupancy	CANCELHOUSINGRESIDENT

The three permissions control ShowEdit, ShowEdit1, and ShowDelete. Visible UI color states are: 45 house approval, 46 awaiting meter readings with services, 47 without services, and 2 occupant.

SQL and Rules

- Snapshot read: MastersDataLoad -> HousingResidentDL -> VWaitingList + VGetFullResidentDetails + BuildingActionType, with a subquery to MeterForBuilding. The filter is IdaraId=@idaraId and LastActionTypeID IN (45,46,47,2).
- Custody: Requires notes in the UI. The SP accepts the HOUSINGESRESIDENTSCUSTDY branch, but the shared validation is written against the unused name CustdyRecord; it then requires an existing record in status 45 and creates action 46 when the building has meters, or 47 when it does not.
- Occupancy: Prevents a beneficiary from existing in VOccupant under another administration or during evacuation, and requires an ActionID and status 47. It creates an action of type 2, calls BackfillResidentServiceBills, and creates a rent bill through fnCalcMonthlyBuildingRentByBuildingDetailsID if the function returns a value.
- Cancellation: Requires valid identifiers and a reason, locks action 45 using UPDLOCK/HOLDLOCK, and verifies that it is the latest action. It traces the parent: 38 returns to 39 (first-allocation rejection), and 40 returns to 42 (final eligibility cancellation), then creates the new action.
- There is no actual UPDATE/DELETE of BuildingAction; 'update/cancellation' operations are append-only state transitions. Bills generated in this branch are not soft-deleted.
- Every successful branch writes dbo.AuditLog. XACT_ABORT is enabled and the transaction is guarded by @@TRANCOUNT; 50001 is for business errors, 50002 for insert/identity/rent failures, and CATCH rolls back then rethrows the error.

Impact on Other Programs

Housing assignment places the beneficiary in VOccupant, on which evacuation procedures depend, and generates Bills that appear in financial auditing and electronic billing/income according to downstream consumers. This data impact is verified in the Snapshot; the UI path within IncomeSystem/ElectronicBillSystem was not traced in this scope.

HousingHandover

Interface, States, and Permissions

The page selects the current status U, then loads buildings whose latest status matches it. HousingHandoverDL returns: (1) buildings, (2) source statuses allowed for the user, and (3) allowed next statuses from the BuildingHandover table. The form sends p01 building, p02 its number, p07 next status, p10 current status, p11 notes, and p12 latest action.

The action button appears if the user has any of five responsible-party permissions: maintenance, quality, general services, housing, or execution. Within the DL, statuses are also restricted by permissionTypeRoleID=20 from dbo.VGetListUserPermission.

Snapshot security gap: MastersCRUD for the HousingHandover page has the permissionTypeNameE=@ActionType condition commented out; it checks only for any permission on the page, unlike the precise gating in the UI and DL.

Validation and Business Rules

- The only Action is HousingHandoverAction; it requires the building identifier/number, current and next statuses, and notes.
- It prevents reusing the same LastActionID as the parent of an active action.
- It creates a BuildingAction with the next status, links it to the latest action, and takes the alias/message from BuildingActionType.
- Transaction, 50001/50002, and AuditLog are present; there is no update/delete and no soft delete.
- BuildingHandover is the transition-and-permission matrix, not an execution record; execution is appended to BuildingAction.

HousingExtend

Interface and Permissions

Function	UI Permission	Submitted ActionType
Create extension request	HOUSINGEXTEND	HOUSINGEXTEND
Update request as new actions	EDITHOUSINGEXTEND	EDITHOUSINGEXTEND
Cancel request and return to residence root	CANCELHOUSINGEXTEND	CANCELHOUSINGEXTEND
Send for financial audit	Same	SENDDHOUSINGEXTENDTOFINANCE
Record precautionary insurance	Same	EXTENDINSURANCE
Approve extension	Controller searches for APPROVEEXTEND	ApproveExtend

Core fields are: approval letter and date p22/p23, start and end p24/p25, notes p26, and reason p27. Insurance displays unpaid amount p28, building rent p29, insurance amount p30 and total p31, and its document number/date/type p33/p35/p36. The DL returns the data table, extension reasons, and insurance types.

States and Rules

- Start: From status 2 it creates 48, and from 48 it creates 49; otherwise, 50001.
- Update requires status 48; it creates 50 and then a new 48, preserving history instead of updating a row.
- Cancellation requires 48; it creates 50, then copies the original residence action of type 2 from fnBuildingActionChainToRoot.
- Sending to finance requires 48, creates 51, and prevents duplicate sending.
- Insurance requires completion of finance/status 61, a reason with InsuranceRequired=1, and no previous execution; the amount must not be zero. It creates action 61 and an active ExtendInsurance record, with an Audit entry for each.
- Approval requires the appropriate financial status and completion of insurance when required by the reason, then creates status 24.
- Dates and the approval letter number/date are required, and the end date must be later than the start date.

- A comprehensive transaction is used, with 50001 for business errors, 50002 for execution errors, and AuditLog for every transition.

Verified Gaps

98. Shared validation names EditExtend/SendExtendToFinance, while the actual branches are uppercase; some branches do not benefit from it.
99. The UI sends ApproveExtend, but the permission flag is named APPROVEEXTEND; gateway matching depends on a permission named exactly like the submitted ActionType, which may prevent approval if the stored permission is uppercase.
100. The Action returns View("HousingProcedures/HousingResident", page) instead of HousingExtend; because the Views are identical, rendering may appear correct, but the path is inconsistent.
101. The EXTENDINSURANCE success message says the extension request was approved, rather than insurance was recorded.

HousingExit

Interface and Permissions

The six operations are HOUSINGEXIT, HOUSINGEXITPENALTYRECORD, EDITHOUSINGEXIT, CANCELHOUSINGEXIT, SENDHOUSINGEXITTOFINANCE, and APPROVEHOUSINGEXIT. Each has a matching flag in the Controller and appears in table buttons. Request/update use evacuation date p22 and notes; penalty uses total p40, BillsID p41, details p31, and meter count p42.

States, Rules, and Effects

- Creation is allowed only from status 2 or 24, creates 54, and then calls GenerateExitRentBills to settle exit rent.
- Update requires 54; it creates cancellation/history action 55, then a new 54, and regenerates exit rent.
- Cancellation requires 54; it calls the bill generator, deactivates penalty Bills (charge 5) and final-service bills (charges 2/3/4 with type 3), deactivates associated MeterRead records, then creates a cancellation action and copies the residence root from the chain. This is the actual soft deletion of financial/reading effects.
- Sending to finance prevents duplication and creates the next status (the UI path colors 58/59/60 within evacuation stages).
- Approval is not allowed before completion of financial auditing; it creates the approval action and shortens any active ResidentRentExemption through the evacuation date.
- Penalties are available only in the permitted status; they create an action and a Bills record with charge type 5, or deactivate the old bill and create a replacement on update; the bill is not updated in place.
- HousingExitDL reads VWaitingList, VGetFullResidentDetails, BuildingActionType, Bills, and MeterForBuilding, and returns the penalty status/bill and meter count. It supports NID and restricts by administration.
- Transactions and AuditLog are present in the branches; 50001 is for rules and 50002 for action/bill/identity failures. The success message for sending to finance incorrectly says 'extension request.'

Impact on Other Programs

It directly affects Bills, MeterRead, ResidentRentExemption, and occupancy status. Therefore, claims consumed by financial-audit and billing/income screens change, and the beneficiary can be housed again only after leaving VOccupant. Completeness of other-program paths is not claimed without tracing them in their dedicated phases.

Relationship Among the Four Operations

102. HousingHandover moves the building through readiness/handover states that feed the BuildingAction chain.
103. HousingResident converts house approval 45 into awaiting meters 46 or ready 47, then into occupant 2, and creates the initial obligations.
104. From occupant 2, HousingExtend (48...) or HousingExit (54...) can begin. Approved extension 24 remains eligible to begin evacuation.
105. Canceling extension restores the original residence action; canceling evacuation restores the residence root and cancels final financial effects; approving evacuation ends the exemption and leads to the beneficiary leaving occupancy.
106. buildingActionParentID and fnBuildingActionChainToRoot preserve the history tree; therefore, states form a ledger rather than a single status column.

Documented SQL Objects

Procedures

dbo.MastersDataLoad, dbo.MastersCRUD, HousingResidentDL/SP, HousingHandoverDL/SP, HousingExtendDL/SP, HousingExitDL/SP, BackfillResidentServiceBills, GenerateExitRentBills.

Tables

Housing.BuildingAction, BuildingActionType, BuildingHandover, MeterForBuilding, Bills, MeterRead, ExtendReasonType, ExtendInsuranceType, ExtendInsurance, ResidentRentExemption, and dbo.AuditLog. AssignPeriod appears in SQL with a commented-out result set, so it is not an actual result.

Views

Housing.VWaitingList, VGetFullResidentDetails, VGetGeneralListForBuilding, VOccupant, VbuildingWithRent, VSumBillsTotalPriceAndTotalPaidForResident, and dbo.VGetListUserPermission.

Functions

dbo.ftUserPagePermissions in the read gateway, Housing.fnCalcMonthlyBuildingRentByBuildingDetailsID, and Housing.fnBuildingActionChainToRoot.

Transactions, Errors, and Audit

The four SPs use SET XACT_ABORT ON, BEGIN TRY/CATCH, and a transaction that starts only when @@TRANCOUNT=0; success commits the local transaction, while failure rolls back then THROWS. Error 50001 represents displayable business errors; 50002 represents programming/write/identity failures. MastersCRUD treats 50001..50999 as business messages, logs unexpected errors in dbo.ErrorLog, and returns a generic message. All 16 operations write AuditLog; insurance writes two traces for the action and insurance record.

Live Reconciliation Update 2026-08-22

- The four DLs, HousingHandoverSP, and BackfillResidentServiceBills match the live definitions.
- The live HousingResidentSP enforces a valid housing date later than the latest evacuation, and verifies the action chain, assignments 38/40, and building status before occupancy.
- The live HousingExtendSP converts insurance amounts to decimal(18,2) and rejects nonnumeric values and totals <=0; the HousingExtendDL difference is the literal 40 versus 40.00.
- The live HousingExitSP expands penalty and exit billing using BillPeriod, BillPeriodType, BillChargeType, dbo.Tax, and period dates; GenerateExitRentBills differs only syntactically in THROW.
- A live Housing.trgBuildingActionAudit trigger was discovered on BuildingAction, in addition to explicit AuditLog entries.

Waiting, Assignment, and Import

Note: Database update 2026-08-23: BuildingAssign relationships to beneficiary, building, and states are used by SQL but are not live FKs; the Housing ERD clearly shows them as inferred relationships.

Status and Evidence Boundaries

- Scope: Assign, AssignStatus, OtherWaitingList, RentExemption, Residents and ResidentsPrint, WaitingList, WaitingListByResident, WaitingListMoveList, UploadExcel, and UploadLab.
- The active code in SmartFoundation.Mvc is the primary source. The scope was reconciled against live DATACORE definitions on 2026-08-22 using catalog reads only; SmartFoundation.Database files are a reference snapshot, and differences are documented in 07A-Live-Database-Reconciliation.md.
- No Upload, Import, or CRUD operations were run, and no business data was read or modified. Excel and SQL analysis was static only.
- Verified shared path: The Action reads Session/query, then MastersServices -> dbo.MastersDataLoad -> feature DL; writes go through CrudController -> dbo.MastersCRUD -> feature SP. The exceptions are UploadExcel, whose Process connects directly to SQL, and UploadLab, which saves only a file and Session data.

Page Matrix

Write/Special Operation	Result sets after permission	Display Model	Action/View	Page
INSERT/UPDATE/DELETE , PDF 1 and 2	7	Table + 3 Forms	Residents(int pdf=0) / WaitingList/Residents and ResidentsPrint	Residents
9 waiting/letter/transfer operations	8	Search + 4 tables + 9 Forms	WaitingListByResident() / thin View	WaitingListByResident
Accept/reject transfer	1	Table + 2 decision Forms	WaitingListMoveList() / thin View	WaitingListMoveList
Move to assignment list, PDF	2	Filter + table + Form	WaitingList(int pdf=0) / thin View	WaitingList
Move to occupancy procedures, PDF	3	Filter + table + Form	OtherWaitingList(int pdf=0) / thin View	OtherWaitingList
Open/close minutes, assign/replace/exclude, PDF	4	Filters + table + 5 Forms	Assign(int pdf=0) / thin View	Assign
End minutes/process status, row PDF	3	Filter + table + 2 Forms	AssignStatus(int pdf=0,int? rowId=null) / thin View	AssignStatus
Add/update/cancel	3	Search + resident table + exemption table + 3 Forms	RentExemption() / thin View	RentExemption
Upload/read, then direct TVP	No DataSet	Preview table + upload and commit Forms	Index, Upload, Process / UploadExcel/Index	UploadExcel
Physical file save only	No SQL	Session table + upload Form	Index, Upload / UploadLab/Index	UploadLab

All listed Views except ResidentsPrint are thin: they set ViewData["Title"] and invoke SmartRenderer. There are no separate domain-row classes for the eight pages; Controllers convert DataRow to Dictionary<string,object?>. The nested UploadLabRow model contains Id, OriginalName, RelativePath, and UploadedAt.

Residents and ResidentsPrint

Read Path and User Interface

Residents validates Session through InitPageContext, sends pageName/idaraId/entrydata/hostname, and runs DDL indexes 2..7 in parallel for rank, unit, marital status, nationality, gender, and resident. The reference ResidentsDL returns:

107. Resident details within the administration from Housing.VGetFullResidentDetails.
108. Active ranks.
109. Military units.
110. Active marital statuses.
111. Active nationalities.
112. Gender.
113. The administration's resident list.

SmartTableDsModel supports search, sorting, and column filtering, and adds positional aliases to populate Forms. FormConfig covers identity data, general number, Arabic and English names, rank, unit, marital status, nationality, dependent count, gender, date of birth, mobile number, and notes.

CRUD, Permissions, and Rules

Permissions: INSERTRESIDENTS, UPDATERESIDENTS, DELETERESIDENTS. The gateway matches permissionTypeNameE = ActionType, then maps pNN to ResidentsSP parameters. The reference procedure:

- INSERT: Validates core fields and uniqueness of identity/general number, creates ResidentInfo and ResidentDetails and administration relationships, and records Audit.

- **UPDATE:** Validates the record and conflicts, closes/updates active versions according to the branch, and preserves details history.
- **DELETE:** Logical deletion/deactivation while preventing invalid states; no physical deletion is verified.

Reports and Printing

pdf=1 and pdf=2 build PDFs in the Controller using QuestPDF; one is a list/report and the other a detailed form based on table data. ResidentsPrint.cshtml is an independent HTML print View that displays resident data, but it is not the only PDF path. The print button is tied in code to insert permission, a coupling that requires permission-policy review.

WaitingListByResident

Read Path and Result Sets

The page searches by NID; no result sets an error message. After the permission table, the reference WaitingListByResidentDL returns eight sets:

114. The matching resident from VGetFullResidentDetails.
115. Resident waiting-list records from VWaitingList.
116. Letters/waiting records associated with the resident.
117. Administration-transfer requests from VMoveWaitingList.
118. Waiting classes available to the administration.
119. Waiting-order types.
120. Other administrations.
121. Cancellation/end-all reasons from BuildingActionType for values 19 and 53.

The Controller displays four SmartTableDsModel instances: resident data, waiting records, letters, and transfer requests. It also calls DDL indexes 5..8 through the shared endpoint. An extra-data request with ActionType=GetWaitingListActions displays action history.

Operations and Permissions

Key Binding	Purpose	ActionType
Resident, decision, class, order type, notes	Add waiting record	INSERTWAITINGLIST
ActionID, then record data	Update record	UPDATEWAITINGLIST
ActionID; the procedure appends an action and does not erase history	Cancel record	DELETEWAITINGLIST
Resident and decision data	Add occupancy letter	INSERTOCCUBENTLETTER
ActionID and decision fields	Update letter	UPDATEOCCUBENTLETTER
ActionID	Cancel letter	DELETEOCCUBENTLETTER
New administration and request decision/reason	Request transfer of all resident lists to another administration	MOVEWAITINGLIST
Transfer-action identifier and notes	Cancel transfer request	DELETEMOVEWAITINGLIST
Reason 19 or 53 and decision data	End all resident records	DELETERESIDENTALLWAITINGLIST

The same nine names are the permissions that govern the buttons. MastersCRUD revalidates them, calls WaitingListByResidentSP, and creates Notifications in transfer branches.

Business Rules

The reference procedure uses a ledger in BuildingAction: it prevents duplicate classes, prevents insert/update when a transfer request exists, verifies that the beneficiary file belongs to the administration, and blocks work after retirement/dismissal states. Update and delete require an ActionID and an actionable record. Transfer requires a target administration and decision data; transfer cancellation requires an existing unresolved request. All branches run in a transaction, use 50001 for business errors and 50002 for programmatic failures, and record Audit.

WaitingListMoveList

WaitingListMoveListDL returns VMoveWaitingList requests directed to @idaraID. The table hides technical identifiers and displays the sending/receiving administrations and action status. The two decision Forms are:

- MOVEWAITINGLISTAPPROVE: Accept transfer.
- MOVEWAITINGLISTREJECT: Reject transfer.

They are also permissions. WaitingListMoveListSP prevents processing a request twice. Rejection appends a rejection action. Acceptance appends an acceptance action, moves the related waiting records, copies/updates resident details and administration association, updates prior-administration records, and then records Audit. The gateway creates notifications and a backlink to WaitingListByResident?NID=....

WaitingList

The page displays only general statutory classes (IDs 1,2,3,4,11 in the Snapshot). FormConfig requires a class, then WaitingListDL returns a list ordered by decision date/number and general number, plus the class DDL. The table permits MOVETOASSIGNLIST only for users with that permission.

WaitingListSP verifies that the record exists, has not been processed, and that the beneficiary is actually first in the class order within the administration; otherwise, it rejects with a business message. Success appends a move-to-assignment-list action and records Audit. pdf=1 generates a QuestPDF report for the selected list.

OtherWaitingList

Handles administration-restricted local classes other than (1,2,3,4,11). OtherWaitingListDL returns the list, local-class DDL, and active-building DDL. Its only operation, MOVETOOCUPENTPROCEDURES, moves the beneficiary directly to occupancy procedures with a selected building.

OtherWaitingListSP verifies the record, that it has not been processed, and the building's availability and valid ownership/status, then appends a building-linked action toward the occupancy path and records Audit. pdf=1 generates a class report. The same permission controls both the operation and printing in the Controller.

Assign

Result Sets and FormConfig

Accepts WaitingClassID and AssignPeriodID from the query. The reference AssignDL returns:

122. A candidate list with latest action and assignment status, linking the current minutes through a CTE.
123. Waiting-class DDL.
124. Open/active allocation minutes with creator and class data.
125. Active buildings available to the administration.

Forms and permissions: OPENASSIGNPERIOD, CLOSEASSIGNPERIOD, ASSIGNHOUSE, UPDATEASSIGNHOUSE, CANCELASSIGNHOUSE. The table applies visibility conditions based on LastActionTypeID and minutes status, and displays status pills. pdf=2 creates an allocation-minutes PDF using QuestPDF.

Assignment Rules

- Opening minutes requires a description and class and prevents a conflicting active period.
- Closing minutes requires an active period and at least one beneficiary assigned a house; it closes the period and generates linked ending actions.
- House assignment requires an existing record, active minutes, and an available building, and applies the allocation-opportunity limit. States 38/40 are used as allocation states in the reference code.
- Exclusion (CANCELASSIGNHOUSE, with the actual spelling) reverses/chains the action and does not delete the previous allocation.
- UPDATEASSIGNHOUSE creates a cancellation action for the previous building, then an allocation action for the new building.
- An AssignNote branch exists in AssignSP but is not connected to the page gateway or an active Form, so it is documented as unreachable from the current path.

AssignStatus

The page selects completed allocation minutes and displays their candidates. AssignStatusDL returns: (1) minutes rows with statuses, building, and meter, (2) DDL of closed minutes with an end date, and (3) processing states 39 and 45. Operations:

- ENDASSIGNPERIOD: End/finalize processing of the minutes.
- ASSIGNSTATUS: Select a processing status for the beneficiary and append a new action linked to the minutes and building.

pdf=2 and rowId generate a PDF form for a specific row. Both AssignStatus and ASSIGNSTATUS forms exist in live routing, but DATACORE uses the case-insensitive SQL_Latin1_General_CP1_CI_AS collation; live read and write paths were verified, so case does not break the current path. Name unification remains necessary, and actual menuNameE data was not read.

RentExemption

The page searches by the identity of a resident occupying a building. RentExemptionDL returns: (1) resident/current occupancy from VGetFullResidentDetails and VOccupant, (2) exemption records, and (3) active exemption types allowed for the administration. The first table provides resident context, while the second provides exemption history with numeric/text status (active, expired, canceled).

Permissions and operations: ADDRENTEXEMPTION, EDITRENTEXEMPTION, DELETERENTEXEMPTION. Forms bind the type, resident, start/end dates, letter number/date, building, and description. RentExemptionSP validates the resident/building, type, and periods, prevents invalid overlap, and inserts/updates the record; deletion is a logical cancellation that records CanceledBy/CanceledDate/CanceledWhy. The exemption is later consumed by BuildingRentForOneMonth, GenerateMonthlyRentBillForResident, and GenerateExitRentBills to intersect the exemption with the rent-calculation period and apply the type percentage.

UploadExcel

Upload and Preview

Index builds SmartTableDsModel dynamically from preview columns in Session. The upload Form accepts one .xls/.xlsx file up to 10 MB, with a 20 MB request limit and antiforgery. Upload revalidates extension, MIME type, and size, saves the file under a UUID name in wwwroot/uploads/excel, reads the first sheet through ExcelDataReader using the first row as headers, and stores column names and up to 200 preview rows in Session. Refresh clears Session and attempts to delete the physical file.

Commit and Import

The processing Form selects three distinct columns (p14,p15,p03). Process rereads the file, confirms the columns exist, rejects empty cells and reports up to 15 row numbers for each column, builds a TVP, calculates the file's SHA-256, and then connects directly to [Housing].[UploadExcelImportSelected3Cols]. The procedure cleans text, prevents duplicate files using UploadExcelImportLog.FileHash, inserts into Housing.UploadExcel, records row count, and returns IsSuccessful/Message/InsertedRows.

This path does not pass through MastersServices, ProcedureMapper, MastersCRUD, or the permission gateway, and the Controller has no Session identity check. A runtime gap is verified from the code and live schema:

Housing.UploadExcelRowType contains five columns (RowNo and UploadExcel1..4), while Process builds a four-column DataTable only (RowNo and 1..3). A TVP requires matching shape/ordinal, so this must be corrected before use. It was also verified live that the log table has column-name and statistics fields, while the current procedure populates only the name, hash, and count.

UploadLab

Index displays UploadLab.Rows from Session in a table (Id, original name, relative path, upload time). The upload Form accepts one PDF/XLS/XLSX file up to 10 MB, with a 20 MB request limit and antiforgery. Upload checks extension, MIME type, and size, saves a UUID-named file under wwwroot/uploads/lab, then appends metadata to Session and returns JSON.

It does not read Excel content, import to the database, provide delete/cleanup, or use a gateway or permission/session guard. MIME/extension checks do not verify the file signature, and placement under wwwroot makes the file serviceable if its URL is known; these are validation and security gaps, not operations executed during documentation.

Gateway Routing and SQL Objects

Entry routes

It was verified live that dbo.MastersDataLoad routes the eight pages to ResidentsDL, WaitingListByResidentDL, WaitingListMoveListDL, WaitingListDL, OtherWaitingListDL, AssignDL, AssignStatusDL, and RentExemptionDL, and that dbo.MastersCRUD contains write routes to the corresponding SPs and checks permissionTypeNameE = @ActionType. UploadExcellImportSelected3Cols is a direct entry point, and UploadLab has no SQL.

Procedures

dbo.MastersDataLoad, dbo.MastersCRUD, dbo.MastersExtraDataLoad, Housing.ResidentsDL/SP, WaitingListByResidentDL/SP, WaitingListMoveListDL/SP, WaitingListDL/SP, OtherWaitingListDL/SP, AssignDL/SP, AssignStatusDL/SP, RentExemptionDL/SP, UploadExcellImportSelected3Cols, plus rent-calculation procedures affected by exemption.

Tables and Types

ResidentInfo, ResidentDetails, administration-link tables, BuildingAction, BuildingActionType, WaitingClass, WaitingOrderType, AssignPeriod, AssignNote, BuildingAssign, BuildingAssignStatus, BuildingAssignCount, BuildingAssignType, ResidentRentExemption, ResidentRentExemptionType, UploadExcel, UploadExcellImportLog, AuditLog, and the Housing.UploadExcelRowType type.

Views and Functions

Direct Views: VGetFullResidentDetails, VWaitingList, VMoveWaitingList, VAssignList, VOccupant, VGetGeneralListForBuilding, VWaitingListByLetter, VWaitingListWithLetters, VWaitingListIds, and VGetListUserPermission. Housing.fnBuildingActionChainToRoot is used in chained-action paths; rent billing uses calculations/procedures that intersect exemptions with the bill period. VWaitingListxx is not treated as an active path merely because it exists.

Relationships and Workflow

The central record is not a replaceable status; it is a BuildingAction chain with LastActionID/LastActionTypeID and a parent relationship. The functional sequence is:

Residents -> WaitingListByResident -> WaitingList/OtherWaitingList -> Assign -> AssignStatus -> HousingHandover/HousingResident, with a transfer branch WaitingListByResident -> WaitingListMoveList -> new administration, and a financial branch Occupant -> RentExemption -> rent billing/exit.

- Individual pages create waiting records and letters and manage transfers.
- WaitingList enforces head-of-list priority before moving a candidate into assignment.
- Assign opens class minutes and allocates resources; AssignStatus closes the minutes cycle and determines the next status.
- OtherWaitingList bypasses assignment minutes for local classes and moves directly to the occupancy path.
- Accepting a transfer repositions the beneficiary file and lists in the target administration, rather than merely changing a display field.
- An exemption does not change housing status; it reduces the rent basis during a specified period and percentage.

Diagrams: 10-Housing-Waiting-Assignment-Workflow.md, 11-Housing-Waiting-Data-and-Gateway.md, and 12-Housing-Excel-Import-Flows.md.

Live Reconciliation Update 2026-08-22

- Of 17 feature procedures in this scope, 15 match, while AssignDL and AssignSP differ from the Snapshot. BuildingRentForOneMonth is an additional difference related to exemption impact, but it is outside the count of 17 procedures.
- The live AssignDL displays buildings in states (5,39,41) or without an action, while the Snapshot adds 42. The live AssignSP accepts a selected building in (5,39,41,42), creating a live inconsistency between selection list and write validation.
- The live Housing.VWaitingList uses roots (1,7,25) rather than (1,7) in the Snapshot.
- The live BuildingRentForOneMonth considers ExitDate and includes exits that are NULL or occur on/after the beginning of the month.

- The live UploadExcelImportSelected3Cols signature confirms four named columns and the TVP; the DataTable/TVP gap is verified live.
- Case alone does not break AssignStatus because the live collation is case-insensitive; matching against actual menuNameE data remains unverified.

7. IncomeSystem

Note: Database update 2026-08-23: No live Schema named IncomeSystem exists; all direct program objects are under Housing. The income and billing ERD documents actual FKs and distinguishes logical links.

Scope and Status

- Verification date: 2026-08-22.
- Pages: DeductListReport, FinancialAuditForExtendAndEvictions, FinancialAuditForUser, ImportExcelForBuildingPayment, ExtendInsurance.
- Source of application behavior: active code. Final SQL source: live DATACORE definitions read from sys.*. The Snapshot was used only for historical comparison.
- The live_database_reconciliation_for_conversations_1_to_7 phase was complete before work began.
- No Business Stored Procedure was executed, no business data was read, and no Upload or Import was performed.

Shared Architecture

The logical Controller is IncomeSystemController, distributed across six files. IncomeSystemController.Base reads userId, Idarald, and HostName from Session and splits the DataSet into permissionTable and dt1..dt15. The five Views are thin and invoke SmartRenderer; the only exception is JavaScript in the import View that populates months by year. There are no dedicated Models; all configuration uses SmartPageViewModel, FormConfig, FieldConfig, and SmartTableDsModel.

General path:

View -> IncomeSystemController -> MastersServies -> ProcedureMapper -> SmartComponentService -> gateway -> live Housing procedure -> live objects.

Page Summary and Live Paths

Verified Live Path	Actions / UI	Purpose	Page
Controller calls MastersDataLoad with the page name, but no live branch exists in MastersDataLoad, MastersCRUD, or MastersExtraDataLoad. Expected gateway output is permissions only; report data is not connected live.	DeductListReport(pdf); two DDL filters; PDF printing; PRINTDEDUCTLISTREPORT permission	Deduction batch report by meter-service type and period	DeductListReport
MastersDataLoad -> Housing.FinancialAuditForExtendAndEvictionsDL; writes use CrudController -> MastersCRUD -> Housing.FinancialAuditForExtendAndEvictionsSP; dynamic details through MastersExtraDataLoad.	One GET builds multiple forms and tables; FINANCIALAUDITFOREXTEND ANDEVICCTIONS, PAYMENTANDREFUNDFOREXTENDANDEXIT, and FINANCIALSETTLEMENT operations	Financial audit of extension/evacuation requests, claim review, payment/refund, and settlement	FinancialAuditForExtendAndEvictions
MastersDataLoad -> Housing.FinancialAuditForUserDL; MastersCRUD -> Housing.FinancialAuditForUserSP; bill details loaded through MastersExtraDataLoad.	One GET; FinancialAuditForUser, REVIEWCLAIMSANDPAYMENTSFORUSER, PAYMENTANDREFUNDFORUSER, and FINANCIALSETTLEMENTFORUSER	Audit a specific beneficiary's financial file and review claims and settlement	FinancialAuditForUser

Verified Live Path	Actions / UI	Purpose	Page
Read: MastersDataLoad -> Housing.ImportExcelForBuildingPaymentDL; details: MastersExtraDataLoad/GetBuildingPaymentByDeductList; processing bypasses MastersCRUD and directly calls Housing.ImportExcelForBuildingPaymentSP with a TVP.	ImportExcelForBuildingPayment, ImportExcelForBuildingPaymentUpload, ImportExcelForBuildingPaymentProcess, and GetMonthsByYear	Upload and preview a batch file, select identity/unit/general-number/amount columns, then create the batch and payments	ImportExcelForBuildingPayment
MastersDataLoad -> Housing.ExtendInsuranceDL; CrudController -> MastersCRUD -> Housing.ExtendInsuranceSP for APPROVEEXTENDINSURANCE.	ExtendInsurance(pdf); approval form; two PDF reports	Approve precautionary-insurance collection resulting from housing extension	ExtendInsurance

DataSet, DDL, and Smart UI

- Every gateway load begins with the ftUserPagePermissions table, followed by the DL-specific result sets.
- Audit pages dynamically convert DataTable columns into TableColumn, add p01.. aliases for dialogs, and build multiple SmartTableDsModel instances for the beneficiary, bills, and claims/payments.
- Application DDL sources pass through CrudController.GetDDLValues and MastersExtraDataLoad: BillChargeType claim types, payment types, beneficiaries, years and months, meter services, and their periods.
- ImportExcelForBuildingPaymentDL returns four feature sets after permissions: claim types, years from 2017 through the current year, months, and prior import batches for the administration.
- ExtendInsuranceDL returns one feature set for insurance requests where InsuranceAmountWithRemaining <> 0.
- The DeductList report requests DDL from table indexes 1 and 2, but there is no live DL routing for the page; therefore, this contract is incomplete live.

Permissions and Operations

Permissions Read by Controller	Page
PRINTDEDUCTLISTREPORT	DeductListReport
FINANCIALAUDITFOREXTENDANDEVICIONS, REVIEWCLAIMSANDPAYMENTS, PAYMENTANDREFUNDFOREXTENDANDEXIT, FINANCIALSETTLEMENT	FinancialAuditForExtendAndEvictions
FinancialAuditForUser, REVIEWCLAIMSANDPAYMENTSFORUSER, PAYMENTANDREFUNDFORUSER, FINANCIALSETTLEMENTFORUSER	FinancialAuditForUser
IMPORTEXCELFORBUILDINGPAYMENT for display; direct processing does not pass a gateway permission check	ImportExcelForBuildingPayment
APPROVEEXTENDINSURANCE	ExtendInsurance

MastersCRUD checks VGetListUserPermission using entrydata/pageName/ActionType before audit and insurance procedures. These values remain client-posted under the shared contract. Direct import relies on session context in the Controller but does not pass through the MastersCRUD filter.

Business Rules and Financial Audit

- The extension/evacuation path works on LastActionTypeID states 51 and 57; approval creates transitions 52 or 58, while preventing completion twice and recording dbo.AuditLog.

- Payment/refund and settlement validate the record and beneficiary, amount, difference between source and destination services, and sufficient balance; they then create DeductList and BuildingPayment records and an audit trail.
- User audit uses bill and payment summaries per beneficiary/service/building and performs similar operations within the user's scope.
- Insurance approval validates the record, amount, and status, updates ExtendInsurance, and creates the associated financial trace according to the Action.
- Business errors use THROW 50001 and the gateway returns their messages; procedures use transactions/TRY-CATCH and XACT_ABORT.

Excel Import Without Execution

126. Upload accepts Excel, reads the first sheet through ExcelDataReader, stores a temporary file/session token, and previews up to 20,000 rows.
127. Process rereads the file and validates four selected columns, empty cells, month/year, and claim type, then computes SHA-256.
128. It builds the live Housing.ImportExcelForBuildingPaymentRowType TVP: RowNo, IDNumber, unitID, generalNoFK, amount.
129. It directly calls Housing.ImportExcelForBuildingPaymentSP; the procedure prevents duplicate hashes, creates DeductList, cleans amounts into decimal(10,2), inserts BuildingPayment per row, then calls Housing.uspClassifyAndLinkBuildingPayments and writes UploadExcelImportLog.
130. The classification procedure links payment to the residence period from BuildingAction and sets BuildingPaymentLinkStatus; the active live trigger TRBuildingPaymentSetLinkStatus sets an initial status if none was provided.

No upload or processing step was executed during this documentation work.

Live SQL Objects

Procedures: dbo.MastersDataLoad, dbo.MastersCRUD, dbo.MastersExtraDataLoad, Housing.FinancialAuditForExtendAndEvictionsDL/SP, Housing.FinancialAuditForUserDL/SP, Housing.ImportExcelForBuildingPaymentDL/SP, Housing.ExtendInsuranceDL/SP, Housing.uspClassifyAndLinkBuildingPayments.

Primary tables: Housing.Bills, BillChargeType, BillPeriod, MeterServiceType, DeductList, DeductType, DeductListStatus, BuildingPayment, BuildingPaymentType, BuildingPaymentLinkStatus, BuildingPaymentLinkAudit, UploadExcelImportLog, ExtendInsurance, ExtendInsuranceType, BuildingAction, ResidentDetails, and dbo.AuditLog.

Verified direct Views/Functions: Housing.VWaitingList, VGetFullResidentDetails, VGetGeneralListForBuilding, VbuildingWithRent, VResidentFinancialSummary, VSumBillsTotalPriceAndTotalPaidForResident, Housing.fnBuildingActionChainToRoot, dbo.VGetFullSystemUsersDetails, and dbo.VGetListIdara.

Important live relationships: DeductList to DeductType, DeductListStatus, and BuildingPaymentType; UploadExcelImportLog.DeductListIDFK -> DeductList; and BuildingPayment.buildingPaymentLinkStatusIDFK -> BuildingPaymentLinkStatus. Several other relationships are procedural without an FK, especially resident/building/service fields.

Live Versus Snapshot

- All four DL/SP pairs exist live, and Snapshot definitions are useful for understanding history but are not authoritative.
- Live adds a payment-link classification subsystem: uspClassifyAndLinkBuildingPayments, BuildingPaymentLinkStatus, BuildingPaymentLinkAudit, and trigger TRBuildingPaymentSetLinkStatus; these are not fully represented in the old Snapshot documentation.
- Live DeductList includes ToBillChargeTypeIDFK and ExtendInsuranceIDFK, and ExtendInsurance uses expanded lengths and types compared with some Snapshot files.
- The live TVP contract matches the five-column Snapshot and the current DataTable in the Controller.
- The most important routing difference: There is no live routing for DeductListReport, so a procedure must not be inferred from its name.
- Live import invokes the classification procedure after insertion; this is a critical dependency that must not be removed when maintaining the Snapshot.

Relationship with Housing and ElectronicBillSystem

IncomeSystem is not an independent schema; it uses the Housing schema. Balance and bill sources are Bills, BillChargeType, and resident summaries, while extension and evacuation states come from VWaitingList/BuildingAction. ElectronicBillSystem writes meter readings and bills that become inputs to auditing and reporting here; IncomeSystem records collection/refund in BuildingPayment and DeductList.

8. ElectronicBillSystem

Note: Database update 2026-08-23: The count of 38 FKs in this phase was a direct-query subset, not the Schema total. Final inventory verifies 75 Housing FKs and two live Triggers; details are in the DATACORE analysis.

Scope and Evidence Level

- Pages: Meters, AllMeterRead, MeterReadForOccubentAndExit, MeterServiceTypeFixedAmount.
- MVC and application behavior is verified from active code as of 2026-08-23.
- Live DATACORE reading was completed on 2026-08-23 using PowerShell and System.Data.SqlClient, with SELECT queries restricted to sys.databases, sys.objects, sys.sql_modules, sys.parameters, sys.sql_expression_dependencies, sys.foreign_keys, and sys.triggers.
- Eight feature procedures and three gateways were verified: all exist and functionally match the Snapshot after normalizing CREATE/ALTER, whitespace, and [DATACORE] qualification. No Business Stored Procedure or DML/DDDL was executed, and no business data or secrets were read.

Shared Architecture and Application Path

ElectronicBillSystemController is one logical Controller distributed across Base and four page files. InitPageContext requires userID in Session and reads IdaraId/usersId/HostName. SplitDataSet interprets table 0 as permissions and dt1..dt9 as data. There are no domain-specific Models; pages build SmartPageViewModel, SmartTableDsModel, FormConfig, FieldConfig, OptionItem, and SmartPrintModel from dynamic DataTables. The four Views are thin and invoke SmartRenderer, with TempData messages in AllMeterRead and fixed prices.

Application-verified path:

View -> ElectronicBillSystemController -> MastersServies.GetDataLoadDataSetAsync -> ProcedureMapper -> SmartComponentService -> dbo.MastersDataLoad.

Write path: Form -> /crud/insert|update|delete -> CrudController -> MastersServies.GetCrudDataSetAsync -> dbo.MastersCRUD. The form carries pageName, ActionType, idaraID, entrydata, hostname, and p01..p50, and the gateway converts the latter positionally to parameter01..50. Gateway bindings to the Housing procedures below were verified live.

Page Summary

Verified Live SQL Path	GET/View	Purpose	Page
MastersDataLoad -> Housing.MetersDL; CRUD -> MastersCRUD -> Housing.MetersSP	Meters(int pdf=0) -> Meter/Meters.cshtml; three Smart tables	Define meters and types and link them to buildings	Meters
MastersDataLoad -> Housing.AllMeterReadDL; CRUD -> Housing.AllMeterReadSP; flexible details use forms with MeterLastBill, MeterNewBill, EditBill operations	AllMeterRead(int pdf=0); U=MeterServiceTypeID filter; MeterRead/AllMeterRead.cshtml	Open/close a monthly period and record/update/delete electricity, water, and gas readings	AllMeterRead
MastersDataLoad -> Housing.MeterReadForOccubentAndExitDL; CRUD -> corresponding SP; additional occupant DDL from result set 2	MeterReadForOccubentAndExit(int pdf=0); U=residentInfoID; same-named View	Occupancy/evacuation readings and linking their approval to building/resident actions and bills	MeterReadForOccubentAndExit
MastersDataLoad -> Housing.MeterServiceTypeFixedAmountDL; CRUD -> corresponding SP	One GET -> Services/MeterServiceTypeFixedAmount.cshtml	Manage a fixed amount per service, administration, and validity date	MeterServiceTypeFixedAmount

Meters

Result Sets and DDL

After permissions, the live MetersDDL returns eight feature sets:

131. dt1: Active meters with service type, meter type, calculation method, price/fixed amount, and the first MeterRead reading of type 4.
132. dt2: Active meter types with service, calculation method, and price or fixed amount.
133. dt3: Meter-to-building links from MeterForBuilding and VGetGeneralListForBuilding.
134. dt4: Meter services enabled for the administration through MeterServiceTypeLinkedWithIdara.
135. dt5: Meter types available to the administration and service.
136. dt6: Active meters not currently linked to a building.
137. dt7: Active buildings with a meter-count summary by service.
138. dt8: Active calculation methods from MeterCalculateType.

The Controller requests DDL indexes 5, 4, 7, and 8; it also uses DDLFiltered to filter meter type by service and meter by type. These numbers are positional DataSet contracts, not stable names.

Operations and Permissions

Snapshot Effect	ActionType	Operation
Insert MeterType, then MeterServicePrice when calculation method is 1, or MeterTypeFixedAmount for the fixed type	INSERTNEWMETERTYPE	Add meter type
End/update the definition and dependent prices according to calculation method	UPDATENEWMETERTYPE	Update meter type
Logical deactivation after association checks	DELETENEWMETERTYPE	Delete meter type
Insert Meter and initial MeterRead reading	INSERTNEWMETER	Add meter
Update meter data	EDITNEWMETER	Update meter
Logical deactivation while preventing deletion when dependencies exist	DELETENEWMETER	Delete meter
Insert MeterForBuilding and link reading	LINKMETERTOBUILDINGS	Link to building
End the link and record the unlink reading	UNLINKMETERTOBUILDINGS	Unlink

The Controller reads only four permissions: INSERTNEWMETER, INSERTNEWMETERTYPE, UPDATENEWMETERTYPE, DELETENEWMETERTYPE. Meter update/delete and link/unlink buttons exist in configuration, but no corresponding independent booleans appear in the permission loop; this UI gap requires testing. The live gateway checks permissionTypeNameE = ActionType for all eight operations, and the live SQL_Latin1_General_CP1_CI_AS collation is case-insensitive.

Business Rules and Relationships

- The Arabic meter-type name must not be duplicated within the administration.
- The calculation type determines whether pricing uses tiers/service price or a fixed amount on the meter type.
- A meter belongs to MeterType, and the type belongs to MeterServiceType and MeterCalculateType.
- MeterForBuilding implements the temporal relationship between a meter and BuildingDetails; the query has one active link, while unlinking preserves history.
- The initial/link reading is stored in MeterRead and becomes the basis for subsequent consumption.
- IdaraId restricts meters, types, buildings, and services. Values and context reach CRUD from client-hidden fields under the shared contract.

The page has no active printing; print/PDF flags are disabled despite report imports.

AllMeterRead

Result Sets and Working Interface

Live AllMeterReadDL takes meterServiceTypeIDFK in addition to context and returns four feature result sets: period and meters, period bills, administration services, and meters without a bill in the period. The Controller infers the current period from dt1.billPeriodID and offers opening when none exists and closing when one exists. It uses dt4 as the meter list and divides display into electricity/water/gas tables according to the selected service. Permissions are not enforced for page access because the denial block is commented out, but operation booleans control buttons.

Live sources include BillPeriod, BillPeriodType, MeterServiceType, Meter, MeterType, MeterForBuilding, MeterRead, Bills, FNEligibleMeters, VGetListMeterSlidesPrice, and VGetListMetersLinkedWithBuildings.

Operations and Permissions

- Period management: OPENMETERREADPERIOD, CLOSEMETERREADPERIOD.
- Electricity: READELECTRICITYMETER, EDITELECTRICITYMETER, DELETEELECTRICITYMETER.
- Water: READWATERMETER, EDITWATERMETER, DELETEWATERMETER.
- Gas: READGASMETER, EDITGASMETER, DELETEGASMETER.

Controller, Form, and live gateway names match exactly. The live procedure groups the three read operations into shared branches, and likewise update and delete. Open creates a monthly period for a service/administration; close requires complete readings and finalizes the period. Read validates the period, meter, previous/maximum value, and duplication; it creates MeterRead and then consumption Bills according to calculation type, prices/tiers, and tax. Update rebuilds the associated reading/claim, while delete logically deactivates the reading and bill. Business errors use THROW 50001, and transactions use XACT_ABORT/TRY-CATCH.

pdf=1 generates a landscape QuestPDF report from the DataTable, but internal export headings were copied from 'beneficiary extension,' and some filenames/titles do not reflect meters; this is a report-quality gap. No visual test or PDF execution occurred in this phase.

MeterReadForOccubentAndExit

Workflow and Result Sets

The user selects an occupant from dt2, then dt1 displays building meters linked to the current Housing action. Fields combine ActionID, resident/identity/general number, action decision, waiting class/order, buildingDetailsID, meter/service/type, previous and maximum readings, meterReadID, BillsID, LastActionTypeID, AssignPeriodID, LastActionID, buildingActionRoot, and reading-completion status.

The live DL returns the meters/action result set and occupant DDL, joining VWaitingList, VGetFullResidentDetails, BuildingAction, MeterForBuilding, Meter/MeterType/MeterServiceType, MeterRead, and Bills. This bridges ElectronicBillSystem to Housing: the occupancy or evacuation action determines the building, resident, root, and date, while linked meters determine required readings.

Operations and Permissions

Purpose	Submitted ActionType	Read Permission
Enter reading	METERREADFOROCCUBENTANDEXIT	MeterReadForOccubentAndExit (case-insensitive in C#)
Update reading before approval	UPDATEMETERREADFOROCCUBENTANDEXIT	UpdateMeterReadForOccubentAndExit
Approve all readings and create the next Housing transition	Same	APPROVEMETERREADFOROCCUBENTANDEXIT

Here, the first two permission names in the Controller differ in casing/wording from the upper ActionTypes. The live gateway requires permission to equal ActionType, but the live collation is case-insensitive; case alone does not cause rejection. The full-name difference still requires actual permission data to prove its impact.

The live procedure distinguishes occupancy root from evacuation, prevents a reading below the previous one except within wrap/max-read logic, prevents duplicates, and creates or updates MeterRead and Bills. Approval requires

completion of all building meters, then creates a new BuildingAction and writes dbo.AuditLog. Many resident/building/meter/bill relationships are procedural and not all have FKs.

The page has pdf=1 for an official landscape report, but table print/export buttons are disabled; actual link reachability and report quality require verification.

MeterServiceTypeFixedAmount

Result Sets and DDL

After permissions, the live DL returns:

139. Active fixed prices for the administration with MeterServiceType name and start/end dates.
140. Services without an active fixed amount in the administration, consumed by GetDDLValues at index 2.

Operations and Rules

- INSERTSERVICEFIXEDAMOUNT: Requires the service, amount, and start date; prevents a duplicate active record; inserts the price; and creates MeterServiceTypeLinkedWithIdara if absent.
- EDITSERVICEFIXEDAMOUNT: Does not update the value in place; it closes the old record with an end date and inserts a new active version, preserving history.
- DELETESERVICEFIXEDAMOUNT: Soft delete by clearing active and setting the end date.
- The three Controller permissions exactly match the upper ActionType names: INSERTSERVICEFIXEDAMOUNT, EDITSERVICEFIXEDAMOUNT, DELETESERVICEFIXEDAMOUNT.
- The live definition confirms an audit defect: the delete branch writes RecordID = ISNULL(@NewID,0), although delete never populates @NewID, so it usually records 0. JSON notes also use a copied buildingClassActive key rather than the domain name.

The general export button is configured internally, but ShowExportPdf=false; the page has no actual dedicated report.

Integration with Housing and IncomeSystem

- Housing owns the actual schema: BuildingDetails, BuildingAction, resident Views, and MeterForBuilding. Occupancy and evacuation determine who bears consumption and during which period, while approval of entry/exit readings completes the building-state transition.
- ElectronicBillSystem converts readings into Bills according to BillPeriod, service, calculation method, prices/tiers, fixed amount, and tax.
- IncomeSystem consumes Bills, BillChargeType, and resident summaries in auditing and settlement, then records collection/refund in DeductList and BuildingPayment. Therefore, updating/deleting a reading after a payment exists can change a financial basis; live procedures inspect links, while testing actual impact is outside catalog-read scope.
- MeterServiceTypeFixedAmount prices a service at the administration level, while MeterTypeFixedAmount applies to a meter type and its calculation method; confusing them changes billing scope.

Live Versus Snapshot

Final live classification:

- Live verified and matching (11): dbo.MastersDataLoad, dbo.MastersCRUD, dbo.MastersExtraDataLoad, Housing.MetersDL/SP, Housing.AllMeterReadDL/SP, Housing.MeterReadForOccubentAndExitDL/SP, Housing.MeterServiceTypeFixedAmountDL/SP.
- Live verified with differences: None within scope.
- Missing from live: None within scope.
- Live signatures: Gateways have 24, 55, and 15 parameters; MetersDL/SP have 4 and 29; AllMeterReadDL/SP have 5 and 30; MeterReadForOccubentAndExitDL/SP have 5 and 30; and MeterServiceTypeFixedAmountDL/SP have 4 and 8.
- Live result sets: MetersDL has eight, AllMeterReadDL four, MeterReadForOccubentAndExitDL two, and MeterServiceTypeFixedAmountDL two, plus a permission result set for each page; total 20 result sets across the four pages.

- The expanded scope yielded 284 catalog dependencies and 38 FKs. One active live trigger was found: Housing.trgBuildingActionAudit on BuildingAction.

Verified Application-Level Deviations

141. Page-access enforcement is commented out in AllMeterRead.
142. Meters reads fewer permissions than the eight configured operations.
143. Mixed-case permission names differ from ActionTypes on the occupancy/evacuation page.
144. AllMeterRead PDF titles were copied from the extension page.
145. Fixed-price delete Audit uses an unset ID and a JSON key from BuildingClass.
146. Security context values in CRUD remain client-posted under the general contract.

9. Home and Login

Note: Database update 2026-08-23: The six Home/Login procedures were included in the final gateway/shared inventory, with no new drift found. See the DATACORE analysis.

Scope and Sources of Truth

- Verification date: 2026-08-23.
- Includes LoginController and the login, password-change, and logout pages; HomeController.Index and Privacy; menu, navigation, Session, and shared components.
- Active code is the source of application behavior, and live DATACORE definitions are the final SQL source. The Snapshot was used only for comparison.
- sys.procedures, sys.sql_modules, sys.parameters, and sys.sql_expression_dependencies were read through PowerShell and System.Data.SqlClient within try/catch.
- No actual account login was performed, no user data was read, no password was changed, no Business Stored Procedure or write/DDDL was executed, and no connection value or default password was displayed.

Executive Summary

Authentication is Session-based, not ASP.NET Core Identity: the login page sends identity and password to GetSessionInfoForMVC, then the Controller creates Session keys on a successful result and navigates to /Home/Index. There are no active Claims or authentication cookies. Logout clears Session.

Home is locally protected through InitPageContext. Before loading it, active code executes Housing.GenerateMonthlyRentBills for the previous month; this is a write side effect within GET and was not executed during documentation. It then reads MastersDataLoad for page Home; the live gateway returns page permissions and routes to ChartsDL, not HomeDL. HomeDL exists live and matches the Snapshot, but it is not the visible active path.

Login

Displaying the Login Page

GET /Login/Index has [AllowAnonymous] and ResponseCache(NoStore=true), always clears Session, reads the previous identity from query u and a message from mt/msg, and converts logout=1 to an inactivity-expiration message and logout=2 to a success message. It also adds cache-prevention headers.

Views/Login/Index.cshtml is standalone without a Layout and displays an identity field with maximum length 10, password field, password-visibility button, and Remember Me checkbox. The form generates an antiforgery token and posts to CheckLogin. login-page.js restricts identity to digits in the browser and stores only the identity in localStorage when Remember Me is selected; it does not store the password.

Validation and Navigation

147. POST /Login/CheckLogin has [AllowAnonymous] and [ValidateAntiForgeryToken].
148. It rejects empty fields, then sends the trimmed identity, password, and Request.Host.Value to MastersServices.GetLoginsDataSetAsync.
149. ProcedureMapper resolves auth:sessions to dbo.GetSessionInfoForMVC, and DataEngine executes it as a stored procedure.
150. ExtractAuth converts the first row into AuthInfo. Success requires a nonempty userId and usersActive != 0.
151. After creating Session, the Controller sets TempData[Success|Warning|Info] according to usersActive, then navigates to Home/Index.

Failure redirects to Login with message type, message text, and the previous identity. Connection and result-processing failures include ex.Message in the query, risking internal-detail disclosure. No CancellationToken is explicitly connected to service execution despite being received by the Action.

Live GetSessionInfoForMVC

Status: Live verified and matching Snapshot. Live signature: @NationalID nvarchar(20), @Password nvarchar(200), @hostName nvarchar(200); lengths here are character units after interpreting nvarchar maxlength.

Behavior verified from the live definition:

- Selects the latest active user within the start/end date range.
- Selects the latest active password record and compares SHA2-256(salt + password bytes) with the stored value.
- Requires active user details and an administrative structure containing Idara and Department.
- Returns identity, organizational and administrative context, display data, logos, ChangedPassword status, and an Arabic message.
- Distinguishes no profile/active account, incorrect password, no password, and administrative-structure defect states.

@hostName exists in the contract but is unused in the live definition body.

Session Keys

Keys Written on Success	Category
usersID, nationalID, GeneralNo	Identity
fullName, usersActive, ChangedPassword	User
OrganizationID/Name, IdaraID/Name, DepartmentID/Name, SectionID/Name, DivisonID/Name, DeptCode	Organization
AdminTypeID, AdminTypeName	Administration
photoBase64, ThameName, OrganaiztionLogo, IdaraLogo	Presentation
HostName, LastActivityUtc	Runtime

The menu adds MENUTREE later. General configuration: memory-backed Session, 10-minute idle timeout, and a cookie with HttpOnly, Secure Always, and Essential. Read names are inconsistent in some locations (usersActive/useractive, usersID/UserId).

Password Change

POST /Login/ChangePassword derives usersID from Session but has no antiforgery attribute. It rejects missing Session, empty fields, and lengths below 8, then the service calls dbo.ReSetUserPassword with action CHANGEUSERPASSWORD and three actual values: user, old password, and new password; remaining context parameters in the service are null.

The live ReSetUserPassword definition matches the Snapshot and has eight parameters. The change branch:

- Verifies that the user is active and within the validity period.
- Verifies the old password using salt and SHA2-256.
- Enforces server-side: at least 8 characters, one digit, and at least one English letter.
- Prevents the new password from equaling the current password, then deactivates old active passwords.
- Creates a random 32-byte salt through CRYPT_GEN_RANDOM, adds a new hash, and sets ChangedPassword=1.

JavaScript additionally requires uppercase, lowercase, a digit, and matching confirmation. Upper/lowercase is therefore UI-only policy and can be bypassed by a direct request; verified server-side policy is 8 + digit + English letter, with no verified special-character, history, expiry, or breached-password check.

On success, Session is updated to ChangedPassword=1. The interface logs the user out afterward to sign in again. If the initial value is 0, Home and Sidebar display a mandatory-change dialog and disable navigation links through JavaScript; this is not a general server-side barrier.

The administrative RESETUSERPASSWORD branch in the live definition still generates a fixed default password inside the procedure, deactivates the previous password, creates a new salt/hash, and sets ChangedPassword=0. The value is not stated. This closes the 'did live differ?' gap negatively and confirms the risk remains.

Logout and Inactivity

- GET /Login/Logout clears Session and returns Login with logout=2. There is no active SignOutAsync.
- POST /session/logout clears Session and returns JSON success; it has no antiforgery.

- session-guard.js counts browser activity, warns after 8 minutes, logs out after 10, and sends keepalive every minute when active.
- POST /session/keepalive updates LastActivityUtc even without checking Session identity.

Historical state at analysis time 2026-08-23: SessionGuardMiddleware existed but was not registered in Program.cs. In version 1.0.0, it became registered after Session, Authentication, and Authorization; this paragraph is retained to document the prior state.

Home

Index

GET /Home/Index prevents caching and calls InitPageContext; absence of usersID redirects to Login with logout=1. It then:

152. Executes GenerateMonthlyRentBillsAsync procedure [Housing].[GenerateMonthlyRentBills] for the previous month using Session context. Exceptions are logged and swallowed, so the page continues. Performing a write from GET increases request latency/risk and violates no-side-effect semantics; it was not executed during documentation.
153. Sets ControllerName=Home and PageName=Home.
154. Sends Home, Idarald, usersId, HostName, usersId to MastersDataLoad.
155. Splits results: table 0 Home permissions, table 1 chart names, table 2 housing statistics.
156. Builds SmartPageViewModel titled 'Dashboard' and renders SmartRenderer. Actual Charts configuration is commented out, so results are not currently bound to Chart cards.

Privacy

GET /Home/Privacy returns a static English placeholder View without InitPageContext, [Authorize], or data. It is therefore available through routing without Session while no general guard exists.

Live Home SQL

Actual Role	Live Status	Object
Returns ftUserPagePermissions first; the Home branch executes ChartsDL.	Matches Snapshot	dbo.MastersDataLoad
Returns user-specific chart names, then distribution/percentages of Idara housing states.	Matches Snapshot	dbo.ChartsDL
Returns only user-specific chart names; exists live but is not called by the discovered active Home path.	Matches Snapshot	dbo.HomeDL

The end-date condition in HomeDL/ChartsDL uses < GETDATE() with OR end IS NULL, accepting records that expired in the past and rejecting future expiry dates; this is inverted from the usual pattern and requires business review.

Menu, Navigation, and GetUserMenuTree

Layout invokes MenuItemsViewComponent on every page using Layout. The component reads usersID from Session, calls MastersServices.GetUserMenuTree, converts JSON to MenuItem, builds the hierarchy, stores it in MENUTREE, then renders SidebarNavbar.

Live GetUserMenuTree matches the Snapshot and has signature @UsersID int. It identifies menus with an active grant directly to the user or through Role, DSD, Distributor, or the general grant, including active dates; then includes all ancestor nodes and creates a first-level node for each Program. The result is ordered by SortKey/LevelNo and includes HasPermissionForUser to distinguish granted nodes from ancestor nodes.

The ViewComponent orders by MPSerial and builds children using parents/ParentMenuIDFK. SidebarNavbar uses the tree for rendering and Breadcrumb and also prints it as JSON in the page. Programs appear only if returned by the procedure through a grant branch or required as an ancestor of an authorized path.

Verified privacy risk: MastersServices.GetUserMenuTree writes the full menu JSON to console and Information log, then also logs the first row. This does not necessarily contain secrets, but discloses program structure and permissions available to the user in logs.

Shared Components and Relationship with Dashboard

- Layout: Menu, Breadcrumb, Toastr, Session guard, Notification, and table/form/chart assets.
- MenuItemViewComponent and SidebarNavbar: Navigation, user data, password change, and logout.
- SmartRenderer: Receives the Home page, but the current model contains no actual Charts/Form/Table.

A fixed /Dashboard link exists in the Sidebar logo, and the excluded Dashboard/Index View resembles Home in displaying password change and SmartRenderer. The Dashboard module itself was not documented, and the link does not prove Home redirects to it; the post-login path is exclusively Home.

Security: Confirmed and Remaining

Live Findings That Closed Verification Gaps

- Live hashing: Random 32-byte salt + SHA2-256, matching the Snapshot.
- Administrative reset: The fixed default value still exists live; the risk was not closed.
- Login validity: User, details, active period, structure, and password are required live.
- Session: Its keys and configuration are verified from code; it does not rely on server-side SQL Session beyond the login result.
- ftUserPagePermissions was previously verified live and matching, and live MastersDataLoad invokes it before Home.
- GetUserMenuTree applies the actual grant network and returns ancestors/programs only for paths with grants.

Differences from Snapshot

There is no drift in the six procedure definitions read for this phase. Important differences are not Live-versus-Snapshot but usage/code-versus-schema:

- Active Home routes to ChartsDL, not HomeDL.
- The Controller expects dt2 for the permissions map although Home results end at table 2.
- Upper/lowercase policy in JavaScript is stricter than live SQL.
- @hostName in the login procedure is unused.

Diagrams and Evidence

- Authentication and Session flow
- Permission and menu flow
- Home and Login user guide

10. Live Database

Database and Schemas

Connection to DATACORE succeeded. Direct objects for the included programs are actually distributed across two schemas:

Live Role Within Scope	Schema
Identity, users, administrative structure, ControlPanel, permissions, notifications, auditing, and shared application gateways.	dbo
Housing and waiting lists, income and financial audit, meters, and electronic billing.	Housing

There are no live schemas named IncomeSystem or ElectronicBillSystem. These names represent two MVC programs, but their SQL objects reside under Housing and are reached through gateways in dbo.

Documented Object Counts

Documented Total	Selected Direct dbo	Live Housing	Type
118	32	86	Tables
44	4	40	Views
76	15	61	Stored Procedures
34	1	33	Functions
2	0	2	Triggers
274	52	222	Total

The selected dbo set is not an inventory of all dbo objects: objects for deferred programs, administrative tools, AI, and database diagrams were excluded. Housing was inventoried fully because the four programs Housing/Income/ElectronicBill use the same schema and have intertwined dependencies.

Important Tables and Column Purposes

ControlPanel, Identity, and Permissions

Key Columns and Purpose	PK	Table
Program names, link, order, and active status.	programID	dbo.Program
programIDFK, English name matching pageName, link, logical parent, active status, and page level.	menuID	dbo.Menu
Permission distribution unit; optionally linked to DSD and Role.	distributorID	dbo.Distributor
Links Menu to Distributor/Role/User with isDenied and active status.	menuDistributorID	dbo.MenuDistributor
permissionTypeNameE is the action name read by Controllers and checked by the gateway.	permissionTypeID	dbo.PermissionType

Key Columns and Purpose	PK	Table
Links Distributor to permission type, validity period, and authorization level.	distributorPermissionTypeID	dbo.DistributorPermissionType
Grant to User/Role/Distributor/DSD with Idara and active period.	permissionID	dbo.Permission
Account identity, National ID, active status, validity period, and password-change state.	usersID	dbo.Users
usersIDFK, name, and supporting organizational/personal data.	usersDetailsID	dbo.UsersDetails
usersIDFK, salt/hash, active status, and period. No value was read.	usersPasswordID	dbo.UsersPassword
Assigns a user to a Distributor with an active period.	UDID	dbo.UserDistributor
Combines Organization/Idara/Department/Section/Division. Some links are logical without actual FKs.	DSDID	dbo.DeptSecDiv
Title, text, link, period, active status, and Idara.	NotificationId	dbo.Notifications
FK to notification, user, and read state.	UserNotificationId	dbo.UserNotifications
Table, action, RecordID, executor, notes, and time.	AuditID	dbo.AuditLog
Error, severity, state, procedure, and input context.	ErrorLogID	dbo.ErrorLog

Other shared tables documented in this scope: Role, DistributorType, ProgramDistributor, UsersAuthType, Organization, Idara, Department, Section, Division, City, Tax, Gender, MaritalStatus, MilitaryUnit, Nationality, and Rank.

Housing, Residence, and Waiting

Purpose	Core Tables	Group
Building classification, location, type, utility, and core data.	BuildingClass, BuildingType, BuildingUtilityType, MilitaryArea, MilitaryAreaCity, MilitaryLocation, BuildingDetails	Definitions
Beneficiary identity, details, contacts, and state.	ResidentInfo, ResidentDetails, ResidentContactInfo, ResidentContactType, ResidentStatus	Beneficiary
Handover/occupancy/extension/evacuation ledger, states, and sources.	BuildingAction, BuildingActionType, BuildingActionSource, BuildingStatus, BuildingHandover	Building Lifecycle
Waiting lists, nomination, assignment, and status.	BuildingAssign, BuildingAssignStatus, BuildingAssignType, AssignPeriod, AssignNote, WaitingClass, WaitingOrderType	Waiting and Assignment
Exemptions, extension insurance, and approval.	ResidentRentExemption, ResidentRentExemptionType, ExtendInsurance, ExtendInsuranceType, ExtendReasonType	Exemption/Extension
Rent definition, bills, settlements, and actual payment.	BuildingRent, BuildingRentType, RentBills, RentBillsAdjustment, BuildingRentActualPayment	Rent

...ID columns are PKs in 79 of 86 Housing tables. ...Active fields, start/end dates, IdaraIDFK, entryDate, entryData, and hostName recur to control scope, activity, and auditing. The absence of a PK in seven tables does not imply they are unimportant, but it is a structural gap requiring operational review.

IncomeSystem and ElectronicBillSystem

Key Columns/Role	Core Tables	Group
Claim/deduction number, period, type, status, amount, and bill linkage.	DeductList, DeductType, DeductListStatus, BillDeductList, BillDeductAction, BillsDeductListDetails	Financial Audit
Raw/historical payment, classification, destination, link status, and link audit.	BuildingPayment, BuildingPayment, BuildingPaymentType, BuildingPaymentDestaination, BuildingPaymentLinkStatus, BuildingPaymentLinkAudit	Payment and Linking
Meter number, service type, building link, reading, period, and reading type.	Meter, MeterType, MeterForBuilding, MeterRead, MeterReadType	Meters
Service, tiers, fixed price, validity period, and administration scope.	MeterServiceType, MeterServicePrice, MeterSlide, MeterServiceTypeFixedAmount, MeterServiceTypeLinkedWithIdara, MeterTypeFixedAmount	Pricing
Bill, period, charge type, payment, and original/replacement bill relationship.	Bills, BillType, BillPeriod, BillPeriodType, BillChargeType, billPayment, billPaymentType	Billing
Staging for selected columns and hash/file/count log with DeductList linkage. Definitions here store no actual data.	UploadExcel, UploadExcelImportLog	Import

Primary Keys, Foreign Keys, and Constraints

- Housing: 79 live PKs and 75 live FKs.
- The 32 direct dbo tables: 32 live PKs and 43 live FKs.
- All read FKs are enabled. One untrusted FK (is_not_trusted=1) was found on Housing.BuildingPayment -> Housing.BuildingPaymentType; historical data must not be assumed to have passed validation.
- ON DELETE and ON UPDATE actions in the read FK set are NO_ACTION; no cascade is verified in this set.

Key Live-Verified FK Relationships

- Menu.programIDFK -> Program.programID.
- MenuDistributor -> Menu/Distributor/Role/Users.
- DistributorPermissionType -> Distributor/PermissionType.
- Permission -> DistributorPermissionType/Users/Role/Distributor/DeptSecDiv.
- UserDistributor -> Users/Distributor.
- UsersDetails.usersIDFK -> Users.usersID and UsersPassword.usersIDFK -> Users.usersID.
- BuildingDetails -> BuildingClass/BuildingType/BuildingUtilityType/MilitaryLocation.
- BuildingAction -> BuildingDetails/BuildingActionType/ResidentInfo/BuildingActionSource/BuildingPaymentType/BuildingStatus.
- MeterForBuilding -> Meter/BuildingDetails; and MeterRead -> Meter/BillPeriod/MeterReadType.
- BillPeriod -> BillPeriodType -> MeterServiceType.
- RentBills -> ResidentInfo/BuildingDetails; and RentBillsAdjustment -> RentBills/ResidentInfo/BuildingDetails.
- ResidentDetails -> ResidentInfo and shared lookups for rank, unit, gender, nationality, marital status, and administration.
- ResidentRentExemption -> ResidentInfo/ResidentRentExemptionType.

Inferred Relationships That Are Not Actual Constraints

These relationships are used in columns or live SQL, but no FK appeared in the catalog; they must therefore not be drawn as constraints:

- Menu.parentMenuIDFK -> Menu.menuID: an inferred tree relationship, not an actual FK.
- DeptSecDiv.OrganizationIDFK/IdaraIDFK: an inferred organizational relationship; actual existing FKs link only Department/Section/Division.
- Permission.IdaraIDFK -> Idara and Permission.InIdaraID -> Idara: inferred, not FKs.
- BuildingDetailsMeterServices -> BuildingDetails/MeterServiceType: inferred from procedures and names, without a live FK.
- BuildingAssign to resident/building/status/type: inferred from procedure definitions/columns and absent from the live FK list.
- MeterRead.residentInfoIDFK, buildingDetailsID, and buildingActionIDFK: logical links without actual FKs.
- ResidentRentExemption.buildingDetailsIDFK -> BuildingDetails: inferred, not a constraint.
- ExtendInsurance to BuildingAction/ResidentInfo/BuildingDetails: inferred, not actual constraints.
- DeductList to BillChargeType/ExtendInsurance/Idara: inferred despite the FK suffix.

Views and Their Dependencies

Direct Shared Views

dbo.VGetListUserPermission, dbo.VGetListUsersInDSD, dbo.VGetFullStructureForDSD, and
dbo.VGetFullSystemUsersDetails.

Views in Housing (40)

VAllResidentsWithMoreThanOneSera, VAssignList, VbuildingWithRent, VFirstMeterReadForAllMeter, VGetFullResidentDetails, VGetGeneralListActionForResident, VGetGeneralListForBuilding, VGetGeneralListForBuildingWithRent, VGetGeneralListForResident, VGetListAllMetersLastAndBeforeLastReadWithWrap, VGetListAllMetersLastRead, VGetListMeterSlidesPrice, VGetListMetersLinkedWithBuildings, VGetListMetersLinkedWithBuildings20260228, VLastActionForBuilding, VLastActionForResident, VLastActionTypeForBuilding, VLastMeterReadType1, VMeterLastBill, VMetersDetails, VmoreOneActionforType1, VMoveWaitingList, VOccupant, VOccupant20260204, VOccupantExtend, VResidentBillsFinancialSummary, VResidentFinancialSummary, VResidentLastBill, VResidentPreviosBill, VResidentServiceAverageBills, VSumBillsTotalPaidByResident, VSumBillsTotalPriceAndTotalPaidForResident, VSumBillsTotalPriceForResident, VSumUnallocatedPaymentsForResident, VUnresolvedBuildingPayments, VWaitingList, VWaitingListIds, VWaitingListByLetter, VWaitingListWithLetters, and VWaitingListxx.

View dependencies are documented from sys.sql_expression_dependencies. Key examples:

VGetFullResidentDetails depends on ten objects, VGetGeneralListForBuildingWithRent on 11, VMetersDetails on six, and VWaitingList and VWaitingListWithLetters on seven each. Dated-copy names and VWaitingListxx are live but do not prove application usage.

Stored Procedures and Transactions

Gateways and Shared dbo (15)

Role	Parameter Count	Procedure
Permission result set, then read routing.	24	MastersDataLoad
Permission/action routing, then write through the feature SP.	55	MastersCRUD
Helper/DDDL load.	15	MastersExtraDataLoad
Authentication and Session-context construction.	3	GetSessionInfoForMVC
Program and menu tree according to permissions.	1	GetUserMenuTree

Role	Parameter Count	Procedure
Load Home/Charts.	5 each	HomeDL, ChartsDL
CRUD for program, menu, and page definitions.	39	PagesManagmentSP
CRUD for permission grants.	19	PermissionSP
User data and DDL.	4	UsersDL
User CRUD.	30	UsersSP
Change/reset password.	8	ReSetUserPassword
Create/update password.	6	SetUserPassword
Create notification and determine recipients.	15	NotificationsCreate
Update user-notification state.	3	NotificationsCRUD

Housing/Income/ElectronicBill Procedures (61)

Live Procedures	Group
BuildingClassDL/SP, BuildingTypeDL/SP, buildingUtilityTypeDL, BuildingUtilityTypeSP, MilitaryLocationDL/SP, BuildingDetailsDL/SP.	Housing Definitions
HousingResidentDL/SP, HousingHandoverDL/SP, HousingExtendDL/SP, HousingExitDL/SP, GenerateExitRentBills, GenerateMonthlyRentBills, GenerateMonthlyRentBillForResident, BackfillResidentServiceBills, BuildingRentForOneMonth, BuildingRentForOneMonth2026-05-12.	Housing Procedures
ResidentsDL/SP, WaitingListByResidentDL/SP, WaitingListMoveListDL/SP, WaitingListDL/SP, OtherWaitingListDL/SP, AssignDL/SP, AssignStatusDL/SP, RentExemptionDL/SP, UploadExcelImportSelected3Cols.	Waiting/Assignment
ExtendInsuranceDL/SP, FinancialAuditForExtendAndEvictionsDL/SP, FinancialAuditForUserDL/SP, ImportExcelForBuildingPayment, ImportExcelForBuildingPaymentDL/SP, uspClassifyAndLinkBuildingPayments, uspClassifyAndLinkBuildingPaymentsV2, uspLinkBuildingPaymentManually.	IncomeSystem
MetersDL/SP, AllMeterReadDL/SP, MeterReadForOccubentAndExitDL/SP, MeterServiceTypeFixedAmountDL/SP.	ElectronicBillSystem

Live feature-procedure parameter counts range from 4 to 36. Previous detailed signatures are correct; key shared contracts are pageName, idaraID, entrydata, hostname, followed by feature parameters. Procedures were not executed to discover dynamic metadata.

Result Sets Inferred from Live Definitions

- MastersDataLoad first returns permissionTypeNameE from ftUserPagePermissions, then the selected DL results.
- MastersCRUD and downstream SP procedures usually return the IsSuccessful/Message contract; branches may return business errors within 50001..50999. This is inferred from live SELECT/THROW statements, not execution.
- ControlPanel: PagesManagment combines program/menu/distributor/permission-type definitions; Permission combines grants and DDL; UsersDL returns users and lookups and was expanded live by four organizational columns documented in 07A.

- **Housing Definitions:** The total feature results after permissions was previously documented as 18 across five pages; BuildingDetailsDL alone returns eight feature sets after permissions, including one not consumed by the Controller.
- **Housing Procedures:** HousingResidentDL and HousingExitDL return only one feature set each; Controller requests for additional results have no live counterpart. Other results are documented in the program document.
- **Waiting/Imports:** 31 feature result sets and 8 permission result sets for pages using the gateway; direct import adds no permission set.
- **IncomeSystem:** DLs return financial/insurance/payment lists and DDL as detailed in the program document. The import path has a DL, SP, and independent import operation.
- **ElectronicBillSystem:** 20 documented result sets, including permission sets for the four pages.
- **Home/Login:** Login, tree, and Home/Charts results are documented from live definitions; no actual account execution occurred.

Tables Read or Modified

Identification relied on sys.sql_expression_dependencies and live module text where the dependency catalog could not resolve dynamic SQL:

- ControlPanel reads/modifies program, menu, distributor, permission-type, grant, user, and password tables, and writes audit/error/notifications according to branch.
- Housing Definitions reads/modifies lookups, BuildingDetails, BuildingRent, and BuildingDetailsMeterServices.
- Housing lifecycle reads/modifies ResidentInfo/Details, BuildingAction, BuildingHandover, ExtendInsurance, RentBills, Bills, and billing periods/types.
- Waiting reads/modifies BuildingAssign and waiting/status/beneficiary/exemption families; upload writes staging/log when run, but it was not run.
- IncomeSystem reads/modifies Deduct/BillDeduct/BuildingPayment/ExtendInsurance families, bills, and audit records.
- ElectronicBillSystem reads/modifies Meter/MeterForBuilding/MeterRead, pricing, periods, and bills.

The catalog recorded 611 dependency rows for Housing objects. This count does not establish the number of unique tables and does not capture all dynamic SQL.

Functions

The direct shared function: dbo.ftUserPagePermissions.

Live Housing functions (33): BuildingNoIsExist, FirstLineInHousingnDashBoard, fnGetResidentID, GeneralNumberIsExist, GetActionDateForBuildingExit, GetBuildingNoByResidentGeneralNo, GetCountAllMeter, GetCountAllResident, GetCountBuildingForActionTypeID, GetCountBuildingForActionTypeWithClassID, GetCountBuildingFromUtilityType, GetCountBuildingWithClass, GetCountBuildingWithType, GetCountForActionTypeByDates, GetCountForHousingWithPrivteConditions, GetCountMeterForUtilityType, GetCountRentType, GetLastActionForBuilding, GetLastActionForResident, GetLastMeterReadFromOldSystem, GetRentForBuilding, GetResdintGeneralNumberByBuildingNo, CalculteElectricityBillsByLastActiveReadPeriodSource, CalculteElectricityBillsByNewReadValue, CalculteElectricityBillsByNewReadValueForInsert, fnBuildingActionChainToRoot, fnBuildingRentForMonth, fnCalcBuildingRentForOneMonth, fnCalcMonthlyBuildingRentByBuildingDetailsID, FNEligibleMeters, GetGeneralListActionForResident, GetListAllMetersReadByPeriodID, and GetListMeterSlidesPrice.

Triggers

Effect Inferred from Live Definition	Status	Table	Trigger
Audits BuildingAction changes; may overlap explicit AuditLog and requires operational testing.	Active	Housing.BuildingAction	Housing.trgBuildingActionAudit
Updates payment-link status according to financial linking/classification.	Active	Housing.BuildingPayment	Housing.TRBuildingPaymentSet LinkStatus

Both triggers exist live and are absent as Trigger definitions from the Snapshot.

Gateway Routing

The live catalog contains 81 pageName values in MastersDataLoad and 65 in MastersCRUD across the entire system. Within the included programs, verified routing includes:

- ControlPanel: PagesManagment, Permission, Users.
- Housing Definitions: BuildingClass, BuildingDetails, BuildingType, buildingUtilityType/BuildingUtilityType, MilitaryLocation.
- Housing Procedures: HousingResident, HousingHandover, HousingExtend, HousingExit.
- Waiting: Residents, WaitingListByResident, WaitingListMoveList, WaitingList, OtherWaitingList, Assign, AssignStatus/ASSIGNSTATUS, RentExemption.
- IncomeSystem: ExtendInsurance, FinancialAuditForExtendAndEvictions, FinancialAuditForUser, ImportExcelForBuildingPayment.
- ElectronicBillSystem: Meters, AllMeterRead, MeterReadForOccubentAndExit, MeterServiceTypeFixedAmount.
- Home: Home in DataLoad. Login calls login procedures directly through ProcedureMapper rather than page routing.

MastersDataLoad also contains routes for excluded programs; they were not added to this documentation. Live MastersCRUD matches ActionType and checks permission in most branches, with previously documented security gaps around trusting client context and the HousingHandover branch.

SQL Agent Jobs

Jobs were not queried. The permitted-source list does not include SQL Agent catalog tables in msdb, and sys.* within DATACORE does not prove scheduling of a Job outside the database. Scheduling of GenerateMonthlyRentBills or backfill/import, and any owner/schedule/step, therefore remain operational verification gaps; Job existence must not be inferred from a procedure name.

Schema Drift: Live Versus Snapshot

Present Live and Absent from Snapshot

- Housing.trgBuildingActionAudit.
- Housing.TRBuildingPaymentSetLinkStatus.

Present in Snapshot and Absent Live

No differently named/missing object exists among the name-matched Housing definitions read or the selected direct dbo list. Excluded-program files are not part of this result.

Verified Functional Definition Differences

The 07A list remains the detailed difference reference: 11 procedures and Housing.VWaitingList differ from the Snapshot, chiefly gateway routing, UsersDL, occupancy/extension/evacuation rules, AssignDL/SP, and BuildingRentForOneMonth. IncomeSystem, ElectronicBillSystem, and Home/Login phases added no new functional drift to the procedures compared.

Drift and Structural Constraints Requiring Attention

- Two live triggers have no project files; deployment from the Snapshot may delete their behavior or fail to recreate it.
- The BuildingPayment -> BuildingPaymentType FK is untrusted.
- Seven Housing tables have no live PK.
- Several columns named FK have no actual constraint; the name must not be treated as referential integrity.
- Live dated/alternate objects exist, such as BuildingRentForOneMonth2026-05-12, VOccupant20260204, and VGetListMetersLinkedWithBuildings20260228; their existence is documented, but active use is unverified.

Diagrams

- Condensed General ERD

- ControlPanel and Permissions ERD
- Housing ERD
- IncomeSystem and ElectronicBillSystem ERD

Each diagram is independent and condensed for A4 printing. A solid line labeled FK means a live constraint; a dashed line or 'Inferred' label means a logical relationship without an FK.

11. Schema Drift

Present Live and Absent from Snapshot

- Housing.trgBuildingActionAudit.
- Housing.TRBuildingPaymentSetLinkStatus.

Present in Snapshot and Absent Live

No differently named/missing object exists among the name-matched Housing definitions read or the selected direct dbo list. Excluded-program files are not part of this result.

Verified Functional Definition Differences

The 07A list remains the detailed difference reference: 11 procedures and Housing.VWaitingList differ from the Snapshot, chiefly gateway routing, UsersDL, occupancy/extension/evacuation rules, AssignDL/SP, and BuildingRentForOneMonth. IncomeSystem, ElectronicBillSystem, and Home/Login phases added no new functional drift to the procedures compared.

Drift and Structural Constraints Requiring Attention

- Two live triggers have no project files; deployment from the Snapshot may delete their behavior or fail to recreate it.
- The BuildingPayment -> BuildingPaymentType FK is untrusted.
- Seven Housing tables have no live PK.
- Several columns named FK have no actual constraint; the name must not be treated as referential integrity.
- Live dated/alternate objects exist, such as BuildingRentForOneMonth2026-05-12, VOccupant20260204, and VGetListMetersLinkedWithBuildings20260228; their existence is documented, but active use is unverified.

Reading rule: This chapter shows historical differences only. In a conflict, the Live description is current behavior; the Snapshot does not prove operational state.

12. Workflows and Diagrams

Source Mermaid diagrams were reviewed and converted into simplified fixed-width print representations suitable for A4. Source files retain full detail, while the figures here focus on core nodes and their flow.

Figure 1: Layers and Project References Diagram

Layers and Project References Diagram

Result		Stage		Path
Program.cs\nComposition Root	←	SmartFoundation.Mvc - Runtime and Presentation	←	Browser / HTTP Client
Middleware + Routing + Session	←	Razor Views	←	MVC Controllers
SmartFoundation.Application - Application Services	←	SmartForm / SmartTableDS / Charts / Print	←	SmartFoundation.UI - Shared Rendering Components
ISmartComponentService\nSmartComponentService	←	SmartFoundation.DataEngine - Data Execution	←	MastersServies / BaseService

Figure 2: HTTP Request Pipeline Diagram

HTTP Request Pipeline Diagram

Result		Stage		Path
2. HSTS	←	1. Response Compression	←	HTTP Request
5. Security Headers + CSP\ninline middleware	←	4. Cookie Policy	←	3. HTTPS Redirection
8. Authentication middleware	←	7. Routing	←	Static File Response
Razor Pages	←	10. Session middleware	←	9. Authorization middleware

Figure 3: Read and Write Flow Diagram

Read and Write Flow Diagram

Result		Stage		Path
MVC Action	←	Browser GET	←	Read Path - Housing-style
ProcedureMapper\nMastersDataLoad:getData	←	MastersServies.GetDataLoadDataSetAsync	←	InitPageContext + Session context
dbo.MastersDataLoad	←	Dapper QueryMultipleAsync	←	SmartRequest\nOperation = sp
SmartResponse.Datasets -> DataSet	←	Result sets	←	Feature DL procedure

Figure 4: ControlPanel Flow Diagrams

ControlPanel Flow Diagrams

Result		Stage		Path
Queries inside the gateway	←	dbo.MastersDataLoad	←	View / SmartRenderer
dbo.MastersCRUD	←	SQL Objects	←	dbo.UsersDL
PermissionType + DistributorPermissionType	←	Program + Menu + Distributor	←	UsersSP / ReSetUserPassword
Navigation menu for all Unified Platform of the Engineering and Works Administration of the Royal Saudi Land Forces programs	←	Users + Details + Password + Distributor	←	Permission assignments

Figure 5: Housing Definitions Flow Diagrams

Housing Definitions Flow Diagrams

Result		Stage		Path
InitPageContext + Session	←	HousingController Action	←	Open a definition page
Feature DL	←	permission result set	←	MastersServices -> dbo.MastersDataLoad
FormConfig + SmartTableDsModel	←	Message + Home/Index	←	dt1..dt8
dbo.MastersCRUD	←	CrudController: pNN -> parameterNN	←	SmartPageViewModel -> SmartRenderer

Figure 6: Partial Housing Definitions ERD

Partial Housing Definitions ERD

Result		Stage		Path
—	←	—	←	19-Housing-Live-ERD.md

Figure 7: Complete Housing Procedures Flow

Complete Housing Procedures Flow

Result		Stage		Path
HousingController Action	←	Thin Razor View / SmartRenderer	←	User opens a Housing Procedure page
ProcedureMapper: MastersDataLoad:getData	←	MastersServices.GetDataLoadDataSetAsync	←	InitPageContext + Session + query filters
HousingResident/Handover/Extend/Exit DL	←	ftUserPagePermissions: result set 0	←	dbo.MastersDataLoad
SmartPageViewModel + forms + table	←	Home/Index + denial message	←	DataSet feature tables

Figure 8: Residence, Handover, Extension, and Evacuation Cycle

Residence, Handover, Extension, and Evacuation Cycle

Result		Stage		Path
—	←	—	←	Diagram documented in the text source

Figure 9: Housing Procedures Impact on Data and Programs

Housing Procedures Impact on Data and Programs

Result		Stage		Path
Bills: exit rent / penalties / final services	←	Bills: rent and services	←	Housing.BuildingAction
—	←	Financial audit, billing, and income	←	VWaitingList / VOccupant / building views

Figure 10: Waiting, Assignment, Transfer, and Exemption Workflow

Waiting, Assignment, Transfer, and Exemption Workflow

Result		Stage		Path
WaitingList\nGeneral Categories	←	WaitingListByResident\nWaiting + Letters	←	Residents\nCreate Beneficiary Profile
WaitingListByResident\nin the New Administration	←	Administration transfer request	←	OtherWaitingList\nLocal Categories
Housing Handover/Resident	←	AssignStatus\nRecord Processing	←	Assign\nAllocation Record
—	←	—	←	Monthly/exit rent calculation

Figure 11: Waiting/Assignment Gateways and Data

Waiting/Assignment Gateways and Data

Result		Stage		Path
dbo.MastersDataLoad	←	HousingController partial Actions	←	SmartRenderer\nFormConfig + SmartTableDsModel
CrudController\np01..p50 -> parameter01..50	←	8 Housing feature DL procedures	←	permission result set\nftUserPagePermissions
BuildingAction ledger	←	8 Housing feature SP procedures	←	dbo.MastersCRUD
—	←	pageName + ActionType\npermission	←	AssignPeriod / AssignNote

Figure 12: UploadExcel and UploadLab Flows

UploadExcel and UploadLab Flows

Result		Stage		Path
UUID file\nwwwroot/uploads/excel	←	extension + MIME + <=10MB	←	POST Upload\nAntiforgery
POST Process\nSelect 3 Columns	←	Session\ncolumns + 200 preview rows	←	ExcelDataReader\nFirst Sheet + Header
Direct SqlCommand\nUploadExcelImportS elected3Cols	←	SHA-256 + TVP	←	Column existence/distinctness/nonemptiness
PDF/XLS/XLSX + <=10MB	←	Gap: TVP schema has five columns\nDataTable has four columns	←	UploadExcel + ImportLog

Figure 13: Live IncomeSystem Paths

Live IncomeSystem Paths

Result		Stage		Path
dbo.MastersCRUD	←	dbo.MastersDataLoad	←	Views / SmartRenderer
FinancialAudit SP procedures	←	FinancialAudit DL procedures	←	dbo.MastersExtraDataLoad
—	←	Routing gap	←	DeductList / BuildingPayment / ImportLog

Figure 14: Financial Data Relationship

Financial Data Relationship

Result		Stage		Path
ElectronicBillSystem: MeterRead -> Bills	←	Financial audits	←	Housing lifecycle: BuildingAction / VWaitingList
—	←	Financial summaries and reports	←	Excel TVP import

Figure 15: Meter-to-Bill Path

Meter-to-Bill Path

Result		Stage		Path
dbo.MastersCRUD	←	dbo.MastersDataLoad	←	Razor Views / SmartRenderer
Meter + MeterType + MeterServiceType	←	Housing Meters/Read SP	←	Housing Meters/Read DL
—	←	IncomeSystem audits / DeductList / BuildingPayment	←	MeterForBuilding -> BuildingDetails

Figure 16: Occupancy and Evacuation Reading Integration

Occupancy and Evacuation Reading Integration

Result		Stage		Path
BuildingAction: occupancy or exit	←	Housing movement / VWaitingList	←	ResidentDetails / VGetFullResidentDetails
Next BuildingAction	←	Remain pending	←	Entry/exit MeterRead
—	←	All building meters read?	←	IncomeSystem financial review

Figure 17: Condensed General ERD for DATACORE

Condensed General ERD for DATACORE

Result		Stage		Path
Housing — income and billing	←	Housing — accommodation and waiting	←	dbo — ControlPanel and security
—	←	—	←	dbo.Idara

Figure 18: Live ControlPanel and Permissions ERD

Live ControlPanel and Permissions ERD

Result		Stage		Path
—	←	—	←	Diagram documented in the text source

Figure 19: Live Housing ERD

Live Housing ERD

Result		Stage		Path
—	←	—	←	Diagram documented in the text source

Figure 20: Live IncomeSystem and ElectronicBillSystem ERD

Live IncomeSystem and ElectronicBillSystem ERD

Result		Stage		Path
—	←	—	←	dbo.Idara

Figure 21: Reports and Shared Integrations Flow

Reports and Shared Integrations Flow

Result		Stage		Path
DATACORE: live DL	←	Included program page	←	User authenticated through Session
QuestPDF: PDF inline	←	SmartTableDS: Screen	←	DataSet / DataTable
Notifications API	←	POST /exports/pdf/table	←	Client Excel / Print
wwwroot/uploads	←	Excel/PDF uploads	←	DATACORE notifications

Figure 22: Operations, Deployment, Backup, and Troubleshooting

Operations, Deployment, Backup, and Troubleshooting

Result		Stage		Path
Versioned Release Artifact	←	Restore / Build / Test / Publish	←	Source + approved configuration names
SmartFoundation.Mvc	←	Dedicated App Pool / No Managed Code	←	IIS + .NET 8 Hosting Bundle
Full + Differential + Transaction Log backups	←	IIS / ANCM / ILogger / SQL ErrorLog	←	Uploads + fonts + static assets
Artifact rollback copy	←	File backup by retention policy	←	Restore Drill\nTarget RPO 15m / RTO 1h

Figure 23: Authentication and Session Diagram

Authentication and Session Diagram

Result		Stage		Path
Login form + antiforgery	←	GET /Login/Index\nClear Session + no-cache	←	Browser
Users active + validity dates	←	auth:sessions	←	POST /Login/CheckLogin
Failure message -> Login	←	active UsersDetails + Idara/Department	←	UsersPassword\n32-byte salt + SHA2256
/Home/Index	←	Session server-side\nidentity + organization + presentation	←	ClaimsIdentity + Cookie Authentication ticket

Figure 24: Authorization and Permissions Diagram

Authorization and Permissions Diagram

Result		Stage		Path
GET page	←	Session usersID	←	Authentication Cookie + NameIdentifier Claim
Direct Permission	←	MastersDataLoad\nentrydata + pageName	←	InitPageContext if invoked by the Action
Distributor grants	←	DSD and structure	←	UserDistributor -> Distributor -> Role
Controller flags	←	result set 0\npermissionTypeNameE	←	Public grant

13. Reports and Integrations

Version 1.0.0 Operational Update

- The reference environment used SQL Server 2019 and Windows Integrated Security. Developer Edition is not a production requirement; production depends on a supported, appropriately licensed Edition.
- The current model uses a Single Application Server with local In-Memory Session. Scaling requires reevaluating distributed session, Data Protection keys, load balancing, health checks, and centralized monitoring.
- CSP is implemented and active, and current compatibility settings are an architectural decision to be reevaluated when Alpine.js, JavaScript, or frontend libraries change.
- Target operational-continuity objectives: RPO = 15 minutes and RTO = 1 hour. Achieving them depends on production policy, Transaction Log Backup schedule, storage, monitoring, and Restore Drill.
- Full Backup, Differential Backup, Transaction Log Backup, and Restore Drill are parts of the operational model. FULL Recovery Model describes the reference environment and does not guarantee production configuration.
- Application Logging must exclude passwords, Session Data, and unnecessary sensitive information; protect logs; and apply rotation, retention, time synchronization, and restricted access. SIEM/Centralized Logging is a hosting-infrastructure responsibility unless otherwise adopted.
- The following IIS considerations are hosting and post-deployment acceptance recommendations, not evidence that they are implemented on a particular production server.

Executive Summary

No independent reporting platform or SSRS is verified in scope. Actual reports are built inside Actions from the DataTable returned by the read gateway, then converted to PDF by DataTableReportBuilder and QuestPdfReportRenderer. A general endpoint, POST /exports/pdf/table, also receives displayed browser columns and rows and uses QuestPdfExportService. On-screen printing relies on SmartTableDS, SmartPrint, and iframe/window.print().

The only verified actual external integration is SQL Server. No active evidence was found for Email, SMS, LDAP/Active Directory, Cloud File Storage, a second external database, or a third-party service.

The application is a .NET 8 Web application whose entry point is SmartFoundation.Mvc/Program.cs. Publish files are Folder profiles only; there are no IIS, pipeline, or container files proving production shape. The following therefore clearly distinguishes verified configuration from requirements the operations team must adopt.

Reporting, Printing, and Export Architecture

Shared Paths

Control and Authorization	Output	Data Source	Input	Path
Print-button visibility is usually controlled by page permission; the URL branch itself adds no independent authorization check in most Actions.	Inline PDF through QuestPDF	The same DataSet/DataTable returned by MastersDataLoad	Query filters and Session context	Domain Action with pdf=1 or pdf=2
[ApiController] with no local Session/permission check or established size limit; returns ex.Message on error.	PDF attachment	Rows submitted by the client; no new SQL query	Title, paper/orientation, headers, columns, rows, filename	POST /exports/pdf/table
ShowExportCsv/Excel/Pdf and ShowPrint control visibility; they are not a substitute for server-side authorization.	Screen, client export, print	On-screen table model	Current rows/columns	SmartTableDS
Only an A4 Portrait template is established; a generic iframe is used in SmartRenderer.	A4 HTML and window.print()	Server-side ViewModel	SmartPrintConfig and Docs/Blocks	SmartPrint

Control and Authorization	Output	Data Source	Input	Path
Test endpoints without local authorization; they must be disabled or restricted in production.	PDF inline	Test data created in code	No business input	/reports/test, /reports/dynamic

QuestPDF.Settings.License = Community and the wwwroot/fonts/Tajawal-Regular.ttf font are registered at startup. A missing font prevents application startup because opening is not wrapped in error handling.

Included-Program Report Inventory

Presentation Channels	Established Authorization/Availability	Live Data Source	Parameters and Filters	Report or Output	Program/Page
Screen and PDF; CSV/Excel are disabled in the displayed model.	ShowExportPdf=true; there is no independent gate in the endpoint.	Page data through the previously documented ControlPanel gateways; the endpoint does not re-query.	Visible rows and columns	Generic table PDF export	ControlPanel / Users
Screen and PDF.	The print button is currently tied to INSERTBUILDINGDETAILS, a coupling that requires review.	MastersDataLoad -> Housing.BuildingDetailsDL; live parameter @buildingUtilityType DFK; depends on BuildingDetails, types, rent, meters, and VLastActionForBuilding.	pdf=1, query U -> utility type	Building register by utility type	Housing / BuildingDetails
Screen and PDF.	Standard print buttons are disabled, but additional Actions invoke the branches.	Housing.BuildingTypeDL; live dependencies are Housing.BuildingType and dbo.City.	pdf=1 report, pdf=2 letter	Types report and test letter	Housing / BuildingType
Screen and PDF; generic export is disabled.	The report button uses INSERTRESIDENTS instead of an independent Print permission.	Housing.ResidentsDL; live dependencies are VGetFullResidentDetails and Gender/MaritalStatus/Nationality/Rank/militaryUnit.	pdf=1/2; no additional business filter.	Beneficiary list and letter	Housing / Residents
Screen and PDF.	MOVETOASSIGNLIST when data exists.	Housing.WaitingListDL(@WaitingClassIDnvar); depends on VWaitingList, resident, class, and action type.	pdf=1, U=WaitingClassID	Waiting-category records	Housing / WaitingList
Screen and PDF.	MOVETOOCUPENT PROCEDURES when data exists.	Housing.OtherWaitingListDL(@WaitingClassIDnvar); resident/building/waiting dependencies are live.	pdf=1, U=WaitingClassID	Other waiting-category records	Housing / OtherWaitingList
Screen and PDF Letter.	Generic table buttons are disabled; a custom letter Action exists.	Housing.AssignDL(@WaitingClassID,@AssignPeriodID); depends on AssignPeriod, VWaitingList, and resident/building.	The PDF branch uses the period/record context as constructed by the Action.	Assignment letter/output	Housing / Assign

Presentation Channels	Established Authorization/Availability	Live Data Source	Parameters and Filters	Report or Output	Program/Page
Screen and PDF Letter.	ShowPrint1=true with no visible direct gate; the state operations themselves are permission-gated.	Housing.AssignStatusDL(@AssignPeriodID); depends on period, waiting list, beneficiary, and building.	pdf=2, rowId, U=AssignPeriodID	Review notice	Housing / AssignStatus
Screen, client-side Excel, and PDF.	The Print1 button is not shown, but generic Excel and PDF are enabled; operation permissions do not establish an independent export permission.	Housing.HousingResidentDL; depends on resident/waiting/action/meter.	pdf=1	Beneficiary list	Housing / HousingResident
Screen and PDF.	The list is tied to HOUSINGEXTEND; the letter is invoked from a selected row.	Housing.HousingExtendDL; resident/waiting/rent/bills/insurance/reasons	pdf=1 list, pdf=2 with rowId/NID.	List and letter	Housing / HousingExtend
Screen and PDF.	ShowPrint1=true without a visible independent print-permission name.	Housing.HousingExitDL(@NationalID); depends on resident/waiting/bills/action/meters.	pdf=1 list, pdf=2 with rowId/NID.	List and deduction-suspension letter	Housing / HousingExit
Screen/PDF are designed, but live data is not connected.	PRINTDEDUCTLISTREPORT	Live gap: DATACORE gateways have no branch for DeductListReport, so no report data source is actually connected.	Service type, then period; pdf.	Deduction schedule by service and period	IncomeSystem / DeductListReport
Screen and PDF.	Audit/review/payment/settlement permissions; no visible independent Print permission.	Housing.FinancialAuditForExtendAndEvictionsDL(@residentInfoID); resident, bill, building, and waiting-list summaries.	pdf=1; resident/action context	Financial audit list	IncomeSystem / FinancialAuditForExtendAndEvictions
Screen; CSV/Excel settings are disabled, and no PDF branch is established in the Action.	Page and operation permissions documented in the IncomeSystem document.	Housing.FinancialAuditForUserDL; resident, bills, payments, rent, waiting	NationalID -> @NationalID	Beneficiary audit screens	IncomeSystem / FinancialAuditForUser
Screen and PDF, but the title and filename are copied from BuildingType.	Standard Print buttons are disabled; additional Actions exist.	Housing.ExtendInsuranceDL; insurance/resident/waiting/users	pdf=1/2	Two designed reports	IncomeSystem / ExtendInsurance
Screen and PDF; table export buttons are disabled.	Open/close/read/update/delete permissions; no independent Print permission.	Housing.AllMeterReadDL; periods, reads, bills, service types, meter views/functions	pdf=1, filter U=MeterServiceTypeID -> @meterServiceTypeIDFK	Readings list	ElectronicBillSystem / AllMeterRead
Screen and PDF accessible through the URL only.	Reading-entry/approval permissions; print/export buttons are currently disabled.	Housing.MeterReadForOccubentAndExitDL; resident/waiting/building/meter/read/bill	pdf=1, U=residentInfoID -> @residentInfoID	Movement/readings report	ElectronicBillSystem / MeterReadForOccubentAndExit
Screen; printing/export are disabled in the current models.	Operations are permission-gated	Housing.MetersDL and MeterServiceTypeFixedAmountDL are documented in Phase 9.	Page filters	Operational tables	ElectronicBillSystem / Meters and FixedAmount

Report Quality and Gaps

- Many reports use reportId: "BuildingType", filename BuildingType.pdf, and titles such as 'Beneficiary List' even for meters, financial auditing, and insurance; this is a verified naming/content issue.
- There is no visual test for current reports, nor tests for long Arabic columns, large volume, or null data.
- The general export trusts client-submitted rows/columns; it must not be treated as source or permission revalidation.
- Print permissions are inconsistent: some reuse Insert/Move permission, some appear without an independent gate, and some PDF branches are query-accessible despite a hidden button.
- No request-size constraint, row limit, or rate limit is verified for /exports/pdf/table.
- CSV/Excel in SmartTableDS is client behavior; no server-side .xlsx generation library is verified. ExcelDataReader 3.8.0 is for reading/import only.

Related Libraries

Operational Notes	Usage	Installed Version	Library
Community license at startup; requires the Tajawal font, permission to read assets, and sufficient memory for report size.	Generic PDF, DataTable reports, and letters	2025.12.1	QuestPDF
First Sheet with header; this is not an export library. Large/encrypted/corrupted files must be tested.	Reading .xls/.xlsx in UploadExcel and Income import	3.8.0	ExcelDataReader + DataSet
The connection string is secret, and timeouts are not standardized.	Executing SQL gateways and direct import paths	Through Application/DataEngine	Dapper / SqlConnection

Integrations

Internal APIs

Timeout/Error handling	Authentication/Authorization	Function	API
The service layer logs errors; get-latest returns a generic message. The client interface aborts after 10 seconds.	Rejects missing userID	Latest notifications and read/clicked marking	/api/Notifications/
try/catch, but it returns ex.Message; no timeout/rate limit.	No established local Session/permission check	PDF from client data	POST /exports/pdf/table
Name whitelist inside DataEngine and error logging; a powerful surface that requires authorization and abuse testing.	No local Session guard in the Controller	Execute SmartRequest through DataEngine	POST /smart/execute
Internal data; it must not be public in production.	No established gate	Test PDF	/reports/test, /reports/dynamic

There is no verified OpenAPI/Swagger documentation, versioning, custom CORS policy, or API keys.

Email, SMS, LDAP, and External Services

- Email/SMTP: No active use of SmtpClient, MailKit, or an email provider in the included projects.
- SMS: No verified Twilio, SMS gateway, or custom HTTP client.
- LDAP/Active Directory: No verified DirectoryServices, LdapConnection, or Windows Authentication; login uses database/Session.

- External databases: One logical connection named ConnectionStrings:Default to DATACORE. No second database connection was verified.
- Third-party/cloud: No verified S3/Azure Blob/SharePoint or third-party API integration in scope.
- Notification names in SQL are not evidence of Email/SMS; only in-application notifications are verified.

File Storage and Uploads

Lifecycle	Storage	Types and Limit	Path
Direct import into DATACORE upon approval; a documented TVP gap exists, and an optional deletion comment does not establish general cleanup.	wwwroot/uploads/excel using a GUID name; metadata in Session	.xls/.xlsx, MIME allow-list, 10 MB, one file, signature validation	Housing UploadExcel
Deletes the file after successful processing/read failure; a cleanup function for files older than 7 days exists, but its invocation and scheduling must be established.	The same uploads/excel folder and Session	.xls/.xlsx, MIME/signature, 10 MB	Income ImportExcelForBuildingPayment
No SQL; no page-level deletion or established retention. The folder is excluded from Content in the csproj and may not be created during publish.	wwwroot/uploads/lab and Session only	PDF/XLS/XLSX, 10 MB, one file	UploadLab

Both directories are under the web root, so files may become directly accessible if the URL is known. No malware scan, at-rest encryption, static-file authorization, or storage quota is verified. Sensitive attachments should be moved outside the web root and served through an authorized Action, but this is a recommendation rather than a description of current state.

Development and Runtime Environment

Verified Requirements

- .NET 8 SDK supports net8.0 and C# 12. No global.json pins an SDK patch; the team must adopt a consistent compatible SDK version.
- Visual Studio 2022 in a version supporting .NET 8 and ASP.NET and web development. The SQL project in the solution may require SQL Server Data Tools; it is unnecessary for building the MVC project alone.
- SQL Server can host DATACORE and its current definitions; server version/edition/compatibility was not queried in this phase, so no specific number is assumed.
- Node/npm are required only to build Tailwind (tw:watch, tw:build) when assets change.
- Correct Startup Project: SmartFoundation.Mvc/SmartFoundation.Mvc.csproj. The repository-root Program.cs is not an entry point.

Documented Commands for Use, Not Executed in This Phase

```
dotnet restore SmartFoundation.sln
dotnet build SmartFoundation.Mvc/SmartFoundation.Mvc.csproj
dotnet run --project SmartFoundation.Mvc/SmartFoundation.Mvc.csproj
dotnet test SmartFoundation.Application.Tests/SmartFoundation.Application.Tests.csproj
npm install --prefix SmartFoundation.Mvc
npm --prefix SmartFoundation.Mvc run tw:build
```

A full solution build may require tooling for SmartFoundation.Database.sqlproj. The SQL database in the repository is a reference Snapshot, not a deployment source or live truth.

Development, Production, and Configuration

- ASP.NET Core merges appsettings.json, appsettings.{Environment}.json, environment variables, and standard sources.
- Important keys: ConnectionStrings:Default, SmartData:Whitelist, SmartData:MaxPageSize, Logging:LogLevel, and AllowedHosts.

- Do not store secrets in Documentation or source control. Use environment variables or an approved secret store; name-only example: `ConnectionStrings__Default`.
- Development launch profiles exist for Project/IIS Express, but do not prove production-server configuration.
- Production config contains connection and AI keys in the file; Git history must be audited and any actual previously published value rotated, without copying it into documentation or logs.
- `UseHsts` and `UseHttpsRedirection` run without an environment condition in code. Secure cookies require HTTPS even in Development; Session may fail on local HTTP.

IIS Deployment: Documentation-Only Runbook

No deployment was performed. `FolderProfile.pubxml` files publish via `FileSystem` to two different local paths with `DeleteExistingFiles=false`; they do not prove an IIS site, App Pool, or promotion method.

Requirements

157. A supported Windows Server, IIS with ASP.NET Core Module, and a .NET 8 Hosting Bundle matching the architecture.
158. A dedicated Application Pool, No Managed Code, Integrated pipeline, and a least-privileged service identity. AnyCPU is verified; select x64 according to server architecture and current runtime libraries.
159. An IIS Site/Binding with approved name and hostname, and a valid TLS certificate with a trusted chain and monitored renewal.
160. A release directory that is not writable by the App Pool identity, except explicitly required runtime directories: uploads, logs if a file provider is used, and Data Protection key storage if adopted.
161. Connection settings are injected from a secure environment. Prevent serving appsettings, logs, and backup files through static hosting.
162. Copy required runtime fonts and assets and verify read permissions before cutover.

Proposed Publish/Deploy Steps

163. Approve the commit/tag and change configuration to Release in a trusted pipeline.
164. Run `restore/build/test/publish` in CI or on a build station; do not build on the IIS server.
165. Scan the artifact for secrets and verify font/model/docs/static assets and generated `web.config`.
166. Take an approved backup before the change according to organizational policy, then stop receiving requests using `drain/app_offline`.
167. Deploy to a new release directory, set ACLs, and switch binding/path or symlink/junction in a rollback-capable manner.
168. Run smoke tests: startup, login test account, Session, a read-only page, a small PDF, a nonsensitive upload in a test environment, AI if enabled, and DATACORE connectivity.
169. Monitor Event Viewer/IIS/ASP.NET/SQL logs, then close the change window or roll back to the previous artifact on failure.

These are documentation steps, not evidence that they were applied.

Reverse Proxy, TLS, and Forwarded Headers

- HSTS/HTTPS redirection/security headers are verified in the application.
- No `UseForwardedHeaders` or `ForwardedHeadersOptions` are verified. Behind out-of-process IIS, ASP.NET Core Module may handle some information, but any additional proxy or load balancer requires Trusted Proxies/Networks and middleware ordering before `redirect/session/logging`.
- Code reads `X-Forwarded-For` in previous contexts without a unified trust policy; do not rely on it for auditing before configuring forwarded headers.
- TLS termination, cipher policy, production certificate, and HTTP-to-HTTPS binding are infrastructure gaps absent from the repository.

Backup and Recovery

No operation was executed. A connected plan and periodic restore testing are required, not merely separate file copies.

Restore and Verification Points	What Must Be Backed Up	Asset
RESTORE VERIFYONLY alone is insufficient; perform a test restore with DBCC and access controls, followed by an application compatibility smoke test. Version/Recovery Model/SQL Agent jobs are current gaps.	Full backups according to RPO, Differential/Log backups when permitted by the Recovery Model and policy, and SQL encryption keys/certificates if present.	DATA CORE
A DB-consistent backup when references exist, clear retention, malware scanning, encryption, and a file-recovery test. Some Excel files are temporary and must not automatically become permanent backups.	wwwroot/uploads/excel and wwwroot/uploads/lab if considered required records	Uploads/attachments
Do not restore old secrets without rotation; verify connection, AllowedHosts, and whitelist after restoration.	Configuration names and secure copies of production values in a secret manager/IIS configuration	Configuration
Retain checksums and the previous release package; source alone is insufficient for rapid recovery.	Release package, web.config, static assets, and the Tajawal font	Application artifact
No PersistKeysToFileSystem/Redis/DB is currently established. Default keys may change across instances/redeployments, affecting cookies/antiforgery. Adopt a protected shared store before scale-out.	key ring if customized later	Data Protection keys
Export configuration securely and do not place private keys in the repo. Test rebuilding a new server.	site bindings, App Pool identity/settings, ACLs, certificate references, and environment variables	IIS/OS
Maintain restricted access, integrity, and size controls; no established retention policy exists.	Operational logs according to retention, and DATA CORE AuditLog/ErrorLog within the database backup	Logs/Audit

RPO/RTO, backup owner, schedule, encryption, off-site/immutable copy, alerting, and restore-drill date must be defined. All are currently organizational gaps.

Troubleshooting

Where to Start

170. Record issue time, URL, environment, user/Idara without password or token, correlation if present, and requested operation.
171. Review IIS access logs and ASP.NET stdout/Event Viewer for startup failure, then application ILogger and SQL ErrorLog under approved read permission.
172. Distinguish a 50001..50999 business error and generic execution message from a connection/initialization exception.
173. Do not automatically rerun a write/import before knowing whether the transaction committed; the system has no general idempotency.

Symptom Matrix

Likely Causes/Action	Safe Checks	Presentation
Missing Hosting Bundle, missing Tajawal-Regular.ttf, invalid config, or insufficient permissions. Disable stdout after diagnosis and do not leave secrets in logs.	Event Viewer/ANCM stdout temporarily, runtime and font presence, ACLs, environment	Application does not start
Runtime mismatch or startup exception. Do not grant Full Control to the entire folder as a quick fix.	App Pool state, operational dotnet --info, web.config, process identity	500/502.5 on IIS

Likely Causes/Action	Safe Checks	Presentation
Cookie Secure=Always; Memory cache is not shared. Scale-out requires distributed cache and shared Data Protection.	HTTPS, Secure cookie, SameSite/domain, Session ordering, multiple instances	Redirect loop/missing Session
Do not log password or connection details. Do not test with a production account without approval.	General log, DATACORE connectivity, presence of the login procedure in whitelist/mapper, server time	Login fails
Separate seed/gateway data from the UI. Hiding the button does not establish endpoint denial.	First DataTable result, permissionTypeNameE, pageName/action casing, Session Idara	Permission button is missing
Home direct command=60s and Income import=120s; other paths may rely on defaults. Do not add retry to a write operation before idempotency.	Network/DNS/TLS, pool, command timeout, blocking through the DBA	SQL timeout/connection
Business validation; correct the request and do not treat it as an infrastructure failure.	Business message, inputs, and current state	Error 50001
MastersCRUD logs errors outside 50001..50999; some Controllers return ex.Message, which is an information-disclosure gap.	Application log + dbo.ErrorLog with the DBA, procedure/page/action	Unexpected SQL error
Some reports print dt1 only, and some use copied titles. DeductListReport is not connected live.	Filter/query, selected dt, permission, logo/font, route	Blank PDF
No general limits; reduce the dataset and add limits and load tests before production.	Row/column count and request size, CPU/memory	Slow PDF/high memory usage
ExcelDataReader; use Save As without encryption in a secure environment. Do not send ex.Message to the user.	Extension/MIME/signature, file protection, first sheet/header, 10 MB	Excel cannot be read
Grant Modify to the specific folder only. uploads/lab may not be copied because of the csproj.	Folder ACL, folder presence after publish, disk quota, antiforgery/Session	Upload 403/500
Files are under the web root. Move them outside the web root or protect delivery through an endpoint.	Test the direct URL under an approved policy	Uploaded file is exposed
The model loads at startup; its absence is fatal. The Provider option does not automatically change the current registration.	Model path, RAM/CPU, file ACL, options	AI does not start
The active timeout is 8s with no retry. Monitor saturation and reduce tokens/context or scale resources after measurement.	AITIMEOUT, queue/pool, max parallel, prompt size	AI timeout
APIs reject a missing Session; no established automatic retry exists.	Session, browser fetch timeout of 10s, SQL procedures/logs	Notifications fail
No established trust policy. Configure Forwarded Headers before relying on them for audit purposes.	Proxy chain and forwarded headers	Incorrect address/host/IP

Logging and Data Risks

- Only default ASP.NET Core providers are verified from code; no Serilog/NLog/file sink or central aggregation is verified.
- No global exception handler is verified in Program.cs; error handling is distributed, and some endpoints return ex.Message.

- MastersServices prints raw menu JSON and the first row to console and logs full parameters in one location; PII/secrets must be reviewed before centralized collection.
- AuditLog and ErrorLog exist live within DATACORE, but their rows were not read and retention/alerting/immunity was not verified.
- Adopt structured logging with correlation ID and redaction, define Production levels, and collect IIS/ANCM/app/SQL in a monitoring platform with restricted permissions.

14. Development and Deployment Environment

Verified Requirements

- .NET 8 SDK supports net8.0 and C# 12. No global.json pins an SDK patch; the team must adopt a consistent compatible SDK version.
- Visual Studio 2022 in a version supporting .NET 8 and ASP.NET and web development. The SQL project in the solution may require SQL Server Data Tools; it is unnecessary for building the MVC project alone.
- SQL Server can host DATACORE and its current definitions; server version/edition/compatibility was not queried in this phase, so no specific number is assumed.
- Node/npm are required only to build Tailwind (tw:watch, tw:build) when assets change.
- Correct Startup Project: SmartFoundation.Mvc/SmartFoundation.Mvc.csproj. The repository-root Program.cs is not an entry point.

Documented Commands for Use, Not Executed in This Phase

```
dotnet restore SmartFoundation.sln
dotnet build SmartFoundation.Mvc/SmartFoundation.Mvc.csproj
dotnet run --project SmartFoundation.Mvc/SmartFoundation.Mvc.csproj
dotnet test SmartFoundation.Application.Tests/SmartFoundation.Application.Tests.csproj
npm install --prefix SmartFoundation.Mvc
npm --prefix SmartFoundation.Mvc run tw:build
```

A full solution build may require tooling for SmartFoundation.Database.sqlproj. The SQL database in the repository is a reference Snapshot, not a deployment source or live truth.

Development, Production, and Configuration

- ASP.NET Core merges appsettings.json, appsettings.{Environment}.json, environment variables, and standard sources.
- Important keys: ConnectionStrings:Default, SmartData:Whitelist, SmartData:MaxPageSize, Logging:LogLevel, and AllowedHosts.
- Do not store secrets in Documentation or source control. Use environment variables or an approved secret store; name-only example: ConnectionStrings__Default.
- Development launch profiles exist for Project/IIS Express, but do not prove production-server configuration.
- Production config contains connection and AI keys in the file; Git history must be audited and any actual previously published value rotated, without copying it into documentation or logs.
- UseHsts and UseHttpsRedirection run without an environment condition in code. Secure cookies require HTTPS even in Development; Session may fail on local HTTP.

15. Backup and Recovery

No operation was executed. A connected plan and periodic restore testing are required, not merely separate file copies.

Restore and Verification Points	What Must Be Backed Up	Asset
RESTORE VERIFYONLY alone is insufficient; perform a test restore with DBCC and access controls, followed by an application compatibility smoke test. Version/Recovery Model/SQL Agent jobs are current gaps.	Full backups according to RPO, Differential/Log backups when permitted by the Recovery Model and policy, and SQL encryption keys/certificates if present.	DATA CORE
A DB-consistent backup when references exist, clear retention, malware scanning, encryption, and a file-recovery test. Some Excel files are temporary and must not automatically become permanent backups.	wwwroot/uploads/excel and wwwroot/uploads/lab if considered required records	Uploads/attachments
Do not restore old secrets without rotation; verify connection, AllowedHosts, and whitelist after restoration.	Configuration names and secure copies of production values in a secret manager/IIS configuration	Configuration
Retain checksums and the previous release package; source alone is insufficient for rapid recovery.	Release package, web.config, static assets, and the Tajawal font	Application artifact
No PersistKeysToFileSystem/Redis/DB is currently established. Default keys may change across instances/redeployments, affecting cookies/antiforgery. Adopt a protected shared store before scale-out.	key ring if customized later	Data Protection keys
Export configuration securely and do not place private keys in the repo. Test rebuilding a new server.	site bindings, App Pool identity/settings, ACLs, certificate references, and environment variables	IIS/OS
Maintain restricted access, integrity, and size controls; no established retention policy exists.	Operational logs according to retention, and DATA CORE AuditLog/ErrorLog within the database backup	Logs/Audit

RPO/RTO, backup owner, schedule, encryption, off-site/immutable copy, alerting, and restore-drill date must be defined. All are currently organizational gaps.

16. Troubleshooting

Where to Start

174. Record issue time, URL, environment, user/Idara without password or token, correlation if present, and requested operation.
175. Review IIS access logs and ASP.NET stdout/Event Viewer for startup failure, then application ILogger and SQL ErrorLog under approved read permission.
176. Distinguish a 50001..50999 business error and generic execution message from a connection/initialization exception.
177. Do not automatically rerun a write/import before knowing whether the transaction committed; the system has no general idempotency.

Symptom Matrix

Likely Causes/Action	Safe Checks	Presentation
Missing Hosting Bundle, missing Tajawal-Regular.ttf, invalid config, or insufficient permissions. Disable stdout after diagnosis and do not leave secrets in logs.	Event Viewer/ANCM stdout temporarily, runtime and font presence, ACLs, environment	Application does not start
Runtime mismatch or startup exception. Do not grant Full Control to the entire folder as a quick fix.	App Pool state, operational dotnet --info, web.config, process identity	500/502.5 on IIS
Cookie Secure=Always; Memory cache is not shared. Scale-out requires distributed cache and shared Data Protection.	HTTPS, Secure cookie, SameSite/domain, Session ordering, multiple instances	Redirect loop/missing Session
Do not log password or connection details. Do not test with a production account without approval.	General log, DATACORE connectivity, presence of the login procedure in whitelist/mapper, server time	Login fails
Separate seed/gateway data from the UI. Hiding the button does not establish endpoint denial.	First DataTable result, permissionTypeNameE, pageName/action casing, Session Idara	Permission button is missing
Home direct command=60s and Income import=120s; other paths may rely on defaults. Do not add retry to a write operation before idempotency.	Network/DNS/TLS, pool, command timeout, blocking through the DBA	SQL timeout/connection
Business validation; correct the request and do not treat it as an infrastructure failure.	Business message, inputs, and current state	Error 50001
MastersCRUD logs errors outside 50001..50999; some Controllers return ex.Message, which is an information-disclosure gap.	Application log + dbo.ErrorLog with the DBA, procedure/page/action	Unexpected SQL error
Some reports print dt1 only, and some use copied titles. DeductListReport is not connected live.	Filter/query, selected dt, permission, logo/font, route	Blank PDF
No general limits; reduce the dataset and add limits and load tests before production.	Row/column count and request size, CPU/memory	Slow PDF/high memory usage
ExcelDataReader; use Save As without encryption in a secure environment. Do not send ex.Message to the user.	Extension/MIME/signature, file protection, first sheet/header, 10 MB	Excel cannot be read
Grant Modify to the specific folder only. uploads/lab may not be copied because of the csproj.	Folder ACL, folder presence after publish, disk quota, antiforgery/Session	Upload 403/500

Likely Causes/Action	Safe Checks	Presentation
Files are under the web root. Move them outside the web root or protect delivery through an endpoint.	Test the direct URL under an approved policy	Uploaded file is exposed
The model loads at startup; its absence is fatal. The Provider option does not automatically change the current registration.	Model path, RAM/CPU, file ACL, options	AI does not start
The active timeout is 8s with no retry. Monitor saturation and reduce tokens/context or scale resources after measurement.	AI_TIMEOUT, queue/pool, max parallel, prompt size	AI timeout
APIs reject a missing Session; no established automatic retry exists.	Session, browser fetch timeout of 10s, SQL procedures/logs	Notifications fail
No established trust policy. Configure Forwarded Headers before relying on them for audit purposes.	Proxy chain and forwarded headers	Incorrect address/host/IP

Logging and Data Risks

- Only default ASP.NET Core providers are verified from code; no Serilog/NLog/file sink or central aggregation is verified.
- No global exception handler is verified in Program.cs; error handling is distributed, and some endpoints return ex.Message.
- MastersServices prints raw menu JSON and the first row to console and logs full parameters in one location; PII/secrets must be reviewed before centralized collection.
- AuditLog and ErrorLog exist live within DATACORE, but their rows were not read and retention/alerting/immunity was not verified.
- Adopt structured logging with correlation ID and redaction, define Production levels, and collect IIS/ANCM/app/SQL in a monitoring platform with restricted permissions.

17. Developer Guide

1. Purpose and Sources of Truth

This guide documents the current application style, especially the current Housing pattern. Trust order is: active code, then live DATACORE definitions and Metadata read through SELECT-only queries, then SmartFoundation.Database files as a reference Snapshot whose currency is not guaranteed. When the Snapshot differs from the live database, the live definition is adopted and the gap documented.

The entry point is SmartFoundation.Mvc/Program.cs. Do not use the root Program.cs or root Views/ and wwwroot/ as application references.

2. Solution Map and Execution Path

- SmartFoundation.Mvc: Composition root, Controllers, Views, Session, and HTTP.
- SmartFoundation.Application: Services and entry-procedure name mapping.
- SmartFoundation.DataEngine: Dapper execution and returning SmartResponse and DataSet.
- SmartFoundation.UI: Page models and shared rendering components.
- SmartFoundation.Application.Tests: Current automated test project.
- SmartFoundation.Database: Snapshot for understanding and comparison only.

Standard read path:

Controller -> MastersServices -> MastersDataLoad -> feature DL -> DataSet -> SmartPageViewModel -> SmartRenderer

Standard write path:

Form/SmartTable -> CrudController -> MastersServices -> MastersCRUD -> feature SP -> IsSuccessful/Message

3. Adding a Module or Page

178. Inspect a similar Housing page, starting with WaitingListByResident when no clearer local pattern exists.
179. Define the program, menu, page, and operations through page management; preserve exact page-name and ActionType matching across UI and live gateway.
180. Add a Controller or partial Controller, View, and required UI models.
181. Add the read branch to MastersDataLoad and write branch to MastersCRUD, then downstream DL/SP as required.
182. Make the first read result set permissions, followed by feature result sets in a documented fixed order.
183. Test loading, every write operation, rejection states, and button hiding, then reconcile the path with the live database definition.

Do not add a downstream feature procedure to ProcedureMapper when an existing gateway performs routing.

4. Controller and Partial Controller

In large modules, use a partial class with one Controller name and one file per page or related group. Preserve the namespace and constructor from the base file.

In a Housing-style Action:

184. Call InitPageContext(out redirectResult) early and return redirect on failure.
185. Read userId, Idarald, and HostName from Session through the shared method.
186. Set ControllerName and PageName to values matching the page and gateway path.
187. Read only documented query parameters and build the parameter array in the order expected by MastersServices.
188. Call GetDataLoadDataSetAsync, then SplitDataSet.
189. Treat Table 0 as the permissions table; do not change subsequent-table ordering without changing all consumers.
190. Build columns, rows, FormConfig, and SmartTableDsModel in the Controller, then pass SmartPageViewModel to a thin View.

Do not trust client-provided identity and scope fields as substitutes for Session. The current contract posts some of them; this is a documented risk that must not be expanded.

5. View and SmartRenderer

Keep the View thin: set the title when needed and invoke SmartRenderer with the composite model. Do not move business logic, DataSet interpretation, or permission checks into Razor.

SmartRendererViewComponent dispatches SmartPageViewModel to subcomponents. Page configuration must therefore be complete and consistent in the Controller: tables, forms, toolbar actions, filters, messages, and DDL data.

6. FormConfig and SmartTableDsModel

Use FormConfig and FieldConfig for every insert/update/cancel operation and specify:

- The shared endpoint (/crud/insert, /crud/update, or /crud/delete).
- Required hidden context fields.
- The correct ActionType.
- Positional field names p01..p50.
- Required, readonly, input type, and DDL source.

Use SmartTableDsModel for table data, columns, rows, key, actions, and toolbar. With multiple result sets, create independent groups such as dynamicColumns and rowsList. Add positional aliases to rows when dialogs must populate p01....

Do not assume a DDL table index is a stable name; it is a positional contract that must be documented and tested.

7. Models and ViewModels

- Use a typed model when the contract is fixed and Dictionary<string, object?> for dynamic rows.
- Preserve nullable semantics and do not convert DBNull into a misleading value.
- Keep ViewModels specific to rendering and configuration, not enforcement of business rules.
- Use PascalCase for types and public properties, and camelCase for variables, parameters, and private fields.
- Add XML documentation for new or substantially changed public members.

8. MastersServices and DataSet

MastersServices is an active gateway, not legacy to be removed. Preserve it on existing pages that rely on multiple result sets.

- Reads use GetDataLoadDataSetAsync to MastersDataLoad.
- General writes use GetCrudDataSetAsync to MastersCRUD.
- Dynamic extra data uses the Extra Data gateway when the pattern is used locally.
- Table 0 in Housing page loads is permissions; subsequent tables are feature data and DDL according to downstream DL ordering.

Verify table and column counts before access, and handle an empty DataSet with a diagnosable message rather than an index exception.

9. SmartComponentService

Receives SmartRequest containing Operation, SpName, and Params. It internally adds @ to parameter names, so do not send @ from the application. It executes QueryMultipleAsync first with fallback to QueryAsync.

Paging/sort/filter features apply only when Operation == "select", while most page gateways use "sp".

Preserve asynchronous I/O and do not create manual connections inside a Controller. Watch value-conversion errors and result-set ordering because they are among the hardest divergence causes in this pattern.

10. ProcedureMapper for Gateways Only

ProcedureMapper registers entry procedures reached by the application, such as shared gateways or a new narrow service. Do not register every Housing DL/SP when MastersDataLoad or MastersCRUD routes to it. The goal is to preserve separation between the application and downstream-procedure details.

11. MastersDataLoad and MastersCRUD

MastersDataLoad:

- Accepts context, then parameter01..parameter50.
- Returns page permissions first.
- Routes by pageName to the feature DL.
- The DL returns page data first, then list sources.

MastersCRUD:

- Routes by pageName and ActionType.
- Checks the operation before calling the downstream SP.
- Adheres to the IsSuccessful and Message contract.
- Passes business errors to the user and logs unexpected errors in ErrorLog.

Preserve contract names, casing, and spelling exactly: pageName, ActionType, idaraID, entrydata, hostname.

12. p01..p50 and parameter01..parameter50 Contract

Form fields are named p01 through p50. CrudController converts them to parameter01 through parameter50. This is a positional contract; p03 may mean something different between operations.

When changing it:

191. Create an operation mapping table: UI field, pNN, parameterNN, downstream procedure parameter.
192. Review table-row alias population and Form fields together.
193. Do not insert a new field in the middle; use an available position or update all layers with a regression test.
194. Validate type/length/nullability against the live parameter, not the Snapshot.

13. Adding a Permission

Add the page operation in page management with an English name matching ActionType. Link it to the page and required parties. In the Controller, read permissionTypeNameE from the first table and use it to display the action. In MastersCRUD, add/verify the branch and operation check before writing. Do not rely on hiding the button alone.

Test: An authorized user can see and execute; an unauthorized user cannot see and is denied on direct request; expired or disabled permission does not work. Do not document a permission name as live verified unless its actual data was verified, because the procedure definition alone does not prove it was granted.

14. Validation and Business Rules

Perform fast validation in the UI to improve experience and repeat it on the server. Place critical business rules in the feature SP within a transaction because it is the final consistency point.

Common live pattern:

- SET NOCOUNT ON, SET XACT_ABORT ON.
- Transaction guarded by @@TRANCOUNT and TRY/CATCH.
- THROW 50001 for expected business errors and 50002 for programmatic failures.
- Prevent duplication and validate active/current state, keys, and administrative scope.
- Soft delete rather than physical deletion where the local pattern follows it.
- Standard success: SELECT 1 AS IsSuccessful, N'...' AS Message.

Do not make the UI the sole source of state, amount, or period rules.

15. Error Handling and Logging

- Business results are displayed through TempData: Success, Warning, Error, Info.

- Errors 50001..50999 are displayable business messages.
- Unexpected errors are recorded in `dbo.ErrorLog` and converted to a generic message.
- Important operations record `dbo.AuditLog` and may create a notification through the gateway.
- Do not log passwords, connection strings, uploaded files, or complete personal/financial data.
- Add safe diagnostic context: page, operation, correlation/time, and a nonsensitive technical identifier.

16. Dependency Injection and Services

Actual registration is in `SmartFoundation.Mvc/Program.cs` and Application extensions. `ConnectionFactory` is singleton, `ISmartComponentService` maps to `SmartComponentService`, and most application services are scoped. `CrudController` is also registered as a service because Controllers use its helpers.

For a new narrow service, use `BaseService`, map only the entry procedure through `ProcedureMapper`, and register the service in `ServiceCollectionExtensions`. Do not migrate an existing Housing page away from `MastersServices + DataSet` without an explicit request.

17. Tests

Test in proportion to risk:

- Unit tests for services, conversion, and validation.
- Integration tests for parameter contracts and result-set ordering in a safe test environment.
- Controller tests for Session, redirects, empty `DataSet`, and permissions.
- Rejection tests for every Business Rule and negative authorization tests.
- Complete mapping test between pNN and procedure parameters.
- Views/`SmartRenderer` tests for the table, form, and buttons.

Reference commands:

```
dotnet build SmartFoundation.Mvc/SmartFoundation.Mvc.csproj
dotnet test SmartFoundation.Application.Tests/SmartFoundation.Application.Tests.csproj
dotnet test SmartFoundation.Application.Tests/SmartFoundation.Application.Tests.csproj --filter
"FullyQualifiedName~EmployeeServiceTests"
```

A full solution build may require SQL project tooling. Do not use a Business SP or production data for testing.

18. Naming Conventions

- Controllers/Actions/Models/Properties: `PascalCase`.
- Locals/parameters/private fields: `camelCase`.
- Partial file: `Controller.Feature.cs` according to the local pattern.
- Do not linguistically improve page names, `ActionType`, or the DB contract; exact matching is more important than elegance.
- Retain historical spellings used by live paths and document them instead of changing them partially.
- Keep usings at the top, remove unused ones, and preserve the formatting of the file you edit.

19. Reconciling Code with the Live Procedure

195. Identify the entry procedure from the service and `ProcedureMapper`.
196. Read the live gateway definition from `sys.procedures` and `sys.sql_modules` and identify the page/operation branch.
197. Read `sys.parameters` for the downstream procedure, then dependencies from `sys.sql_expression_dependencies`.
198. Compare name, order, type, length, expected nullability, result sets, and affected tables.
199. Classify the result as Live verified matching, Live verified with differences, or Not Verified.
200. Update code/documentation based on live state, and do not transfer secrets or business data into documentation.

Queries must be `SELECT` against metadata/definitions only within try/catch. Do not execute a Business SP, write commands, or DDL. If a query fails, record one gap and continue without a loop or helper executable.

20. Delivery Checklist

- ☐ Page and operation match across UI, gateways, and live procedure.
- ☐ Session context, permission table, and server-side authorization are preserved.
- ☐ DataSet and DDL ordering is documented and tested.
- ☐ All pNN fields match live parameters.
- ☐ View is thin and SmartRenderer receives complete configuration.
- ☐ Business Rules are inside a transaction and success/failure messages are standardized.
- ☐ Audit/Error logging does not leak sensitive data.
- ☐ DI and service are registered using the current pattern.
- ☐ Success, rejection, and permission tests were executed.
- ☐ Snapshot was not used to prove live behavior.

21. Gaps That Must Remain Visible

End-to-end tests, actual permission-grant data, import execution, and visual verification of some PDF reports remain outside completed verification scope. Documented gaps also remain in the UploadExcel contract, live DeductListReport binding, some operation names/states, and modal fields. Do not convert any of them into a confirmed fact without appropriate testing.

18. User Guide

1. Before Use

Sign in with your account and select the correct administration. Menus and buttons appear according to your account scope and record state; a missing button means the operation is unavailable to you or the record is not in the appropriate state. After a success message, do not resubmit. For an unexpected error, retain only the page name, operation time, and record identifier; do not send a password, identity, or financial data in the report.

2. Login

Signing In

- Function: Secure access to the system.
- Access: Open the system link.
- Use: Enter the national identity and password, select Remember Me if you want to store the identity number on the device, then click Sign In.
- Conditions: The account is active and the credentials are correct. Remember Me does not store the password.
- Messages: An error may indicate incorrect credentials or an unavailable account/administrative structure.
- Result: Navigation to Home, or to the password-change screen when required.

Password Change and Logout

- Function: Mandatory or optional password change and Session termination.
- Access: The change screen appears after reset or through account settings. Logout is available from the top menu.
- Use: Enter the current password, then the new password and confirmation. At least 8 characters, an uppercase English letter, a lowercase English letter, and a digit are required, and the new password must differ from the current one.
- Conditions: Confirmation matches and the current password is correct.
- Messages: The screen explains any unmet requirement. After success, the system logs out automatically.
- Result: Sign in again with the new password. The system warns after about 8 minutes of inactivity and logs out after about 10 minutes.

3. Home

Home Page

- Function: Starting point and navigation among available programs.
- Access: After sign-in or from the Home button in the top bar.
- Use: Open a program and then a page from the sidebar; use the breadcrumb to identify your location.
- Conditions: Only programs and pages available to your account appear.
- Messages: Returning to Login usually means Session expiration. A missing program requires an access review.
- Result: The selected page opens. Indicator cards may not appear in the current version even though the page loads.

Privacy

- Function: General information page.
- Access: The Privacy link, if displayed.
- Use: View only.
- Conditions and messages: No verified operation.
- Result: Displays temporary technical text; it is not a completed official privacy policy.

4. ControlPanel

PagesManagment

- Function: Manage programs, menus, pages, and their operations.

- Access: ControlPanel, then Page Management.
- Use: Add a program, then a menu or page under the correct location; enter both names, description, link, and order. To add an operation, select the page and enter the operation name and level.
- Conditions: Names are not duplicated, order is positive, and parent/link values are correct.
- Messages: 'Name already used' or 'Operation not registered' indicates a conflict or incomplete definition.
- Result: The new structure appears for appropriate users after access is assigned. Deactivation preserves the record historically.

Permission

- Function: Grant, modify, or stop access for a user, role, or organizational entity.
- Access: ControlPanel, then Permissions.
- Use: Select the target type and complete the search, then select program, page, operation, period, and note. Full access can be granted to a defined scope with care.
- Conditions: Search criteria are complete, assignment is not duplicated, and the authorized user has grant capability.
- Messages: A missing table means a criterion is missing; save rejection means the operation is unavailable to the grantor.
- Result: Adds missing operations or updates their duration; stopping is logical and does not erase history.

Users

- Function: Add, update, and deactivate users, and reset passwords.
- Access: ControlPanel, then Users.
- Use: Enter identity, general number, dates, names, classification, and administration, then save. For update or deactivation, select one user and enter the reason. Reset the password through its dedicated operation.
- Conditions: Identity and general number are unique, and required fields are valid. Identity change is not available to every user.
- Messages: Duplication, lack of permission, or identity-change failure explains the rejection reason.
- Result: Creates/updates the account, deactivates it without deletion, or requires the owner to change the password after reset.

5. Housing - Definitions

Usual path: Housing, then Definitions, then the required page.

BuildingClass - Building Classes

- Function: Define building classes.
- Use: Search or sort, then add the Arabic name and optional English name and notes; or select one record to update/deactivate.
- Conditions: Arabic name is required and unique within scope.
- Messages: Duplication or an inactive record prevents saving.
- Result: A new/updated active class, or its deactivation with history preserved.

BuildingType - Building Types

- Function: Define building types used in building data.
- Use: Add or update the name and available data, or deactivate a selected record; use printing if its button appears.
- Conditions: The required name is unique and the record exists and is active.
- Messages: Duplicate warning or record not found.
- Result: Updates the list of available types. The final print file was not visually verified.

BuildingUtilityType - Building Utility Types

- Function: Define the utility, validity, and whether it requires rent.
- Use: Enter name, description, validity/rent options, then save, or update/deactivate one record.
- Conditions: Required fields and no duplication.
- Messages: Duplicate rejection or unavailable record.

- Result: A utility available for building linkage or historically deactivated.

MilitaryLocation - Locations and Neighborhoods

- Function: Define a location and link it to a city.
- Use: Select the city and enter the name and data, then add, update, or deactivate.
- Conditions: A valid city and a required unique name.
- Messages: Missing city/name or duplicate record.
- Result: A location appears among building-data options.

BuildingDetails - Building Details

- Function: Manage a building, definitions, location, rent, and services.
- Use: Search for the building; to add, select class, type, and location and enter unit, rent, and visible service data. To update, select one building; before deactivation, review associations.
- Conditions: Required fields and links are complete, with no conflict in the unit or current associations.
- Messages: Duplication, existing association, or invalid value prevents the operation.
- Result: A new or updated building becomes available to downstream paths; deactivation does not erase its history. Some update-dialog population still requires operational verification.

6. Housing - Housing Procedures

HousingHandover - Building Handover

- Access: Housing, then Building Handover.
- Function and use: Select the status, review the list, select one building, choose an available action, enter notes, and save.
- Conditions: The building is in the appropriate state and the same action was not previously performed.
- Messages: A duplicate action or changed state causes rejection.
- Result: The building transitions to the next state with action history preserved. Actual operation testing remained unperformed.

HousingResident - Beneficiary Housing

- Access: Housing, then Beneficiary Housing.
- Function and use: Select an approved beneficiary, record custody and notes; complete meter readings if requested; then perform final occupancy with letter number/date and housing date.
- Conditions: The beneficiary is neither housed nor undergoing evacuation in another house, and the building and readings are ready.
- Messages: 'Awaiting meter reading,' 'Not eligible,' or latest-action change.
- Result: Records residence. Cancellation is allowed only in the appropriate state and may affect eligibility.

HousingExtend - Beneficiary Extension

- Access: Housing, then Extension.
- Function and use: Enter approval letter, period, and reason; update/cancel before the next stage; send for financial audit; record required insurance, then approve.
- Conditions: End date is after start, beneficiary is eligible, finance is complete, and required insurance amount is greater than zero.
- Messages: 'Insurance required,' 'Already sent,' or 'Financial audit not complete.'
- Result: Approves or cancels the extension period while preserving the trace. The current page contains documented naming/display differences and requires practical verification.

HousingExit - Beneficiary Evacuation

- Access: Housing, then Evacuation.
- Function and use: Select the occupant, enter evacuation date, note, and penalty, send for financial audit, then approve after completion.
- Conditions: Residence state permits it, financial data is complete, and penalty is a valid number (zero when none).

- Messages: Ineligibility, incomplete audit, or prior execution.
- Result: Ends residence and updates associated effects. Cancellation before approval restores the residence root and deactivates associated ending effects.

7. Housing - Beneficiaries, Waiting, and Assignment

Residents and ResidentsPrint

- Access: Housing, then Beneficiaries; printing from the report button or system navigation to the print page.
- Function and use: Search and add/update identity, general number, name, rank, and unit data, or deactivate the record; select a list report or detailed form.
- Conditions: Required fields and no duplication.
- Messages: Duplicate identity/number or missing data.
- Result: Updated beneficiary file or printable report. Not all PDF outputs were visually verified.

WaitingListByResident

- Access: Housing, then Waiting List by Beneficiary.
- Function and use: Search by identity; add a waiting class and decision data; review waiting, letter, and transfer tables; update/cancel the correct row or end all lists with a documented reason.
- Conditions: No duplicate class, file belongs to the administration, and eligibility/state permits it.
- Messages: Duplicate class, existing transfer request, different administration, or expired eligibility.
- Result: Creates, updates, or ends waiting while preserving history.

WaitingListMoveList

- Access: Housing, then Waiting-List Transfer Requests.
- Function and use: Review identity and sending administration, then accept or reject the incoming request.
- Conditions: The request is still pending and was not previously processed.
- Messages: 'Action already taken' when state changed.
- Result: Acceptance transfers the waiting file and its relationships; rejection keeps it at its source.

WaitingList

- Access: Housing, then Waiting List.
- Function and use: Select the class, review ordering, and move only the head of the list to assignment; print the list if needed.
- Conditions: Beneficiary is at the class head and state permits it.
- Messages: 'Not at the head of the waiting list' or record changed.
- Result: Moves the eligible beneficiary to assignment or displays/prints the list. Printing was not fully visually verified.

Assign

- Access: Housing, then Assignment.
- Function and use: Select the class, open allocation minutes, assign an available building, replace it or exclude the beneficiary as needed, then close the minutes.
- Conditions: No conflicting active minutes, building is available, and at least one allocation exists before closing.
- Messages: Existing minutes, unavailable house, or changed record.
- Result: Closed minutes preserve all allocation, replacement, and exclusion actions.

AssignStatus

- Access: Housing, then Assignment Statuses.
- Function and use: Select closed minutes, process each beneficiary's status, print the row form if needed, then end the minutes after processing completes.
- Conditions: Minutes are closed and statuses were not previously processed.
- Messages: Status unavailable or already executed.
- Result: Finalizes minutes results. Actual access-grant data and printing remained practically unverified.

OtherWaitingList

- Access: Housing, then Other Waiting Lists.
- Function and use: Select local class, beneficiary, and building, then move the beneficiary to occupancy procedures; print a class report when needed.
- Conditions: Beneficiary and building are available and state permits it.
- Messages: Building unavailable or beneficiary state changed.
- Result: Moves directly to occupancy without general allocation minutes.

RentExemption

- Access: Housing, then Rent Exemption.
- Function and use: Search by current occupant identity; enter exemption type, period, letter, and description; update or cancel with a clear reason.
- Conditions: Current occupant, valid period, and no conflicting active record.
- Messages: Ineligible beneficiary or conflicting period/record.
- Result: An active, expired, or canceled exemption affects the overlapping rent portion according to its type.

UploadExcel

- Access: Housing, then Excel Upload.
- Function and use: Upload one .xls or .xlsx file up to 10 MB; review the first sheet and preview (up to 200 rows), select three distinct columns without empty cells, then commit.
- Conditions: Valid type/size and no duplicate file hash.
- Messages: Duplicate file, repeated columns, or rows with empty values.
- Result: Preview, then import attempt. Do not use commit in production before the import gap is declared closed and operationally verified; no actual import was performed during documentation.

UploadLab

- Access: Housing, then Upload Lab.
- Function and use: Upload one PDF or Excel file up to 10 MB and review its name, path, and time in the Session list.
- Conditions: Allowed type and size.
- Messages: Type/size rejection or save failure.
- Result: Saves a file and displays it in Session only; it does not import data and has no verified page deletion, so do not upload sensitive files.

8. IncomeSystem

DeductListReport

- Access: IncomeSystem, then Deduction Batch Report.
- Function and use: Select service type, then period, and click display/print.
- Conditions: An available service and period.
- Messages: The page may display no data even with valid filters.
- Result: Intended report. The live data path is not currently connected, so the page is not functionally verified and printing is not visually verified.

FinancialAuditForExtendAndEvictions

- Access: IncomeSystem, then Extension and Evacuation Audit.
- Function and use: Select the request, review claims and payments, then complete audit or pay/refund/settle between two services as available.
- Conditions: Settlement services differ, amount is valid and within balance, and request was not previously completed.
- Messages: Invalid amount, insufficient balance, or completed action.
- Result: Updates audit and financial action while preserving the trace.

FinancialAuditForUser

- Access: IncomeSystem, then Beneficiary Financial Audit.
- Function and use: Search for the beneficiary, review summary and bills, then review or pay/refund/settle according to available action.
- Conditions: Valid beneficiary and records, with amount within balance.
- Messages: Record not found, invalid amount, or insufficient balance.
- Result: Updates the beneficiary's financial file.

ExtendInsurance

- Access: IncomeSystem, then Extension Insurance.
- Function and use: Review amount, remaining amount, and total; enter approval-document number/date and note, then approve; use reports if shown.
- Conditions: Request is visible, total is nonzero, and it was not previously approved.
- Messages: Record not found, invalid amount/state, or prior approval.
- Result: Approves collection and links it to the extension request. Reports were not fully visually verified.

ImportExcelForBuildingPayment

- Access: IncomeSystem, then Excel Batch Import.
- Function and use: Upload the file; map identity, unit, general-number, and amount columns; select claim type, year, month, and batch data; review preview, then process once.
- Conditions: Four valid columns, no empty cells, numeric amounts, and a nonduplicate file.
- Messages: Empty row, invalid amount, or previously imported file.
- Result: Creates a batch and payments and displays processing counts. No actual import was performed during documentation; do not test with a production file without an approval plan.

9. ElectronicBillSystem

Meters

- Access: ElectronicBillsSystem, then Meters.
- Function and use: Define meter type, service, and calculation method; add the meter and initial reading, then link it to a building; record the unlink reading when unlinking.
- Conditions: Type name is unique, meter and building are available, and no links prevent deactivation.
- Messages: Existing record, existing link, or missing data.
- Result: A defined and historically linked meter. Some meter/link-management buttons require practical visibility verification.

AllMeterRead

- Access: ElectronicBillSystem, then Periodic Reading.
- Function and use: Select electricity, water, or gas; open the monthly period if absent; enter readings; correct them before closing; close the period after completion.
- Conditions: One period per service, administration, and month; reading is not below the previous one except for a valid rollover; and all meters are complete before closing.
- Messages: Period not open, reading lower, existing record, or incomplete readings.
- Result: Converts closed readings into the claim basis. PDF link and title were not visually verified.

MeterReadForOccupentAndExit

- Access: ElectronicBillSystem, then Occupancy and Evacuation Reading.
- Function and use: Select the occupant, review action type, record each linked meter reading, then approve after all are complete.
- Conditions: Valid occupancy/evacuation action and all readings complete.
- Messages: 'Readings not complete,' invalid reading, or action state changed.
- Result: Completes the occupancy/evacuation action and creates or updates the financial effect when applicable. Casing of some operation names requires actual verification.

MeterServiceTypeFixedAmount

- Access: ElectronicBillSystem, then Fixed Prices.
- Function and use: Select the service and enter amount and start date; update or deactivate the price when needed.
- Conditions: No other active record exists for the service and administration, and amount/date are valid.
- Messages: Active record exists or invalid value.
- Result: Update ends the previous price and starts a new one; deactivation may stop calculation for buildings without an alternative, so review before confirming.

10. Shared Messages and Alerts

- Success: The record was updated; verify the result and do not resubmit.
- Warning/existing record: Review the current record before creating a replacement.
- Not permitted: The operation is unavailable to your account or record state; contact the administrator.
- Record not found/action already performed: Refresh the page because state changed.
- Unexpected error: Do not manually alter data or repeatedly retry; send page name, time, and record identifier to support.

11. Verification Boundaries

All in-scope pages were documented from active code, and database-related information was reconciled from live definitions without executing business operations or writing data. Operationally unverified items remain:

DeductListReport due to missing live data path, execution of Housing and Income imports, UploadLab deletion and operational policy, some dialog population, operation names and state transitions, actual granted-access data, and comprehensive visual verification of PDF outputs. No screenshots or unverified steps were created.

19. Gap and Risk Register

Note: Historical closure note (2026-08-30): component-specific evidence below is retained for audit history. Component removed from official release. Final security assessment status: NOT STARTED.

Gap 01 — Client-controlled CRUD context

- Status: CLOSED
- Closed Date: 2026-08-24
- trusted Session/server-side entrydata
- trusted Session/server-side idaraID
- trusted server-side hostname
- fixed identity fallbacks removed
- pageName constrained
- ActionType permission checked and explicitly constrained
- The retired component's historical identity-tampering and feedback-ownership checks passed before removal from the official release.
- Support ActionType whitelist tests PASS
- unknown page/action rejection PASS
- missing Session test PASS
- full MastersCRUD block review PASS
- MVC Build: 0 Errors / 0 Warnings
- live changes applied successfully to:
- dbo.MastersCRUD
- the retired component's feedback procedure (removed from the official release)
- test records cleaned with ROLLBACK
- final decision: Gap 01 CLOSED

Gap 02 — Fixed/default password in user password reset

- Status: CLOSED
- Closed Date: 2026-08-24
- Reset password remains Aa123456 by approved operational decision.
- Reset password validity is limited to 24 hours.
- Reset creates new Salt/Hash.
- ChangedPassword = 0 after reset.
- userPasswordStartDate and userPasswordEndDate enforce reset validity.
- expired temporary password login is denied.
- legacy ChangedPassword=0 with NULL expiry is denied.
- password change is enforced Server-side.
- direct application page access before password change is blocked.
- /crud/ and /api/ are blocked Server-side while password change is required.
- successful password change sets normal password state and clears Session.
- user must login again with the new password.
- existing users with ChangedPassword=1 are unaffected.
- no bulk UPDATE was executed against the 75 legacy accounts.
- live SQL deployment succeeded for:
- dbo.ReSetUserPassword
- dbo.GetSessionInfoForMVC
- Middleware tests: PASS.
- SQL tests A/B/F/G/H/I/J: PASS.
- transaction safety and rollback: PASS.
- test user count after rollback: 0.

- test password rows after rollback: 0.
- related test rows after rollback: 0.
- MVC Build: PASS, 0 Errors / 0 Warnings.
- final decision: Gap 02 CLOSED

Gap 03 — Authentication Scheme / Claims / Authorize coverage

- Status: CLOSED
- Closed Date: 2026-08-24
- ASP.NET Core Cookie Authentication enabled.
- Authentication Scheme: Cookies.
- Cookie name: .SmartFoundation.Auth.
- ClaimsPrincipal created on successful Login.
- ClaimTypes.NameIdentifier uses trusted usersID.
- ClaimTypes.Name uses trusted fullName.
- SignInAsync implemented.
- SignOutAsync implemented.
- Session continues to hold application context.
- Fallback Authorization Policy requires authenticated users by default.
- Controllers reviewed: 23.
- [AllowAnonymous] limited to:
- LoginController.Index
- LoginController.CheckLogin
- Reports, Exports, Uploads, SmartComponent, CRUD and business APIs are no longer anonymously accessible.
- SessionGuard performs authenticated-cookie/session consistency validation.
- Tests A-G: PASS.
- Tests H-I-J-K-L: PASS.
- Fallback Policy audit: PASS.
- Gap 02 regression: NO.
- Authentication Cookie + Session behavior: PASS.
- Logout behavior: PASS.
- Cookie-without-Session protection: PASS.
- Test users cleaned successfully.
- Remaining test users: 0.
- Related password rows: 0.
- Related test rows: 0.
- Build Errors: 0.
- New warnings introduced by Gap 03: 0.
- Existing legacy warnings: 324.
- final decision: Gap 03 CLOSED

Gap 04 — SessionGuardMiddleware registration / enforcement

- Status: CLOSED
- Closed Date: 2026-08-24
- Pipeline:
- UseStaticFiles
- UseRouting
- UseSession
- UseAuthentication
- UseAuthorization
- SessionGuardMiddleware
- ForcePasswordChangeMiddleware
- MapRazorPages / MapControllers / MapControllerRoute
- SessionGuard registered exactly once.

- Required Session keys:
- usersID
- fullName
- IdaraID
- nationalID
- ClaimTypes.NameIdentifier is compared Server-side with Session["usersID"].
- Missing or mismatched Session identity triggers SignOutAsync + Session.Clear.
- MVC redirects to Login.
- API/CRUD return 401.
- Session-only without Authentication Cookie is rejected.
- Tests A-L: PASS.
- Gap 02 regression: NO.
- Gap 03 regression: NO.
- Build: 0 Errors.
- New warnings introduced by Gap 04: 0.
- Legacy warnings: 324.
- No final source-code modification was required during Gap 04 verification.
- final decision: Gap 04 CLOSED

Gap 05 — Antiforgery coverage

- Status: CLOSED
- Closed Date: 2026-08-24
- AutoValidateAntiforgeryTokenAttribute registered centrally on MVC.
- Antiforgery header: RequestVerificationToken.
- Tokens generated centrally using IAntiforgery.GetAndStoreTokens() and exposed by the Layout.
- SmartForm uses @Html.AntiForgeryToken() and does not derive tokens from request headers.
- Central JavaScript adds the token to same-origin POST, PUT, PATCH, and DELETE requests.
- State-changing GET remediation:
- Logout converted to protected POST.
- SignOut and Session.Clear removed from GET Login/Index.
- state-changing Session/file cleanup removed from Excel GET paths.
- Unsafe endpoints reviewed: 24.
- Protected: 24.
- Unprotected: 0.
- Not Applicable: 0.
- IgnoreAntiforgeryToken: 0.
- Runtime tests A-O: PASS.
- Gap 01 regression: NO.
- Gap 02 regression: NO.
- Gap 03 regression: NO.
- Gap 04 regression: NO.
- Rejected requests were blocked before business logic:
- ChangePassword state unchanged.
- AI rows: 0.
- Notification rows: 0.
- SmartComponent did not reach DataEngine.
- Export was not executed.
- Logout did not execute SignOut or Session.Clear.
- Build: PASS, 0 Errors / 0 Warnings.
- Cleanup: PASS.
- Test users: 0.
- Test permissions: 0.
- Test password rows: 0.

- Test notifications: 0.
- Test AI rows: 0.
- Test upload files: 0.
- Related test data: 0.
- final decision: Gap 05 CLOSED

Gap 06 — Sensitive exception messages exposed

- Status: CLOSED
- Closed Date: 2026-08-24
- Confirmed exception-detail response findings remediated: 27/27.
- Review findings sanitized: 45/45.
- Client-facing exception messages replaced with safe Arabic messages.
- Stored-procedure and SQL exception details are retained only in server-side logging/inner exceptions.
- Business-validation messages returned intentionally by stored procedures/DataSets remain unchanged.
- Final static scan:
- VULNERABLE: 0.
- REVIEW: 0.
- SAFE server-side only: 2.
- MVC Release Build: PASS, 0 Errors.
- Existing legacy warnings: 324.
- Focused application tests: PASS, 12/12.
- Runtime verification of CRUD, Login, Export, Excel, Support, Vehicle, AJAX, and Network responses: PASS.
- Final decision: Gap 06 CLOSED.

Gap 07 — Housing.UploadExcelRowType schema mismatch

- Status: CLOSED
- Closed Date: 2026-08-24
- The mismatch between Housing.UploadExcelRowType and Housing.UploadExcelImportSelected3Cols is limited to an unused legacy/orphan flow.
- UploadExcelImportSelected3Cols has one project reference, in the legacy UploadExcelController.
- No other active project references target Housing/UploadExcel.
- No matching page exists in dbo.Menu or dbo.VGetFullPermissionDetails.
- No SQL dependencies target Housing.UploadExcelImportSelected3Cols.
- Housing.UploadExcelRowType is not used by another stored procedure.
- No current execution appeared in dmexecprocedurestats during verification.
- The legacy direct route /UploadExcel was disabled by marking UploadExcelController with [NonController].
- Runtime verification confirmed /UploadExcel is no longer accessible.
- The active /IncomeSystem/ImportExcelForBuildingPayment flow remained operational.
- The active import flow had previously completed a runtime import of 1,591 rows with the correct column selection.
- No SQL schema, stored procedure, or user-defined type change was required.
- Legacy SQL object cleanup is deferred as independent dead-code/legacy cleanup.
- Final decision: Gap 07 CLOSED.

Gap 08 — DeductListReport live routing not proven

- Status: CLOSED
- Closed Date: 2026-08-24
- The /IncomeSystem/DeductListReport route exists and its routing has been proven operational.
- The page itself is an incomplete feature and has not entered active production use.
- Completion remains separate future feature work and is not an open security gap.
- No change was made to the feature as part of the current security-gap work.
- Final decision: Gap 08 CLOSED.

Gap 09 — Stored Procedures drift between SQL project and live DATACORE

- Status: CLOSED
- Closed Date: 2026-08-24
- Live DATACORE is the source of truth for the database schema.
- Git commit 75beb428808b85ea0b81613a9da5f94a0e6c7e87 dated 2026-08-22 synchronized live definitions into SmartFoundation.Database.
- Stored procedures, tables, views, functions, Maintenance, MoveData, Security objects, and the SQL project file were synchronized or cleaned.
- Housing.ImportExcelForBuildingPaymentSP was added from its live definition.
- Obsolete views, stale WH references, environment-specific SAMIDEV security entries, a duplicate Housing.GenerateExitRentBills definition, and stale project references were cleaned.
- SSDT-inappropriate leading statements were removed without changing business logic.
- Final SmartFoundation.Database Rebuild: PASS, 0 Errors.
- Final decision: Gap 09 CLOSED.

Gap 10 — Live triggers missing from SQL snapshot

- Status: CLOSED
- Closed Date: 2026-08-24
- Live DATACORE contains two active triggers that were missing from SmartFoundation.Database.
- Housing.TRBuildingPaymentSetLinkStatus:
 - Parent: Housing.BuildingPayment
 - Events: AFTER INSERT, UPDATE
- Housing.trgBuildingActionAudit:
 - Parent: Housing.BuildingAction
 - Events: AFTER INSERT, UPDATE, DELETE
- Both trigger definitions were added to the SQL snapshot with live DATACORE retained as the source of truth.
- Final decision: Gap 10 CLOSED.

Gap 11 — In-Memory Session risks

- Status: CLOSED
- Closed Date: 2026-08-24
- The current deployment uses one application server; the database may be on a separate server without changing the Session model.
- Hosting and infrastructure are managed externally and are outside the application's current responsibility boundary.
- The application uses AddDistributedMemoryCache() and AddSession() with process-local Session storage.
- Session idle timeout: 10 minutes.
- Session cookie: HttpOnly = true, IsEssential = true, SecurePolicy = Always.
- Authentication cookie duration: 10 minutes with sliding expiration.
- Middleware order: Routing, Session, Authentication, Authorization, then SessionGuardMiddleware.
- Runtime restart verification:
 - Login succeeded and /Housing/BuildingType was accessible.
 - After stopping and restarting the application, refreshing the page redirected to /Login?ReturnUrl=%2FHousing%2FBuildingType.
 - The old Session was lost as expected.
 - Authentication Cookie alone did not preserve access.
 - SessionGuardMiddleware failed closed without HTTP 500.
- In-memory Session is accepted for the current single-server environment.
- Reassess if multiple application servers/instances, a load balancer without Session affinity, or persistence across application restarts becomes required.
- Final decision: Gap 11 CLOSED.

Gap 12 — Weak Content Security Policy

- Status: CLOSED
- Closed Date: 2026-08-24
- Content Security Policy is present and active on protected application responses.
- Runtime response verification on /Housing/BuildingType confirmed the configured CSP header.
- OWASP ZAP 2.17.0 baseline scan:
- High: 0
- Medium: 0
- Low: 0
- Informational: 5
- False Positives: 0
- ZAP reported no alert for CSP, unsafe-inline, or unsafe-eval.
- Removing 'unsafe-eval' from script-src was tested at runtime and caused multiple JavaScript failures.
- Alpine.js generated explicit CSP EvalError messages and the page was functionally affected.
- 'unsafe-eval' was restored to preserve current application behavior.
- The current CSP is accepted for the existing architecture; replacing Alpine.js or redesigning JavaScript is outside this gap.
- Reassess only after relevant JavaScript/CSP architecture changes or new security evidence.
- Final decision: Gap 12 CLOSED.

Gap 13 — Forwarded Headers not proven

- Status: CLOSED
- Closed Date: 2026-08-24
- The system operates inside a fully closed local network and is hosted directly on IIS.
- Users connect directly to IIS.
- No reverse proxy, load balancer, WAF, application gateway, or similar intermediary exists in front of the application.
- There is no known operational dependency on X-Forwarded-For, X-Forwarded-Proto, or X-Forwarded-Host.
- The absence of UseForwardedHeaders() is appropriate for the current hosting topology.
- Reassess if the hosting architecture changes and an intermediary begins altering client IP, protocol, or host.
- Final decision: Gap 13 CLOSED.

Gap 14 — Critical end-to-end tests incomplete

- Status: CLOSED
- Closed Date: 2026-08-24
- End-to-end runtime verification covered three operational pages from different modules:
- IncomeSystem: /IncomeSystem/FinancialAuditForExtendAndEvictions
- Housing: /Housing/BuildingType
- ElectronicBillSystem: /ElectronicBillSystem/AllMeterRead
- Page Load, Data Load, User Actions, operation results, and absence of visible runtime errors were verified.
- Prior integration evidence also covers Login, SessionGuard, Session loss after restart, CRUD Insert/Update/Delete, Antiforgery, ChangePassword, Excel import, PDF/Print, AJAX/Modals, and Network/Console regression checks.
- Representative critical workflows were proven end-to-end across multiple modules; exhaustive testing of every page is not required for Gap 14 closure.
- Final decision: Gap 14 CLOSED.

Gap 15 — Operational coverage of business stored procedures incomplete

- Status: CLOSED
- Closed Date: 2026-08-24

- Operational pages follow the established PageDL data-loading and PageSP business-operation pattern.
- PageDL routes through dbo.MastersDataLoad.
- PageSP routes through dbo.MastersCRUD.
- The central routing and execution pattern was previously reviewed and verified.
- Runtime testing across Housing, IncomeSystem, and ElectronicBillSystem proved Data Load, CRUD, Business Actions, and Stored Procedure routing.
- Testing every stored procedure individually is not required because the central route is proven, the execution pattern is consistent, and operational pages from multiple modules were tested.
- Final decision: Gap 15 CLOSED.

Gap 16 — Excel import end-to-end testing incomplete

- Status: CLOSED
- Closed Date: 2026-08-24
- The active /IncomeSystem/ImportExcelForBuildingPayment flow was verified end-to-end.
- Verification covered Excel selection and reading, column selection/mapping, validation, TVP/DataTable conversion, stored-procedure execution, persistence in DATACORE, and the returned user result.
- A real import completed successfully with approximately 1,591 rows.
- Incorrect column selection, invalid amount values, and unsuitable data were handled as validation/business-data errors rather than failures in the TVP, stored procedure, or import path.
- Final decision: Gap 16 CLOSED.

Gap 17 — Real users / roles / permissions testing incomplete

- Status: CLOSED
- Closed Date: 2026-08-24
- Real users with and without the required permissions were tested.
- Allowed actions were shown and executed according to permission; unauthorized actions were denied server-side, not merely hidden in the UI.
- Prior tests also covered MastersCRUD permission and ActionType validation, cross-page tampering, unknown pages, missing Session, and server-side enforcement.
- Final decision: Gap 17 CLOSED.

Gap 18 — Runtime verification of modals and state transitions incomplete

- Status: CLOSED
- Closed Date: 2026-08-24
- Runtime tests verified modal open/close, correct modal data, Save/Confirm/Cancel, UI refresh, and allowed state transitions.
- No incorrect transition, visible runtime failure, or HTTP 500 occurred in the tested CRUD/AJAX/modal flows.
- Final decision: Gap 18 CLOSED.

Gap 19 — SQL Server edition/version not formally documented

- Status: CLOSED
- Closed Date: 2026-08-24
- Reference environment used for development and verification: SQL Server 2019; Developer Edition is stated only as the Edition of the reference environment.
- The production environment uses a supported SQL Server version and an appropriately licensed Edition in accordance with the hosting entity's policies; Developer Edition is not a Production Requirement.
- A personal development-machine SQL Server installation is not the reference for this decision.
- Final decision: Gap 19 CLOSED.

Gap 20 — Recovery Model not documented

- Status: CLOSED
- Closed Date: 2026-08-24

- Official database: DATACORE.
- Recovery Model: FULL.
- Final decision: Gap 20 CLOSED.

Gap 21 — SQL Server Agent Jobs not fully documented

- Status: CLOSED
- Closed Date: 2026-08-24
- Verification of the actual SQL Server environment found no currently defined or used SQL Server Agent Jobs.
- Final decision: Gap 21 CLOSED.

Gap 22 — RPO / RTO not defined

- Status: CLOSED
- Closed Date: 2026-08-24
- Official RPO: 15 minutes.
- Official RTO: 1 hour.
- These values are the project's operational disaster-recovery targets.
- Final decision: Gap 22 CLOSED.

Gap 23 — Backup / retention / restore testing incomplete

- Status: CLOSED
- Closed Date: 2026-08-24
- Daily Full Backup, Differential Backup, and Transaction Log Backup are present.
- Retention: one year.
- A real Restore test was previously completed.
- Restoring under a different database name requires attention to explicit references such as DATACORE.dbo., which may continue targeting DATACORE.
- The hosting/operator organization owns backup scheduling needed to meet RPO 15 minutes and RTO 1 hour, including an appropriate Transaction Log Backup schedule.
- Final decision: Gap 23 CLOSED.

Gap 24 — External integrations runtime verification incomplete

- Status: CLOSED
- Closed Date: 2026-08-24
- The system runs in a fully closed local network without Internet connectivity.
- There are currently no operational external APIs, external SMTP/SMS/SSO, Internet web services, or Internet-dependent cloud integrations.
- The database and core services remain inside the internal environment.
- Reassess only when a new external integration is introduced.
- Final decision: Gap 24 CLOSED.

Summary

- TOTAL: 24
- CLOSED: 24
- OPEN: 0
- IN PROGRESS: 0
- DEFERRED: 0

20. Final Coverage Audit

Version 1.0.0 Update - 2026-09-05

- Successful Solution build, publish output generation, and database project build using its dedicated tools were previously verified.
- Release-specific security verification was completed within its scopes: SAST, SCA, SBOM, DAST/ZAP, manual verification, and OWASP ASVS 5.0 alignment.
- IAST was not performed within the development team's scope and remains part of verification and acceptance by the responsible entity before final hosting.
- These results do not represent Final Production Approval or Security Certification; acceptance remains tied to the actual hosting environment.
- Historical audit results and counts below are preserved and are not reinterpreted as execution of every Business Stored Procedure or testing of every permission.

Audit date: 2026-08-23. Scope is limited to the included programs: ControlPanel, Housing, IncomeSystem, ElectronicBillSystem, Home, and Login. Deferred/excluded programs are absent from every count and percentage. Shared surfaces are counted only when directly serving included programs.

Official Status Update - 2026-08-24

- This file preserves historical coverage-audit results dated 2026-08-23.
- The current reference for gap status is Documentation/security-gap-register.md and Documentation/security-gap-status.md.
- Current official status: TOTAL 24, CLOSED 24, OPEN 0, DEFERRED 0, IN PROGRESS 0.
- Unverified observations below describe the historical audit point and do not represent currently open gaps.

Evidence Method

- Application behavior and counts of programs, pages, Controller files, Logical Controllers, Actions, Views, services, report paths, and Workflows: Active repository Code.
- Tables, SQL Views, Stored Procedures, Functions, and Triggers: Live Database catalog and DATACORE definitions read through SELECT-only queries.
- SmartFoundation.Database is not a final SQL source; it was used only for historical comparison and Schema drift measurement.
- 'Documented' means an element was inventoried and linked to its function or description in the document; it does not mean a Business Stored Procedure was executed or the path was tested E2E.

Final Coverage Table

Not Verified	Evidence Source	Percentage	Documented	Discovered	Category
None within scope; 8 Excluded programs are outside the denominator.	Code	100%	6	6	Programs
No comprehensive runtime execution was performed for every file.	Code	100%	38	38	Controller files
None at the inventory level.	Code	100%	8	8	Logical Controllers
Not all Actions were tested using actual accounts and permissions.	Code	100%	45	45	Actions

Not Verified	Evidence Source	Percentage	Documented	Discovered	Category
No comprehensive Browser/E2E test was performed for every View.	Code	100%	35	35	Views
Runtime integration verification was limited; it covered MastersServices, CrudController, Chart, and SmartComponentService.	Code	100%	4	4	Services
Business data, row quality, and tables outside the program scope were not read.	Live Database	100%	118	118	Tables
Its results were not compared with actual business data.	Live Database	100%	44	44	SQL Views
Business Stored Procedures were not executed; coverage was catalog/definition/dependency based.	Live Database	100%	76	76	Stored Procedures
Results were not tested against actual business data.	Live Database	100%	34	34	Functions
They were not run; impact is documented from definitions and dependencies only.	Live Database	100%	2	2	Triggers
Actual grants for users/roles were not read; no E2E authorization test was performed.	Code + Live Database	100%	113	113	Permissions
DeductListReport has no live routing; PDF output was not inspected for every report.	Code + Live Database	100%	17	17	Reports
State transitions and imports were not executed E2E.	Code + Live Database	100%	24	24	Workflows
Operational behavior was not tested page by page with an actual account.	Code	100%	35	35	User Manual pages

Count Summary

- 6 included programs, 35 user pages, 38 Controller files representing 8 Logical Controllers, and 35 Views.
- 45 Actions and 4 shared operational services directly related to scope.
- 274 live SQL objects in scope: 118 tables + 44 Views + 76 Stored Procedures + 34 Functions + 2 Triggers.
- 113 permission names/controls, 17 report paths, and 24 documented Workflow/source diagrams.

Live and Snapshot Differences

- The final reference is live DATACORE. Reconciliation 7A documented 11 Stored Procedures and Housing.VWaitingList that differ from the Snapshot; live descriptions replaced old claims in final chapters.
- Two live Triggers are absent from the Snapshot: Housing.trgBuildingActionAudit and Housing.TRBuildingPaymentSetLinkStatus.
- No in-scope Snapshot object was proven missing from Live in the final inventory.
- The live Housing.UploadExcelRowType has five columns while the Controller builds a four-column DataTable; this is a Code-vs-Live gap, not a fact inferred from the Snapshot.
- IncomeSystem, ElectronicBillSystem, and Home/Login phases added no new functional drift to compared procedures; the DeductListReport difference is a live routing gap.

Gap and Risk Register at the Historical Audit Point

Note: All following items were formally closed on 2026-08-24. This list is retained as historical evidence and is not used to determine current status.

201. Critical: The CRUD contract accepts entrydata, idaraID, pageName, and ActionType from the client instead of deriving context exclusively from Session.
202. Critical: Administrative reset uses a fixed default value in the live definition; the value itself is not displayed in documentation.
203. High at historical audit time: Authentication Scheme/Claims/general SessionGuard were incomplete. This state was later closed; see the version 1.0.0 update above and the gap register.
204. High: UploadExcelRowType mismatch, and no import was executed.
205. Medium: DeductListReport has no verified live routing, and some modal/state transitions require E2E.
206. Medium: Actual permission-grant data was not read, and Business Stored Procedures or E2E tests were not executed.
207. Medium: SQL Server version/edition, Recovery Model, SQL Agent jobs, RPO/RTO, retention, and restore drills were unverified.
208. Medium: Two live Triggers absent from the Snapshot increase drift risk when deploying from the database project.

Privacy and Secrets

Documentation sources and final text were scanned for connection strings, passwords, keys/tokens, email, phone numbers, and identity. No connection secrets, reset password, business-row data, or actual personal data are displayed. Object names and technical paths are not personal data.

Delivery Decision

Documentation coverage of discovered surfaces within included programs is 100% complete. The list above represented declared operational/security verification gaps or Schema drift at the 2026-08-23 audit point; all gaps were officially closed on 2026-08-24. The pdf_and_final_audit phase may be marked complete only after successful Word and PDF rendering, inspection of every page, and updating the TOC and page numbers.

Documentation Closure Update - 2026-08-30

Removal of the excluded component from source, SQL Project, and live DATACORE was completed, and its live object count became zero according to official manual verification provided by the owner. Build, unit, startup, clean-publish, and manual operational tests succeeded; documentation sources and prompts were updated, and DOCX/PDF regenerated. No stale active operational references remain. Removal status: CLOSED. The next phase ready to begin is Final Security Assessment, which has not started.

21. Release 1.0.0 Security and Operational Status

Update date: 2026-09-05.

Purpose and Claim Boundaries

This chapter documents the verified current state of release 1.0.0 after completion of pre-delivery verification activities. It is a technical appendix to the comprehensive document, not a security certificate or final production approval. Results apply to the release, scope, and environment at verification time; post-deployment operational acceptance remains tied to the actual hosting environment.

Trust order used: Current code and project files, then final verification evidence, then approved delivery document, then historical documentation. The document contains no connection secrets, passwords, or actual user data.

Release Readiness

Qualification	Documented Status	Item
According to the previous release verification record; execution was not repeated during this documentation update.	Success Verified	Solution build
This does not constitute production deployment or operational acceptance.	Successfully Produced	Publish output
The database project is a reference Snapshot, while live DATACORE remained the source of truth in the previous reconciliation.	Verified using the designated build tools	Database project build
This does not mean every user, permission, or business path was tested in every possible environment.	Verified within the defined release scope	Functional Scope
This does not constitute absolute security certification or a guarantee that the system is vulnerability-free.	The specified activities were performed within their scopes	Pre-delivery Security Verification
Includes IIS, TLS, connectivity, backup, restore, monitoring, and appropriate functional tests.	Required in the actual hosting environment	Post-deployment Acceptance

Secure Coding and OWASP Top 10

Secure coding practices and controls related to authentication, authorization, session management, input validation, error handling, and protection of sensitive operations were considered during release development, guided by common OWASP Top 10 risk categories. This description does not claim full compliance with every OWASP requirement and is not an independent certification.

Final Security Verification Status

Result Boundaries and Source	Result	Activity
Quality Gate = PASS, Security Rating = A, and 0 Open Vulnerabilities according to the final assessment record and final Sonar result. The result is limited to the code and configuration scope at the time of scanning.	PASS	SAST
NuGet and npm analysis recorded 0 known vulnerable packages remaining at scan time. This result does not automatically extend to later updates or vulnerability information published in the future.	PASS within the scan scope	SCA

Result Boundaries and Source	Result	Activity
A separate CycloneDX inventory was created for NuGet and npm components, with a SHA-256 checksum manifest. The SBOM is a component and dependency inventory, not an independent vulnerability test.	COMPLETED	SBOM
The scan was performed against a representative operational scope. The final report recorded High=0, Medium=0, and Low=0, with informational notes; this does not represent testing every Endpoint.	PASS WITHIN TESTED SCOPE	DAST / OWASP ZAP
The result is recorded in the final security assessment tracking file for representative scenarios. The current evidence package contains no clearly independent files under the Manual folder; therefore, this document does not claim that they exist.	PASS WITHIN TESTED SCOPE	Manual Security Verification
Evidence-based reference alignment at the ASVS 5.0.0 chapter level; it is neither Certification nor Full Compliance.	COMPLETED WITHIN ASSESSED SCOPE	OWASP ASVS 5.0 Alignment
It was not performed by the development team within the current verification scope; it is left to the competent authority or its assignee as part of verification and acceptance before final hosting.	NOT PERFORMED	IAST

Interactive Application Security Testing

Interactive Application Security Testing could not be performed within the development team's verification scope because no suitable free or open-source tool was available that supports the application's technical environment and achieves the required testing level. The IAST result must not be recorded as PASS or COMPLETED.

The responsible entity, or its assigned party, performs the test as part of verification and acceptance before final hosting. If it produces a Finding or Vulnerability, the development team handles remediation and re-verification before final approval.

Software Component Inventory

The following names and versions were verified from csproj files, package-lock, local asset headers, or SBOM. Where package.json contains a dependency range, the table uses the version actually locked in package-lock.

Verification Source or Note	Verified Version	Component
TargetFramework = net8.0 and the MVC project.	8	.NET / ASP.NET Core MVC
SmartFoundation.DataEngine.csproj.	6.1.1	Microsoft.Data.SqlClient
SmartFoundation.DataEngine.csproj.	2.1.66	Dapper
SmartFoundation.Mvc.csproj; its use is subject to its license terms, and this document provides no legal opinion.	2025.12.1	QuestPDF
SmartFoundation.Mvc.csproj.	3.8.0	ExcelDataReader and ExcelDataReader.DataSet
package-lock; package.json permits a range starting at 4.3.2.	4.3.3	Tailwind CSS
package-lock.	3.14.9	Alpine.js

Verification Source or Note	Verified Version	Component
package-lock.	3.1.2	Flowbite
package-lock.	1.3.2	Flowbite Datepicker
package-lock and the local asset.	4.6.13	Flatpickr
The local asset and the approved release inventory.	3.0.11	DOMPurify
The local asset and the approved release inventory.	4.4.1	Chart.js
The local asset and the approved release inventory.	2.2.0	Chart.js DataLabels Plugin
The local asset name and its header.	3.7.1	jQuery
The version header/property in the local asset.	2.1.3	Toastr
The local asset and the approved release inventory.	0.20.3	SheetJS / XLSX
The local asset name and version property.	5.65.16	CodeMirror
The local asset header.	2.30.1	Moment.js
The local asset header.	2.0.0	printThis
The local asset and the approved release inventory.	3.0.2	normalize.css
The local asset and the approved release inventory.	2.0.4	Skeleton
Present locally; the version number was not sufficiently established from the directly inspected source.	—	Select2
Present locally; the version number was not sufficiently established from the directly inspected source.	—	Moment Hijri
package-lock.	7.0.0	Font Awesome npm
Local assets exist in more than one path; do not automatically assume they equal the npm version.	—	Font Awesome local assets

Third-Party Components and Licenses

Third-party components and dependencies used within release scope, including associated licenses, were reviewed. No known observations preventing their use in the delivered release appeared within the current review scope, subject to each component's license terms. The SBOM retains the detailed component and dependency inventory. This is a technical review within release scope, not legal advice or a declaration that components are unrestricted.

Reference SBOM Files

Purpose	Format	File
NuGet component inventory.	CycloneDX JSON	SmartFoundation-NuGet-SBOM.json
npm component inventory.	CycloneDX JSON	SmartFoundation-npm-SBOM.json

Purpose	Format	File
SHA-256 checksum manifest for verifying the integrity of both inventory files.	CSV	SmartFoundation-SBOM-SHA256.csv

Current Authentication, Authorization, and Sessions

The application uses Cookie Authentication with Claims after successful credential validation through DATACORE. It also retains operational user context in Session. The application registers Authentication and Authorization, then applies SessionGuardMiddleware to compare Cookie/Claims identity with Session and close access when context is missing or mismatched.

The relevant middleware order in current code is:

Routing -> Session -> Authentication -> Authorization -> SessionGuardMiddleware -> Endpoints

The historical state without registered Cookie Authentication, Claims, or SessionGuard is preserved in the gap register but does not represent the Current State of release 1.0.0.

Session Architecture and Scaling

The current operational model is a Single Application Server. The application uses AddDistributedMemoryCache and AddSession, but actual Session storage is process-local/in-memory, which is suitable for the current approved model. This is not automatically classified as an open security vulnerability.

When moving to multiple servers, instances, Load Balancer, or High Availability, reevaluate Distributed Session Storage, shared Data Protection keys, Session Affinity when needed, Load Balancing, Shared Storage for required files, Health Checks, and Centralized Monitoring. Current code contains no evidence that this multi-server design is implemented.

Content Security Policy

CSP is implemented and active within the application. Some current UI components, including the Alpine.js pattern used, depend on compatibility settings required for operation. The current decision records this as an accepted architectural compatibility decision within the current architecture, not as an open vulnerability by itself.

Review the policy again when Alpine.js or UI libraries change, JavaScript is redesigned, a stricter policy becomes feasible without breaking functions, or new security evidence appears.

Application Logging

The application uses ILogger, operational logs, and database-related AuditLog and ErrorLog according to path. Required operational controls are: do not log passwords, Session data, or unnecessary sensitive information; protect log files from modification and unauthorized access; apply Log Rotation and Retention according to organizational policy; synchronize time; make logs available to authorized investigators; and monitor recurring patterns and errors.

Application Logging is a shared responsibility between application and operations. SIEM or Centralized Logging and centralized collection of IIS, Windows, SQL, and application logs are hosting-infrastructure capabilities not proven by application code; they depend on the hosting entity's decision.

Database and SQL Server

DATACORE remains the documented operational database name, and live DATACORE was the source of truth in previous documented reconciliations. The reference environment used SQL Server 2019 for development and verification. SQL Server 2019 Developer Edition is not a production requirement; production uses a supported, appropriately licensed SQL Server Edition according to hosting-entity policy.

The documented reference-environment connection uses Windows Integrated Security. The document does not display the Connection String value or service identity.

Backup and Recovery

The operational model includes Full Backup, Differential Backup, Transaction Log Backup, and Restore Drill. Target operational objectives are RPO = 15 minutes and RTO = 1 hour. These are targets, not automatically achieved guarantees.

Achievement depends on Production Backup Policy, Transaction Log Backup schedule, storage capacity/location, monitoring and alerting, backup-chain integrity, periodic restore testing, and hosting-environment characteristics. FULL Recovery Model is documented for the reference environment; production suitability must be verified against Log Backup schedule and approved objectives.

IIS Hosting Considerations

Current Application Architecture

The application is ASP.NET Core MVC on .NET 8 and configured to run behind IIS/ASP.NET Core Module. Source files alone do not prove configuration of a particular production server.

Hosting Recommendations

- A supported Windows Server and IIS with ASP.NET Core Hosting Bundle for .NET 8.
- A dedicated Application Pool and dedicated least-privileged service identity.
- HTTPS, a trusted TLS certificate, and approved Firewall and Binding settings.
- File permissions limited to necessary runtime, upload, and log paths.
- Enable IIS Logs with appropriate Retention and prevent Detailed Errors from reaching production users.
- Protect settings and secrets outside source and monitor certificate and resource validity.

Post-Deployment Acceptance

The hosting team verifies site startup, DATACORE connectivity, HTTPS/TLS, authentication, Session, permissions, critical functions, IIS/Application Logs, initial backup and restore, monitoring, and alerting. Document readiness must not be recorded as Final Production Approval.

22. Terms and Abbreviations

Table 5: Terms and Abbreviations

Standardized Definition	Term
The live SQL Server database and the definitive source for SQL definitions in this documentation.	DATACORE
The reference SmartFoundation.Database project; used for comparison and understanding intent, but it does not establish live state.	Snapshot
The web layer containing Program, Controllers, Views, and the actual runtime entry point.	MVC
Shared presentation components that interpret SmartPageViewModel and build tables and forms.	UI / SmartRenderer
An active application gateway service for reads and CRUD in the Housing pattern.	MastersServices
A read gateway Stored Procedure that routes by pageName_.	Masters_DataLoad
A write gateway Stored Procedure that routes by pageName_ and ActionType and applies shared checks.	Masters_CRUD
A container of result sets; the first table is usually for permissions, and subsequent tables contain page data and DDL data.	DataSet
Selection lists and reference data that populate fields.	DDL
The administrative context that scopes many data sets and operations.	Idara
End-to-End testing from the user interface to the database and back.	E2E
A structural or functional difference between live DATACORE and the Snapshot.	Schema drift
Recovery Point and Recovery Time Objectives in business continuity.	RPO/RTO
A standards-based inventory of components and dependencies; it is not an independent vulnerability test.	SBOM
Static code scanning, component analysis, and dynamic scanning within the defined verification scopes.	SAST / SCA / DAST
Interactive Application Security Testing that was NOT PERFORMED within the development team's scope for release 1.0.0.	IAST